ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIEURS DE MONASTIR

HONORIS UNITED UNIVERSITIES

National Engineering Degree

2025 / 2026

Major : Computer Science

Software Engineering And Data Science

Enterprise Knowledge Intelligence Platform Based On Artificial Intelligence

Realized by : Adam Farhat

Academic Supervisor : Jacem Mhenni

Industry Supervisor : Mohamed Aziz Rezgui

Dedication

I dedicate this work to my parents, whose unconditional love, support, and sacrifices have been the foundation of all my achievements. Their encouragement has been a constant source of strength throughout my academic journey.

To my family and friends, I express my sincere gratitude for their continuous support, patience, and understanding during both the difficult and rewarding moments of this work.

I also dedicate this project to my own efforts and perseverance, which allowed me to overcome challenges and stay committed to my goals.

Finally, I extend my deepest appreciation to my professors, whose guidance, expertise, and dedication have greatly contributed to my learning and to the successful completion of this project.

Acknowledgements

I would like to express my sincere gratitude to everyone who contributed to the success of this final-year engineering project.

My thanks go first to my industrial supervisor at HyprCraft Solutions, for providing a meaningful problem statement, regular feedback, and access to the organisational context that shaped the requirements of the Knowledge Platform.

I am equally grateful to my academic supervisor at ESPRIM, Jacem Mhenni, for methodological guidance, critical review of the technical choices, and support in aligning the deliverable with engineering-degree expectations.

I also thank the examination jury for their time and constructive evaluation, as well as the ESPRIM faculty who prepared us for this capstone experience.

Finally, I acknowledge my colleagues and peers whose discussions, testing, and code reviews improved the quality of the implemented solution.

Abstract

Organisations produce large volumes of internal documentation, from policies and technical guides to reports and day-to-day procedures, yet employees often struggle to find reliable information quickly. Knowledge tends to be scattered across different storage systems, search stays mostly keyword-based, and access rules are hard to apply in a consistent way. Together these constraints slow down decisions and push people back towards informal channels.

This report presents the **Knowledge Platform**, an enterprise web application that centralises document management and adds intelligent search and question-answering grounded in the organisation's own content. It brings together a modern full-stack web architecture, hybrid retrieval, and department-scoped access control, so that users receive answers backed by traceable sources instead of unsupported generated text.

The main contribution is support for a complete knowledge lifecycle. Documents are ingested securely, indexed semantically, and retrieved through a hybrid approach that combines meaning-based and keyword matching. Answers stream back with citations, conversations are kept for later reference, administrators govern the platform, and user feedback makes it possible to monitor answer quality over time. Three organisational roles are supported, namely Administrator, Manager, and Employee, with permissions aligned to the department structure.

The work was carried out at HyprCraft Solutions following an Agile Scrum process spread over five development sprints. Validation relied on automated integration testing, repeatable deployment packaging, manual browser scenarios, and a dedicated evaluation workflow for the retrieval-augmented answers. The results show that the platform meets its functional and non-functional targets while staying open to further improvement in a real organisational setting.

Keywords: Retrieval-Augmented Generation, Enterprise Search, Vector Database, Access Control, Full-Stack Web Application, Large Language Models

Contents

List of Figures

List of Tables

List of abbreviations and acronyms

Acronym	Definition
AI	Artificial Intelligence
ABAC	Attribute-Based Access Control
API	Application Programming Interface
BM25	Best Matching 25 (lexical ranking function)
BullMQ	Redis-backed job queue library used for document ingest
CORP	Cross-Origin Resource Policy (browser security header)
CSP	Content Security Policy
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DTO	Data Transfer Object
FR	Functional Requirement
HNSW	Hierarchical Navigable Small World (approximate nearest-neighbor index)
HTTP	Hypertext Transfer Protocol
HyDE	Hypothetical Document Embeddings (retrieval augmentation technique)
IDE	Integrated Development Environment
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
MIME	Multipurpose Internet Mail Extensions (file type identification)
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping (Prisma)
PFE	Final-Year Project (Projet de Fin d'Études)
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
SSE	Server-Sent Events
TTL	Time To Live (cache expiration)
UML	Unified Modelling Language

General introduction

Recent years have accelerated digital transformation inside organisations and, with it, the volume of internal material that staff must rely on: technical reports, procedures, contracts, operational guidelines, and similar documents. That material underpins everyday decisions, yet it is often spread across several systems, shared drives, and communication channels, so finding the right piece at the right time becomes slow and unreliable.

The practical consequence is familiar in engineering settings. People hunt through folders, ask colleagues informally, or rebuild knowledge that already exists. Time is lost, answers diverge, and productivity suffers, precisely where accurate information matters most.

Storage alone is therefore no longer enough. Organisations need systems that organise knowledge, honour the structure of teams and departments when granting access, and make retrieval efficient. Artificial intelligence adds a second opportunity: natural-language interfaces that let users ask questions of the corpus instead of browsing it manually.

This final-year project was carried out during an internship at HyprCraft Solutions. Its aim is to design and implement a platform that centralises organisational documents and improves how people reach information through intelligent search and interaction.

The problem can be stated as follows: *How can an engineering organisation structure its internal knowledge, ensure secure and role-based access to information, and enable efficient retrieval of reliable answers from its documentation?*

In response, the project pursues the objectives listed below.

1. design a secure platform supporting multiple user roles with controlled access to organisational documents;

2. improve information retrieval from heterogeneous document sources;
3. enable more efficient access to knowledge through intelligent assistance mechanisms;
4. ensure system reliability through testing and validation approaches;
5. provide a reproducible and deployable software solution.

The scope covers a full-stack web application for document management and knowledge access, including authentication, document organisation, and intelligent retrieval, with access structured by roles and departments. Mobile applications, large-scale distributed infrastructure, and external enterprise identity providers fall outside this work.

The remainder of the report is organised as follows. Chapter 1 presents the state of the art and situates the project. Chapter 2 covers requirements and methodology. Chapter 3 describes design and architecture. Chapter 4 reports on implementation. Chapter 5 presents testing and validation. The conclusion then summarises the contributions and outlines perspectives.

CHAPTER 1

State of the Art

This chapter surveys the scientific and industrial landscape around enterprise knowledge management, information retrieval, and grounded answer generation over organisational documents. It compares existing approaches and situates the Knowledge Platform in that landscape. Requirements, implementation, and validation belong to later chapters and are left aside here.

1.1 Scientific and Technical Context

1.1.1 The Enterprise Knowledge Lifecycle Problem

Modern organisations generate a steady stream of heterogeneous digital assets: technical documentation, procedures, contracts, internal reports, spreadsheets, emails, and operational guidelines. At the same time, those assets are scattered across shared drives, email, messaging tools, and departmental repositories that rarely talk to one another.

The problem is therefore broader than search alone. Knowledge is hard to collect consistently, hard to centralise across tools and departments, hard to organise by ownership and business logic, hard to govern with access policies, and hard to retrieve in a way that respects context. Employees often fall back on informal channels or personal memory instead of consulting documented material. Surveys of knowledge work still report that a substantial share of working time goes into looking for information rather than using it [1].

1.1.2 Limitations of Traditional Document Systems

Tools such as Google Drive, OneDrive, and Microsoft SharePoint were built mainly for storage and collaboration, not for intelligent knowledge management. They are dependable for keeping files and applying basic access control, but they offer little semantic understanding of content. Search remains largely keyword-oriented, structuring is weak, information is frequently duplicated, and the systems do not answer questions in natural language. Permission models are also often too coarse to reflect how departments actually share knowledge. In practice, these platforms behave as passive repositories rather than active knowledge systems.

1.2 Information Retrieval: From Keywords to Semantics

1.2.1 Classical Retrieval Models

Classical information retrieval ranks documents with lexical methods such as TF-IDF and BM25 [2], which look at term frequency and inverse document frequency. They work well when the query and the document use the same wording. They break down whenever two phrases mean the same thing but share little vocabulary, which is common in enterprise writing.

1.2.2 Dense Retrieval and Semantic Search

Dense retrieval addresses that gap by encoding queries and documents as continuous vectors so that conceptually related texts can match even with low lexical overlap [3, 4]. The approach is attractive for paraphrase-heavy corpora, yet it still underperforms on rare or highly technical terms, depends heavily on the quality of the embedding model, and makes ranking decisions harder to explain.

1.2.3 Hybrid Retrieval Systems

Most recent enterprise search stacks therefore combine lexical and semantic signals [5,6]. Ranking fusion methods such as Reciprocal Rank Fusion (RRF) merge the two result lists so that exact keyword hits and meaning-based matches reinforce each other. That hybrid pattern is now the usual baseline for production search.

1.3 Retrieval-Augmented Generation (RAG)

1.3.1 Concept and Architecture

Retrieval-Augmented Generation (RAG) [7] couples retrieval with a Large Language Model. Relevant passages are fetched first and then supplied as context for generating an answer, instead of returning only a list of documents. A typical pipeline starts with a retriever over document chunks, optionally refines the shortlist with a reranker, and finishes with a generator that produces a grounded natural-language response.

1.3.2 Advantages of RAG

Relative to a standalone LLM, RAG grounds replies in external documents, reduces unsupported invention by constraining generation to retrieved material, lets knowledge be updated without retraining the model, and improves traceability when answers carry citations.

1.3.3 Limitations in Enterprise Context

Those gains come with new obligations. Answer quality is tightly bound to how well retrieval, chunking, and indexing are designed. Systematic evaluation is still difficult, native access control is rarely built into open RAG toolkits, and moving a prototype into a governed production environment remains costly.

1.3.4 Evaluation of RAG Systems

Classical IR metrics do not fully capture whether an answer is faithful or useful. Recent work therefore proposes LLM-based evaluation frameworks that score faithfulness, relevance, and completeness [8], which helps to spot hallucinations and regressions once a system is in production.

1.4 Access Control and Knowledge Governance

1.4.1 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) [9,10] remains the usual way to manage permissions in enterprise systems: users receive roles, and roles carry permissions over resources. Administration stays tractable because rights are edited at the level of a small, stable set of roles rather than for every individual. Expressiveness is the weak point. When access depends on context, for example which department owns a resource, RBAC either ignores that nuance or multiplies narrowly specialised roles until the scheme becomes hard to maintain (a phenomenon often called role explosion).

1.4.2 Attribute-Based Access Control (ABAC)

Attribute-Based Access Control (ABAC) [11] decides access by evaluating policies over attributes of the subject (department, seniority), the resource (owner, visibility, classification), and sometimes the environment (time, location). Rules become boolean conditions over those attributes, which fits contextual, data-dependent constraints much better than pure roles.

The price is operational. A general ABAC deployment needs a policy engine, careful attribute management, and policies that are harder to audit and test. Mistakes can silently open or close paths. For a platform of this size, a fully general ABAC stack would add more complexity than the problem warrants.

1.4.3 Choosing a Model: A Hybrid RBAC/ABAC Approach

RBAC and ABAC answer different needs: roles keep administration simple and auditable, while attributes add the missing contextual detail. The decisive requirement here is department-scoped document and knowledge access, which is effectively RBAC enriched with a small set of attributes (department membership and document visibility).

The platform therefore keeps roles as the coarse backbone and layers those attributes on top, rather than introducing a general policy engine. That hybrid choice preserves the clarity of role management while still filtering retrieval and answers by department. The concrete design (roles, department and visibility attributes, and per-user restriction flags enforced down to the retrieval layer) is spelled out in Chapter 2 (Subsection 2.2.3).

1.4.4 Challenges in Knowledge Systems

In a knowledge platform, access control cannot stop at page-level visibility. It must also decide which documents enter search results, which sources may influence generated answers, how department boundaries are respected, and how access and usage remain auditable. In particular, filters have to run inside the retrieval pipeline; otherwise a generated answer can leak content that the user was never meant to see.

1.5 Enterprise Knowledge Systems and Existing Solutions

1.5.1 Shared Drives and Document Management Systems

Google Drive, OneDrive, and Microsoft SharePoint [12] remain the everyday foundation for storage and collaboration. They offer reliable synchronisation, basic access control, and broad adoption. They do not, however, provide semantic search, do not answer questions from content, structure knowledge weakly, and often organise files in folders that do not mirror how information actually flows across teams.

1.5.2 Vector Databases and RAG Frameworks

Libraries and services such as Pinecone, Weaviate, LangChain, and LlamaIndex [13–15] give developers building blocks for semantic retrieval and modular RAG pipelines. They scale well and leave room for architectural choices, but they are not complete products: there is no end-user application, no document lifecycle, and little built-in governance. Turning them into a production system still demands substantial engineering.

1.5.3 Commercial AI Copilots

Microsoft Copilot and ChatGPT Enterprise [16] expose conversational interfaces over enterprise data, with strong ecosystem ties and a low barrier for users. Transparency is limited: retrieval and ranking internals are hard to inspect, embedding strategies cannot be freely retuned, vendor lock-in is real, and access-control logic can rarely be customised in depth.

1.5.4 Knowledge Base Platforms

Dedicated knowledge base products focus on curated internal documentation rather than search over large unstructured corpora. **Guru** [17] acts as an enterprise wiki and AI search layer that surfaces verified "Cards" inside tools people already use (browser extension, Slack, Microsoft Teams) and keeps content fresh through an expert verification workflow. **Tettra** [18] is a Slack-centred internal wiki that pairs Q&A with ownership assignment and stale-page detection. Both now add generative assistants on top of that curated base. Their strength is organised, validated content and team-oriented sharing. Their weakness is the dependence on manual authoring: they rarely offer automated ingestion, AI retrieval over raw corpora is limited, and they struggle once the content is large and unstructured.

1.5.5 Enterprise AI Search Systems (Glean)

Glean [19] is among the most complete commercial attempts at unified enterprise search, and more recently at a broader "Work AI" assistant. Built by engineers with a

strong search background, it connects to a large set of workplace applications (Google Workspace, Microsoft 365, Slack, Jira, Confluence, Salesforce, and others), builds a company-wide knowledge graph of documents and people, and answers natural-language questions with inline citations. Retrieval is permission-aware: results respect each source system's access controls so that users only see what they are already entitled to [20]. An API and connector framework also support custom agents over the same corpus [20]. Architecturally, though, Glean stays a proprietary hosted SaaS. The ranking pipeline is opaque, self-hosting and deep customisation are unavailable, and feedback-driven optimisation of the stack remains limited for the customer organisation.

1.6 Comparative Analysis

To position the proposed platform within the current landscape of enterprise knowledge management solutions, a comparative analysis of representative systems is presented. The comparison highlights their main strengths, limitations, and relevance to this project, thereby identifying the gap addressed by the proposed Knowledge Platform.

Table 1.1 − Comparative analysis of enterprise knowledge management solutions

Solution	**Strengths**	**Limitations**	**Relevance to this work**
Shared Drives (Drive, SharePoint)	Reliable storage and basic access control	No semantic search, no QA, weak structuring	Baseline storage layer only
Vector DB + RAG frameworks	Strong retrieval and modular pipelines	No full system or governance layer	Core retrieval component only
Commercial Copilots	Ease of use and enterprise integration	Black-box systems, limited control	Demonstrates demand but lacks transparency
Knowledge Base Tools	Structured documentation workflows	Manual maintenance, no scalable ingestion	Not suitable for large corpora
Glean	Unified enterprise AI search with citations	Closed system, no customisation or transparency	Closest industrial solution but not open
Proposed System	Hybrid RAG + RBAC + full lifecycle + feedback loop	External LLM dependency	Full enterprise knowledge lifecycle solution

1.7 Project Positioning

Taken together, the review points to a clear gap: no widely available solution simultaneously covers document lifecycle management, centralised governance, semantic retrieval, and controlled AI-based question answering inside one transparent architecture.

The Knowledge Platform is designed to close that gap. It centralises ingestion and organisation of documents, applies role-based and department-scoped access control, retrieves content with both lexical and semantic methods, generates answers with source grounding through RAG, and keeps a feedback path so that answer quality can be monitored. Unlike tools that solve only one slice of the problem (storage, retrieval toolkit, or closed SaaS assistant), the contribution here is an integrated lifecycle that can be inspected, deployed, and extended.

1.7.1 Key Contribution

In short, the work aims to keep knowledge collectable and centralised, access governed and auditable, retrieval hybrid and context-aware, answers grounded and traceable, and the whole stack reproducible enough for production use.

1.8 Chapter Conclusion

The state-of-the-art chapter has set out why traditional document platforms are insufficient and why hybrid retrieval with retrieval-augmented generation, under enterprise access rules, is a more credible direction. The comparison of existing products motivated an integrated platform that ties lifecycle management, controlled access, semantic search, and cited AI assistance together. Chapter 2 turns these observations into functional and non-functional requirements and introduces the Agile Scrum delivery process used thereafter.

CHAPTER 2

Requirements and Methodology

This chapter turns the gaps identified in Chapter 1 into a verifiable specification for the Knowledge Platform. After analysing the host organisation, it defines functional and non-functional requirements with acceptance criteria, then sets out the Agile Scrum approach and five-sprint plan that frame the design (Chapter 3), implementation (Chapter 4), and validation (Chapter 5) work that follow.

2.1 Analysis of the Existing Situation and Usage Context

2.1.1 Host Organisation Context

HyprCraft Solutions is a technology company whose teams produce and consume large volumes of internal documentation: technical specifications, onboarding guides, project reports, API references, and operational procedures. Prior to this project, knowledge was scattered across disconnected tools such as shared drives, email threads, and informal messaging channels, with no unified governance model and no mechanism for intelligent retrieval.

Several pain points stood out when the existing situation was analysed. Documents lived in multiple systems with no single point of access, so a comprehensive search was impossible. There was no systematic control over which teams or roles could reach which documents, which tended to produce either over-sharing or information silos. Retrieval

was limited to filename and keyword matching and returned documents rather than answers. New employees had no reliable way to find answers to operational questions and therefore interrupted experienced colleagues repeatedly. When information found through search proved incorrect or outdated, there was no way to signal that fact or to stop the same error from recurring.

2.1.2 System Actors

Four actor groups interact with the Knowledge Platform, as summarised in Table 2.1 and detailed in the global use-case diagram (Subsection 2.1.3, Figure 2.1):

Table 2.1 – System actors and their primary capabilities

Actor	**Primary capabilities**
Employee	Browse and search the document library, perform semantic search and RAG question-answering, manage personal profile, maintain conversation history, and rate assistant answers.
Manager	All Employee capabilities, plus access to the manager dashboard showing managed departments and their members, and the ability to send announcements to managed departments.
Administrator	Full platform control: user lifecycle management, department CRUD and merge, per-user restrictions, bulk operations, audit log access, KPI dashboards, activity exports, and platform-wide notifications.
System	Autonomous actor: runs the background document ingestion pipeline (extract, chunk, embed), emits automatic notifications on document and role events, and executes the RAG evaluation harness.

2.1.3 Global Use-Case Model

Figure 2.1 presents the global use-case diagram of the Knowledge Platform. It illustrates the main actors interacting with the system and the principal functionalities offered, providing an overall view of the platform's behaviour and scope.

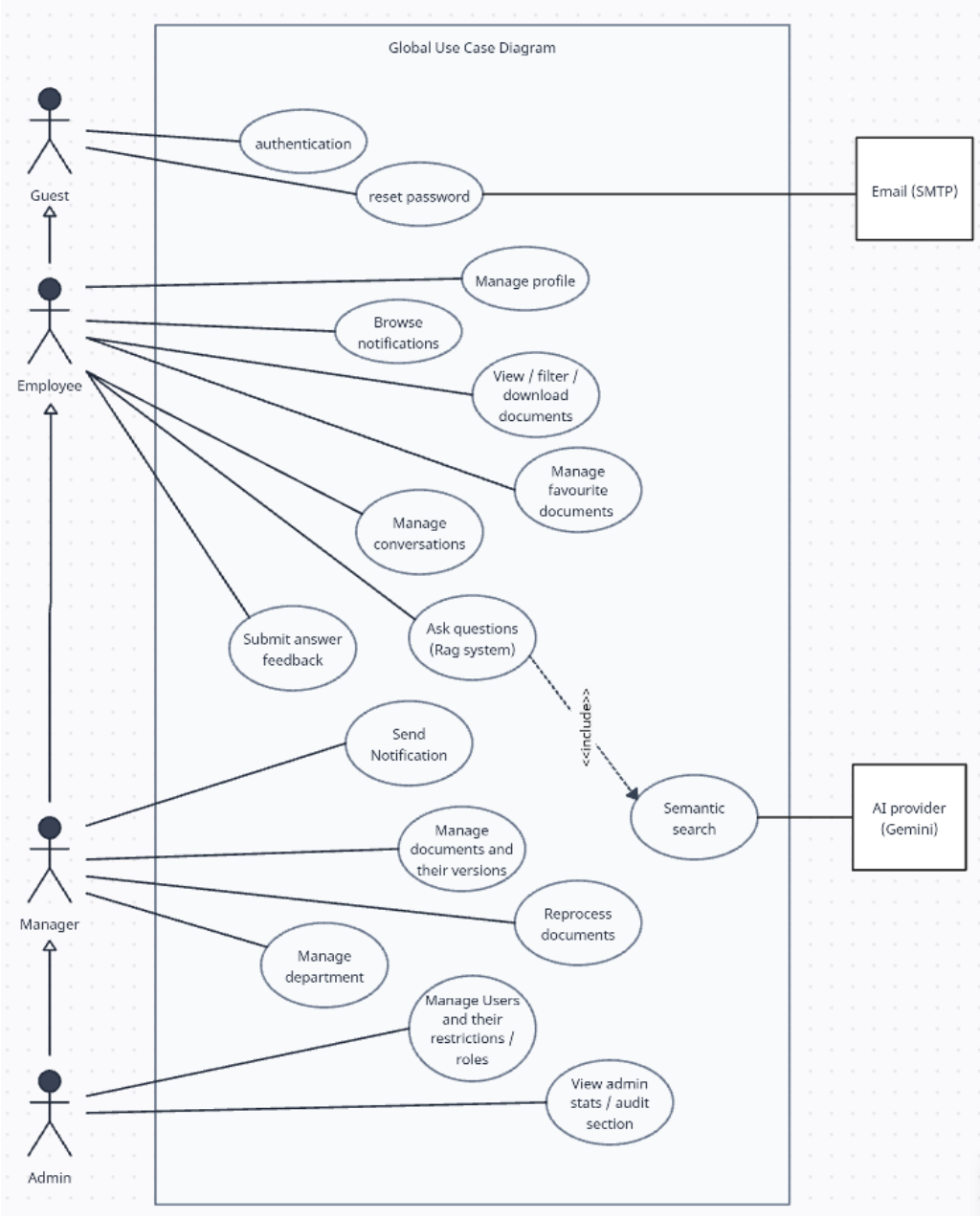

Figure 2.1 – Global use-case diagram of the Knowledge Platform

2.2 Functional and Non-Functional Requirements

The requirements grew out of an analysis of user workflows and of the intended system architecture, so that stakeholder needs and the design choices in later chapters stay aligned.

2.2.1 Functional Requirements

Functional requirements are organised around **authentication and identity** (FR-01 to FR-06), **document management** (FR-07 to FR-16), **search and AI capabilities** (FR-17 to FR-21), **notifications** (FR-22 to FR-25), and **governance and administration** (FR-26 to FR-36). Access to every capability is governed by a hybrid authorization model that combines Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), and per-user restriction flags (see Subsection 2.2.3).

Table 2.2 – Functional requirements

ID	Requirement	Description
FR-01	User Authentication	The system shall authenticate registered users before granting access to protected resources.
FR-02	Session Management	The system shall maintain authenticated user sessions and allow users to securely log in and out of the platform.
FR-03	Profile Management	Users shall be able to view and update their personal profile information.
FR-04	Avatar Management	Users shall be able to upload, replace, and remove their profile picture.
FR-05	Password Management	Users shall be able to change their password and recover access through a password reset process.
FR-06	Feature Restrictions	The system shall enforce feature-specific restrictions according to user permissions and assigned privileges.

ID	Requirement	Description
FR-07	Document Upload	Authorised users shall be able to upload documents in supported formats for storage and processing.
FR-08	Document Access Control	The system shall ensure that users can access only the documents they are authorised to view.
FR-09	Document Organisation	Users shall be able to organise documents using metadata such as tags, favourites, and archive status.
FR-10	Document Versioning	The system shall maintain multiple versions of uploaded documents while preserving their version history.
FR-11	Document Reprocessing	Authorised users shall be able to request the reprocessing of previously uploaded documents without re-uploading them.
FR-12	Document Viewing and Download	Authorised users shall be able to view document details, preview supported files, and download document versions.
FR-13	Document Audit Trail	The system shall record significant document activities to ensure traceability and accountability.
FR-14	Document Search and Filtering	Users shall be able to search and filter documents using metadata and access permissions.
FR-15	Administrative Document Management	Administrators shall be able to manage all documents across the platform.
FR-16	Document Export	Administrators shall be able to export document information for reporting and analysis purposes.
FR-17	Semantic Search	Users shall be able to perform semantic searches across accessible enterprise documents.
FR-18	AI Question Answering	The system shall answer natural language questions using enterprise documents and provide relevant supporting references.

ID	Requirement	Description
FR-19	Conversation Management	Users shall be able to create, rename, view, and delete conversation histories.
FR-20	AI Response Feedback	Users shall be able to provide positive or negative feedback on AI-generated responses and optionally include comments.
FR-21	Feedback Analytics	Administrators shall be able to view aggregated statistics on AI response quality and user feedback.
FR-22	Automatic Notifications	The system shall automatically notify users of relevant platform events according to their roles and departments.
FR-23	Manual Announcements	Administrators and managers shall be able to publish announcements to selected users, roles, or departments.
FR-24	Notification Attachments	Announcements may include file attachments that authorised recipients can access.
FR-25	Notification Inbox	Users shall be able to access, read, and manage their personal notifications.
FR-26	Manager Dashboard	Managers shall be able to monitor the departments under their responsibility and view department members.
FR-27	User Management	Administrators shall be able to create, update, deactivate, restore, lock, unlock, and manage user accounts.
FR-28	Department Management	Administrators shall be able to create, update, and delete departments within the platform.
FR-29	Department Merge	Administrators shall be able to merge departments while preserving associated users, documents, and related information.
FR-30	Department Access Management	Administrators shall be able to assign users to departments and manage their access levels and responsibilities.

ID	Requirement	Description
FR-31	Platform Statistics	Administrators shall be able to monitor platform statistics and key performance indicators.
FR-32	Audit Logs	Administrators shall be able to view and export authentication and document activity logs.
FR-33	System Health Monitoring	Administrators shall be able to monitor the operational status of platform services and dependencies.
FR-34	Bulk User Management	Administrators shall be able to perform bulk administrative operations on multiple user accounts.
FR-35	Bulk Document Management	Administrators shall be able to perform bulk operations on multiple documents.
FR-36	Data Import and Export	Administrators shall be able to import and export platform data where appropriate to support administration and reporting.

Story-level acceptance criteria for these requirements appear in the sprint backlogs (Tables 2.6–2.10). Chapter 3 maps selected requirements to design elements and validation activities (Table 3.1).

2.2.2 Non-Functional Requirements

Table 2.3 – Non-functional requirements

ID	Requirement	Acceptance criterion
NFR-01	Health monitoring endpoint	Health check returns success when the database and cache are reachable; otherwise returns service-unavailable with per-component detail
NFR-02	Security headers and rate limiting	Security headers enforced on all responses; login limited to 15 requests per 15 minutes per address; Ask limited to 12 requests per minute per address
NFR-03	Single-flight refresh token (web client)	Parallel requests from the same browser tab share a single in-flight refresh operation rather than issuing duplicate refresh calls
NFR-04	Docker Compose reproducibility	Container orchestration starts all four application services in a healthy state; database migrations run automatically
NFR-05	Integration test coverage	Full integration test suite passes on a live database and cache instance with seeded data
NFR-06	RAG evaluation harness	RAG evaluation workflow completes without error and produces a structured quality report

2.2.3 Access Control Model

Building on the RBAC concepts introduced in Chapter 1, the platform enforces a **hybrid authorization model** at three complementary layers. The subsections below explain how those foundations are applied to the requirements already defined.

Under **Role-Based Access Control (RBAC)**, every user holds one platform role (`ADMIN`, `MANAGER`, or `EMPLOYEE`) that determines coarse-grained capabilities. Administrators manage users, departments, and platform configuration (FR-27, FR-28, FR-30

to FR-32, FR-34 to FR-36); managers reach department-scoped dashboards and announcements (FR-26, FR-23); employees use the document library, search, and AI assistant within their permitted scope.

Attribute-Based Access Control (ABAC) further constrains permissions by attributes such as department membership and document visibility (`ALL`, `DEPARTMENT`, `PRIVATE`). Retrieval and RAG pipelines apply these filters before returning results, so that semantic search and generated answers never expose unauthorised content (FR-08, FR-17, FR-18).

Per-user feature restriction flags let administrators disable specific capabilities for individual users (for example, blocking document-library access or AI queries) via middleware that intercepts requests before business logic runs (FR-06).

Taken together, the three layers give fine-grained control over platform capabilities while remaining practical to scale across authentication, document management, search, and AI workflows.

2.3 Specifications and Performance Indicators

The Key Performance Indicators (KPIs) below link requirements to the validation chapter (Chapter 5):

Table 2.4 – Key performance indicators

KPI	**Measurement method**	**Target**
Ingest reliability	Percentage of uploaded documents reaching ready state without failure	$\geq 95\%$
Integration test pass rate	Integration test suite pass ratio (passed / total)	74 / 74 (100%)
RAG topic classification (no API key)	Quick evaluation optimiser pass rate	$\geq$ 15/34 (degraded mode)
RAG judge scores (with API key)	Faithfulness and relevance from automated judge	Documented in eval report
Security: refresh replay	Reuse of rotated token triggers family revocation	Verified by auth test suite
Security: avatar CORP	Cross-origin resource policy header present on avatar responses	Verified by avatars test

2.4 Selected Methodology

2.4.1 Agile Scrum Framework

The project used the **Agile Scrum** methodology for its iterative structure, its focus on a working deliverable at the end of each sprint, and its ability to absorb changing requirements. That mix suits a complex full-stack product built by a single engineer over a bounded internship period.

2.4.1.1 Scrum Roles

Mohamed Aziz Rezgui (HyprCraft Solutions) acted as **Product Owner**, validating feature priorities and acceptance criteria. Adam Farhatserved as both **Scrum Master** and developer, owning sprint planning, execution, daily self-review, and retrospective.

Jacem Mhenni (ESPRIM) provided methodological and scientific guidance at sprint boundaries as Academic Advisor.

2.4.1.2 Scrum Artefacts

The **Product Backlog** holds the full set of functional and non-functional requirements (FR-01 to FR-36, NFR-01 to NFR-06), ordered by business value and technical dependency. Each **Sprint Backlog** is the subset of those items committed for a given sprint, broken into implementation tasks. An **Increment** is a working, tested vertical slice of the platform delivered at sprint end. Under the **Definition of Done**, a requirement counts as done only when (a) the feature is implemented, (b) at least one integration test covers the happy path and one error path, and (c) the corresponding capability has been manually checked against its acceptance criteria.

2.4.1.3 Sprint Structure

Each sprint followed a fixed rhythm. **Sprint Planning** reviewed the backlog, selected items, decomposed them into tasks, and estimated effort. **Development** then implemented features, wrote integration tests, and revieweded code against security and style guidelines. At **Sprint Review** the working increment was demonstrated and checked against acceptance criteria. The **Sprint Retrospective** closed the cycle by recording what worked, what blocked progress, and what should change next time.

2.4.2 Sprint Plan

Delivery was organised as **five sprints**, each producing a coherent, independently testable platform capability. Detailed interaction design for each sprint appears in Section 3.5 of Chapter 3.

Table 2.5 – Agile Scrum sprint plan

Spr.	Theme	Deliverables	Req.	Des.
S1	Auth., users, RBAC	JWT login/logout/refresh (HttpOnly cookies), refresh-token family revocation, password policy, session cap, per-user restrictions, profile/avatar, RBAC middleware	FR-01 to FR-06 NFR-02, NFR-03	3.5.1
S2	Document library & ingest	Multi-format upload (PDF, Office, HTML, XML, ZIP), BullMQ worker (extract→chunk→embed→persist), versioning, tags, favourites, recents, archive, visibility, audit log	FR-07 to FR-14	3.5.2
S3	Hybrid RAG & Ask	Query optimiser, hybrid retrieval (pgvector + BM25 + RRF), LLM reranking, neighbour expansion, SSE answers with citations, confidence scoring, critique, claim verification, follow-ups, feedback loop, conversations	FR-17 to FR-21 NFR-06	3.5.3
S4	Admin, manager, notifications	Admin CRUD (users, departments, merge, bulk ops, CSV), dept-access management, document administration and export, KPI dashboard, activity/audit exports, manager view, auto/manual notifications	FR-15, FR-16, FR-22 to FR-32, FR-34 to FR-36	3.5.4
S5	Tests, eval & deploy	74 Vitest/Supertest integration tests, RAG eval CLI (automated judge), Docker Compose prod config, migration pipeline, 54 UML diagrams, README	FR-33 NFR-01, NFR-04, NFR-05, NFR-06	3.5.5

The **Des.** column points to the sprint interaction-design subsections of Section 3.5 (Chapter 3).

2.4.2.1 Sprint Timeline

Figure 2.2 illustrates the sprint sequence defined in Table 2.5 and the iterative delivery of working increments across the project duration. Each sprint builds on the previous one: authentication and access control (S1), document corpus (S2), hybrid RAG (S3), governance (S4), then operational hardening and validation (S5).

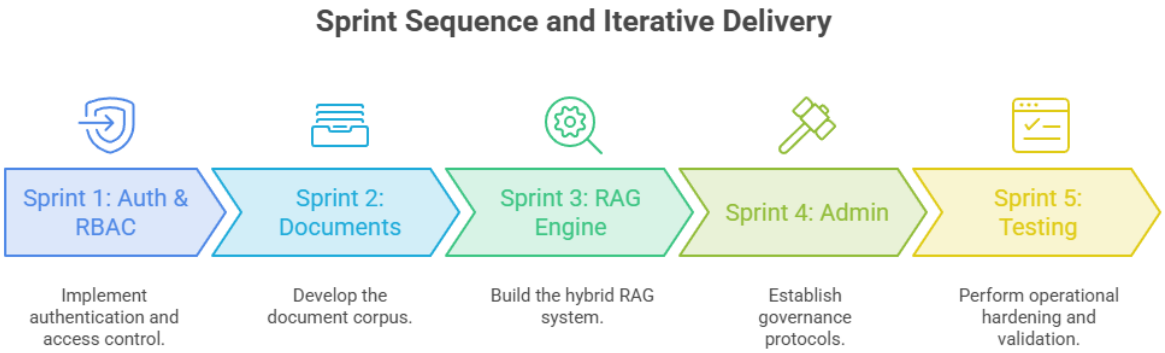

Figure 2.2 – Scrum sprint timeline — iterative delivery across five sprints

2.4.2.2 Sprint Backlogs

Tables 2.6–2.10 detail each sprint backlog at user-story level. Every story is linked to one or more requirements from Table 2.2 (or Table 2.3 for non-functional items) so that the expanded FR-01 to FR-36 catalogue remains traceable through planning, design, and validation. Interaction flows for every story are documented in Section 3.5.

Each story carries two planning attributes: a story-point estimate (**SP**) and a **priority**. Story points are a *relative*, unitless measure of the effort, complexity, and uncertainty of a story rather than a duration in hours; they are estimated collaboratively (planning-poker style) on a Fibonacci-like scale (2, 3, 5, 8, 13), where larger values indicate greater size or technical risk. This relative scale makes estimates more consistent than absolute time predictions and, once summed per sprint, gives an indication of the team's capacity (velocity). Priority (**High** or **Medium** here) reflects how important and how urgent a story is, and is chosen from three factors: business value for end users, technical dependencies (stories that unblock later work are ranked higher), and risk. Foundational stories such as authentication and access control are therefore rated High, since subsequent sprints build directly upon them.

Sprint 1 — Authentication and access control.

Table 2.6 – Sprint 1 backlog: authentication and access control

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S1-01	FR-01	As a guest, I want to sign in securely so that I can access authorised platform features.	5	High	Valid credentials issue an access token and secure refresh cookie; invalid credentials are rejected; login rate limiting is enforced.
US-S1-02	FR-02	As an authenticated user, I want seamless session continuity so that refresh operations are secure and race-condition free.	8	High	Session refresh rotates the token family atomically; replay of a rotated token revokes the family; parallel tab requests share a single refresh operation.
US-S1-03	FR-02	As an authenticated user, I want to sign out so that active refresh sessions are revoked.	2	Med.	Logout revokes the current refresh session; subsequent refresh attempts fail.
US-S1-04	FR-03	As a user, I want to view and update my profile so that my account information remains accurate.	3	High	Profile field updates persist and are returned on subsequent session loads.
US-S1-05	FR-04	As a user, I want to upload, replace, or remove my profile picture so that my identity is visible in the interface.	3	High	Avatar upload succeeds; replacement and removal work; avatars are served with cross-origin headers for display in the web client.

continued on next page

continued from previous page

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S1-06	FR-05	As a user, I want to change my password and recover access by email so that I can regain or update credentials safely.	5	High	Signed-in password change requires the current password and revokes other sessions; forgotten-password flow sends a time-limited reset link and closes existing sessions after success.
US-S1-07	FR-06	As an administrator, I want role-based and per-user access restrictions so that capabilities can be controlled precisely.	8	High	Role boundaries enforce administrator, manager, and employee capabilities; restriction flags block protected features and redirect to a restricted notice page.
US-S1-08	NFR-02, NFR-03	As a security owner, I want baseline HTTP protections and safe session refresh so that authentication endpoints resist abuse.	3	Med.	Security headers are active globally; login and Ask endpoints are rate-limited; parallel browser requests share a single in-flight refresh.

Sprint 2 — Document library and ingest.

Table 2.7 – Sprint 2 backlog: document library and ingest

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S2-01	FR-07	As an employee, I want to upload documents in common formats so that they enter the shared library for processing.	8	High	Upload validates file type and size, stores the file, creates a version record, and queues ingestion.
US-S2-02	FR-07	As the system, I want to ingest uploaded files asynchronously so that text is searchable and embeddable.	13	High	Background worker extracts text, chunks content, computes embeddings, and persists passages; version reaches ready state on completion.
US-S2-03	FR-08	As a security owner, I want visibility rules enforced so that unauthorised users cannot discover restricted documents.	8	High	Organisation-wide, department, and private visibility levels are enforced; unauthorised access returns 404 rather than revealing existence.
US-S2-04	FR-09	As an employee, I want to organise documents with tags, favourites, and archive status so that I can manage my working set.	5	High	Tag updates, favourite toggle, and archive/unarchive persist per document within the user's scope.
US-S2-05	FR-09	As an employee, I want recently viewed documents so that I can return to files I opened earlier.	2	Med.	Recent-view tracking persists per user; library home surfaces the recent list.

continued on next page

continued from previous page

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S2-06	FR-10	As a document owner, I want document versioning so that updates preserve prior file history.	5	High	New version upload creates a version record, queues ingestion, and keeps earlier versions available.
US-S2-07	FR-11	As an authorised user, I want to reprocess an existing version so that embedding upgrades do not require re-upload.	5	Med.	Reprocess triggers re-ingestion of an existing version and updates processing status to ready on success.
US-S2-08	FR-12	As an employee, I want to open document details, preview supported files, and download versions so that I can use the content.	5	High	Detail view returns metadata and active version; preview works for supported types; download delivers file content after access checks.
US-S2-09	FR-13	As a manager, I want a document audit trail so that significant changes remain traceable.	5	Med.	Title, tags, visibility, and version changes are recorded in a per-document audit log.
US-S2-10	FR-14	As an employee, I want to search and filter the library by metadata so that I can find authorised documents quickly.	5	High	Listing supports filters, pagination, and permission-aware results without loading full text content.

Sprint 3 — Search, RAG and conversations.

Table 2.8 – Sprint 3 backlog: search, RAG and conversations

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S3-01	FR-17	As an employee, I want semantic search over the corpus so that I can find relevant passages by meaning.	8	High	Query is embedded and ranked against department-scoped passages; results include snippet previews.
US-S3-02	FR-18	As an employee, I want cited answers streamed in real time so that I receive grounded responses to natural-language questions.	13	High	Hybrid retrieval and reranking produce an answer stream with source citations delivered before answer tokens and a completion signal.
US-S3-03	FR-19	As an employee, I want persistent conversation threads so that I can create, rename, review, and delete prior questions and answers.	8	High	Conversations support create, list, rename, and delete; opening a thread loads its full message history.
US-S3-04	FR-20	As an employee, I want to rate assistant answers so that poor responses can be identified.	5	High	Thumbs up/down feedback with optional comment is stored against the answer message.
US-S3-05	FR-21	As an administrator, I want feedback analytics so that I can monitor AI answer quality across the organisation.	5	Med.	Administrator statistics aggregate feedback volume, satisfaction ratio, and recent comments.

continued on next page

continued from previous page

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S3-06	NFR-06	As a data scientist, I want a reproducible RAG evaluation harness so that retrieval quality can be measured over time.	8	Med.	Quick and full evaluation modes produce structured quality reports without disrupting continuous integration.

Sprint 4 — Governance and notifications.

Table 2.9 – Sprint 4 backlog: governance and notifications

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S4-01	FR-22	As an employee, I want automatic notifications on document events so that I stay informed of library changes.	5	High	Document create, update, and delete events generate department-scoped inbox entries.
US-S4-02	FR-23	As a manager or admin, I want to send manual announcements so that I can communicate with targeted teams.	5	High	Manual send validates audience scope (users, roles, or departments) before delivery.
US-S4-03	FR-24	As a manager or admin, I want announcement attachments so that recipients can download related files.	3	Med.	Optional attachments are stored with the announcement and downloadable by authorised recipients.

continued on next page

continued from previous page

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S4-04	FR-25	As an employee, I want a notification inbox so that I can read and manage my messages.	5	High	Inbox supports listing, unread count, mark-read, and mark-all-read; web client displays an unread badge.
US-S4-05	FR-26	As a manager, I want a department dashboard so that I can oversee teams I manage.	8	High	Dashboard returns only departments within the manager's scope; member lists are correctly filtered.
US-S4-06	FR-27	As an administrator, I want full user-account administration so that I can create, update, lock, unlock, deactivate, and restore accounts.	8	High	User lifecycle operations are supported; last-administrator safety is enforced.
US-S4-07	FR-28	As an administrator, I want to create, update, and delete departments so that the platform reflects organisational structure.	5	High	Department CRUD persists unique department identifiers and is recorded for audit.
US-S4-08	FR-29	As an administrator, I want to merge departments so that users and documents move safely into a target unit.	8	High	Merge preserves associated users, documents, and access relationships without data loss.

continued on next page

continued from previous page

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S4-09	FR-30	As an administrator, I want to assign department access so that users receive the correct responsibilities.	5	High	Single and bulk department-access grants and revocations update the access matrix correctly.
US-S4-10	FR-31	As an administrator, I want platform statistics so that I can monitor key performance indicators.	5	Med.	KPI dashboard returns user, document, conversation, and indexing indicators.
US-S4-11	FR-32	As an administrator, I want authentication and document activity logs so that governance reviews are exportable.	5	Med.	Activity and document audit views support filtering and spreadsheet export.
US-S4-12	FR-15	As an administrator, I want to manage all documents across the platform so that governance covers every library item.	5	High	Administrators can list, inspect, and manage documents beyond a single department scope.
US-S4-13	FR-16	As an administrator, I want to export document catalogue data so that reporting and reconciliation are supported.	3	Med.	Document inventory export includes titles, departments, tags, and processing status.
US-S4-14	FR-34	As an administrator, I want bulk user operations so that onboarding and access reviews scale efficiently.	5	Med.	Bulk user updates are applied atomically where specified; spreadsheet imports sanitise formulas.

continued on next page

continued from previous page

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S4-15	FR-35	As an administrator, I want bulk document operations so that large clean-up tasks can be performed safely.	5	Med.	Bulk delete and related multi-document actions require confirmation and remove related records.
US-S4-16	FR-36	As an administrator, I want import and export of platform data so that administration and reporting stay portable.	5	Med.	Supported administrative import/export paths complete with validation errors reported clearly.

Sprint 5 — Operations, testing and deployment.

Table 2.10 – Sprint 5 backlog: operations, testing and deployment

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S5-01	FR-33, NFR-01	As an operator, I want a health endpoint so that I can verify database and queue connectivity.	3	High	Health check returns success when database and cache are reachable; otherwise returns service-unavailable with per-component detail.
US-S5-02	NFR-05	As a developer, I want automated integration tests so that regressions are caught before release.	13	High	Integration suite runs 74 cases across ten domains on live database and cache; target is 100% pass rate.

continued on next page

continued from previous page

ID	Req.	User Story	SP	Pri.	Acceptance Criteria
US-S5-03	NFR-04	As an operator, I want reproducible containerised deployment so that the platform can be started consistently.	8	High	Container orchestration starts database, cache, API, and web services in a healthy state; migrations run automatically.
US-S5-04	NFR-06	As a data scientist, I want automated RAG judge evaluation so that answer quality can be tracked with an API key.	8	Med.	Full evaluation produces faithfulness and relevance scores; quick mode runs without an external API key.
US-S5-05	—	As a maintainer, I want complete design documentation so that the system is defensible and maintainable.	5	Med.	Fifty-four UML diagrams are exported; architecture guide is updated; sequence flows are catalogued in Chapter 3.

2.4.3 Development and Modelling Tools

Table 2.11 – Development and modelling tools

Tool	Usage in project
Prisma Schema DSL	Domain modelling and 29 sequential migrations
PlantUML	54 UML diagrams (3 structural, 5 sprint use-case, 5 sprint class, 41 sequence)
Vitest + Supertest	Integration testing across all API domains
Docker Compose	Reproducible dev and production environment
GitHub Actions	CI pipeline (lint, audit, typecheck, integration tests)
Zod	Runtime request validation for all API inputs

2.5 Chapter Conclusion

This chapter set out a verifiable specification for the Knowledge Platform: functional and non-functional requirements, measurable acceptance criteria, and key performance indicators. It also formalised delivery through a five-sprint Scrum plan with detailed backlogs, each increment tied to design, implementation, and validation goals in the chapters ahead. Chapter 3 presents the resulting system design, including sprint-aligned interaction diagrams in Section 3.5.

CHAPTER 3

Design and Architecture

This chapter turns the requirements and sprint plan of Chapter 2 into a concrete system design. Two complementary views organise the presentation:

— **Static design** (Sections 3.1–3.4) — three-tier platform architecture, technology justification, global domain class model, and subsystem responsibilities;

— **Dynamic design** (Section 3.5) — sprint-scoped use-case and domain class models, followed by interaction sequences for every major API and UX flow.

Implementation details and validation evidence follow in Chapters 4 and 5.

3.1 Three-tier Platform Architecture

The Knowledge Platform uses a three-tier logical architecture: a presentation layer, an application layer, and a data and artificial intelligence layer. Keeping the user interface, business logic, and persistence in separate components makes the system easier to maintain, secure, and evolve.

At the presentation layer, users work through a responsive web application that covers document management, semantic search, conversational AI, notifications, and administration. Authentication uses short-lived access tokens held in memory and refresh tokens stored as HttpOnly cookies, which limits exposure of sensitive credentials.

The application layer is a RESTful API that coordinates business operations. It exposes services for authentication, document management, semantic search, conversational AI, notifications, administration, and manager-specific work. Beyond handling client requests, this layer enforces authorisation policies, checks permissions, orchestrates document processing, and talks to storage and AI services.

The data and AI layer stores documents, metadata, vector embeddings, user accounts, notifications, and conversation history. After each upload or version update, an asynchronous ingestion pipeline extracts text, splits it into semantic chunks, generates embeddings, and indexes them for retrieval.

That work runs in the background, so the interactive API stays responsive while newly uploaded documents become searchable once processing finishes.

Figure 3.1 summarises this layout. The diagram is intended to show how the web client, API services, background ingest worker, and data stores relate across the three tiers, and where Retrieval-Augmented Generation (RAG) depends on indexed document content. It establishes the structural baseline used throughout the remainder of this chapter.

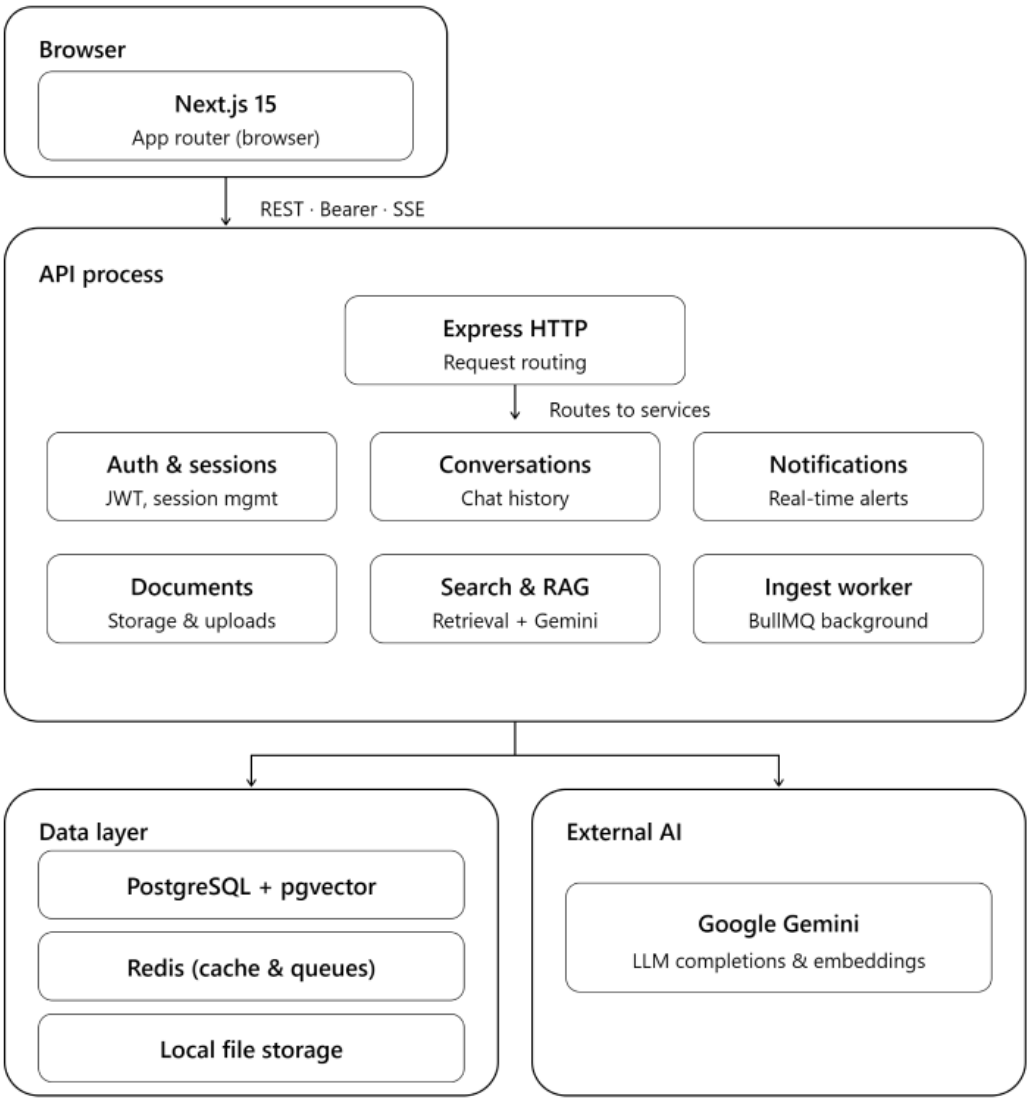

Figure 3.1 – Overall three-tier architecture of the Knowledge Platform

3.1.1 Requirements Traceability

To ensure that this overall architecture satisfies the requirements identified during the analysis phase, Table 3.1 establishes the relationship between representative functional and non-functional requirements, the corresponding design decisions, and the validation activities presented in Chapters 4 and 5.

Table 3.1 – Requirements-to-design traceability (excerpt)

Req.	**Design choice**	**Implemented element**	**Validation**
FR-01	Short-lived tokens with refresh rotation	Authentication API and web session client	Integration tests for sign-in and session renewal
FR-04	Profile images with CORP isolation	Public avatar delivery endpoint	Integration tests for avatar access
FR-06	Per-user feature restriction flags	Authorization middleware	Integration tests for restricted access
FR-07	Asynchronous document ingest	Background worker with Redis-backed job queue	Integration tests for upload and processing
FR-08	Department and visibility filtering	Access control layer in document and search paths	Integration tests for unauthorised access
FR-17	Semantic vector retrieval	Hybrid search service over department-scoped corpus	Integration tests for semantic search
FR-18	Hybrid retrieval with streamed answers	Search and AI completion services	Integration tests for Ask with citations
FR-22	Event-driven notification delivery	Notification service with department scoping	Integration tests for automatic notifications
FR-26	Manager department overview	Manager workspace API	Integration tests for manager dashboard
FR-27	Role-based user administration	Administration service layer	Integration tests for governance endpoints
FR-33	Operational health probes	Public health endpoint	Integration tests for service readiness
NFR-02	HTTP hardening and rate limiting	API security middleware stack	Manual review and integration tests

3.2 Technology Choice Justification

The implementation of the Knowledge Platform required selecting technologies that satisfy the functional and non-functional requirements identified in Chapter 2 while remaining appropriate for the project's scope. The selected technology stack prioritises development efficiency, maintainability, scalability, and smooth integration with AI-powered document retrieval. The following subsections justify the main architectural and technology decisions adopted throughout the project.

3.2.1 Selection of the Application Architecture

Several architectural approaches were considered during the design phase. Table 3.2 compares the principal alternatives using evaluation criteria relevant to the project, including development time, streaming capabilities, team expertise, and operational complexity.

Table 3.2 – Decision-support matrix for platform architecture (scores: 1 = weak, 3 = strong)

Criterion	**Monorepo**	**Django**	**Microservices**	**Rationale**
Time to market	3	2	1	Shared TypeScript types; one repository
SSE streaming fit	3	2	2	Native Express streaming for RAG
Team skill alignment	3	2	1	Matches React + Node stack
Operational complexity	3	3	1	Single API process + worker

Based on this evaluation, a TypeScript monorepo composed of a Next.js frontend and an Express backend was selected. This architecture provides a unified development environment, facilitates code sharing, and naturally supports Server-Sent Events (SSE), which are required for real-time AI response streaming. The chosen stack also aligns

with the team's previous experience, which reduces implementation risk within a fixed internship schedule.

Language and framework choices follow from that architecture. **TypeScript** is used across the API and the web application so that domain types, validation contracts, and shared utilities stay consistent in one repository. **Next.js** [21] implements the presentation layer with the App Router and a mix of server and client rendering suited to dashboards, document browsing, and the Ask workspace. **Express** [22] hosts the REST API and streams RAG answers over SSE without introducing a second backend runtime. **Prisma** [23] provides the ORM, schema definitions, and sequential migrations that keep the relational model under version control. Together these choices match the title of this section: they are not incidental libraries, but the concrete application stack that realises the three-tier design of Section 3.1.

3.2.2 Database and Retrieval Technology

PostgreSQL enhanced with the pgvector extension was selected as the primary persistence layer [24]. Relational metadata (users, departments, documents, conversations, notifications) and vector embeddings then coexist in a single database, which simplifies transaction management and supports hybrid retrieval that combines semantic similarity with SQL filters such as department scope and visibility. Dedicated vector databases such as Pinecone, Weaviate, or Milvus [13] would specialise embedding search further, but would also add another service to operate, synchronise, and secure. For the expected corpus size and deployment model of this project, PostgreSQL with pgvector cuts infrastructure complexity while remaining sufficient for hybrid search and RAG.

3.2.3 AI Service Selection

Google Gemini was adopted as the external AI provider for embedding generation, optional reranking, and answer synthesis [25]. Managed AI services remove the operational burden of hosting and maintaining large language models on internship hardware, while still offering strong multilingual quality suited to enterprise document understanding. Self-hosting open-weight models was considered but rejected for this scope: it would have required GPU capacity, model packaging, and continuous maintenance that would

divert effort from the platform lifecycle (ingest, access control, hybrid retrieval, and governance) that forms the core of the work.

3.2.4 Background Processing Strategy

Document ingestion is performed asynchronously through a Redis-backed BullMQ queue [26, 27]. Parsing documents, generating chunks, and creating embeddings are computationally intensive; moving them off the request path keeps interactive API calls responsive. Running the BullMQ consumer in the same API process simplifies deployment for a prototype-scale platform (one process to start, monitor, and containerise). The design remains extensible: the queue abstraction allows a dedicated worker service later if volume or isolation needs grow, without rewriting the ingest pipeline itself.

3.3 Global Domain Class Model

The persisted domain model represents the core entities of the Knowledge Platform and the relationships between them. It provides a structural view of the system by modelling users and roles, departments and access rights, documents and versions, conversation threads and feedback, and notifications.

Figure 3.2 presents the corresponding global UML class diagram. The figure is meant to make the main entity groups and their associations visible in one place, so that later sprint-scoped class diagrams can be read as focused extracts of this same model rather than as unrelated schemas.

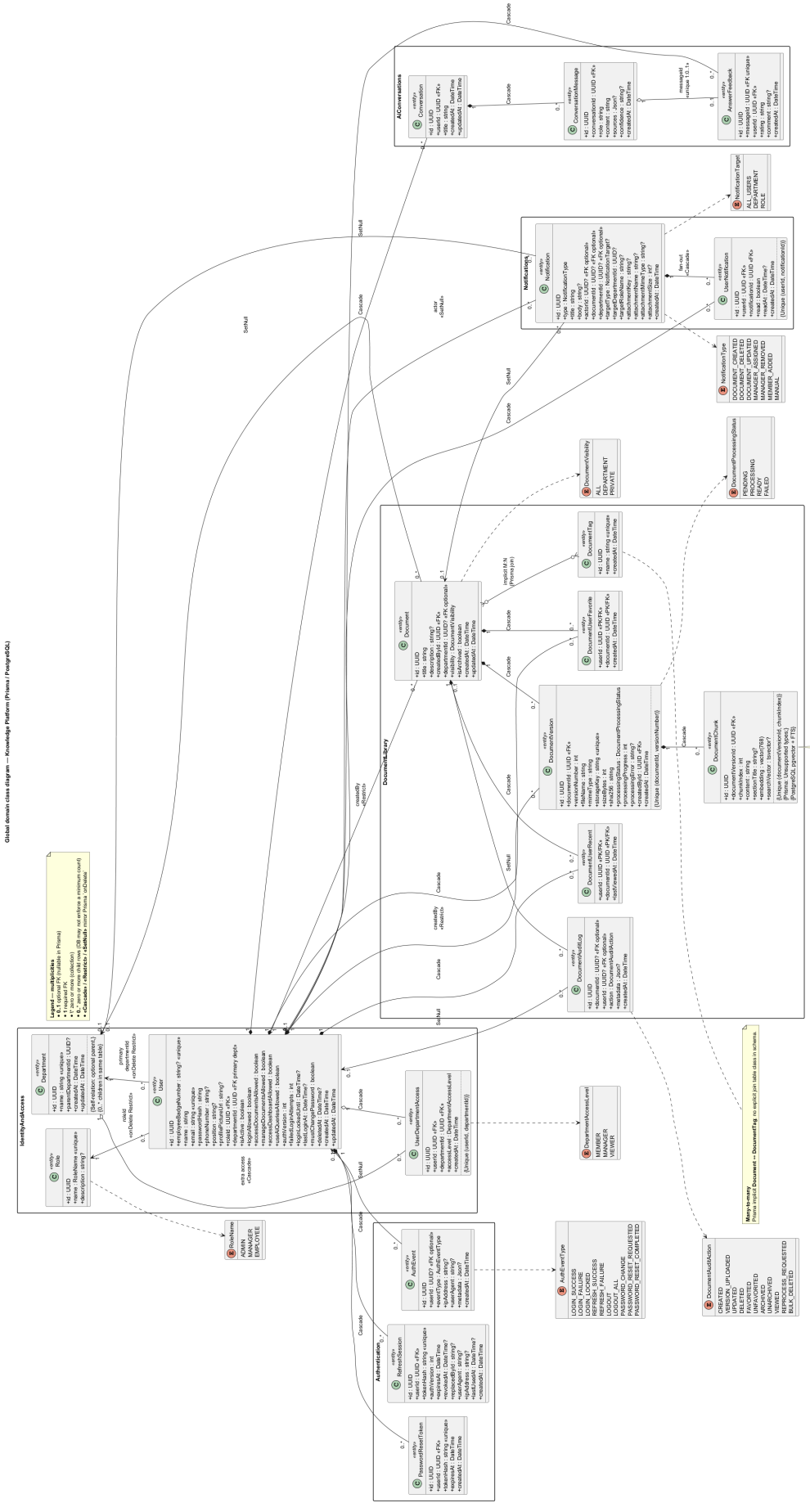

Figure 3.2 – Global domain class diagram (persisted entities and main relationships)

3.4 Logical Subsystem Decomposition

To improve modularity and maintainability, the Knowledge Platform is decomposed into several logical subsystems, each responsible for a specific set of functionalities. This separation of responsibilities promotes clear boundaries between core business services while facilitating independent development, testing, and future extension. Table 3.3 summarises the main subsystems and their respective responsibilities.

Table 3.3 – Subsystem responsibilities

Subsystem	**Responsibility**	**Key components**
Authentication	Login, refresh families, restrictions, password reset	Authentication API Web session client
Documents	CRUD, versions, visibility, audit, download	Document management service File storage layer
Ingest worker	Extract, chunk, embed, persist chunks	Background ingest worker Indexing pipeline
Search / RAG	Hybrid retrieval, SSE answers, cache	Hybrid search service AI answer streaming
Conversations	Threads, messages, feedback	Conversation management service
Notifications	Auto events and manual announcements	Notification service
Admin / Manager	Governance dashboards	Administration API Manager workspace API

3.5 Sprint-scoped Dynamic Design

Sections 3.1–3.4 describe **what** the platform is made of. This section describes **how** people, services, and background processes interact during real use. For each development sprint in Table 2.5 of Chapter 2, the design is presented in three steps:

1. **Sprint scope** — the capabilities delivered in that iteration;
2. **Sprint structural models** — the sprint use-case diagram and domain class diagram for that iteration;
3. **Interaction sequences** — step-by-step flows between the user interface, application services, background workers, and external systems.

The full diagram catalogue is presented in Section 3.5; implementation details appear in Chapter 4 and validation in Chapter 5.

3.5.1 Sprint 1: Authentication and access control

Sprint 1 defines how people enter and move through the platform. It covers sign-in and sign-out, password recovery, profile management, and the rules that limit access when an account or feature is restricted.

3.5.1.1 Sprint 1 structural models: authentication and access

Sprint 1 use-case diagram.

Figure 3.3 bounds the functional scope of Sprint 1. It shows the Guest–Employee–Manager–Admin role hierarchy and the use cases for authentication, password reset, profile management, department member visibility, and feature-restriction enforcement, with Email (SMTP) as the external system for reset delivery. The aim is to make permission inheritance and the authentication boundary explicit before the detailed sequence flows.

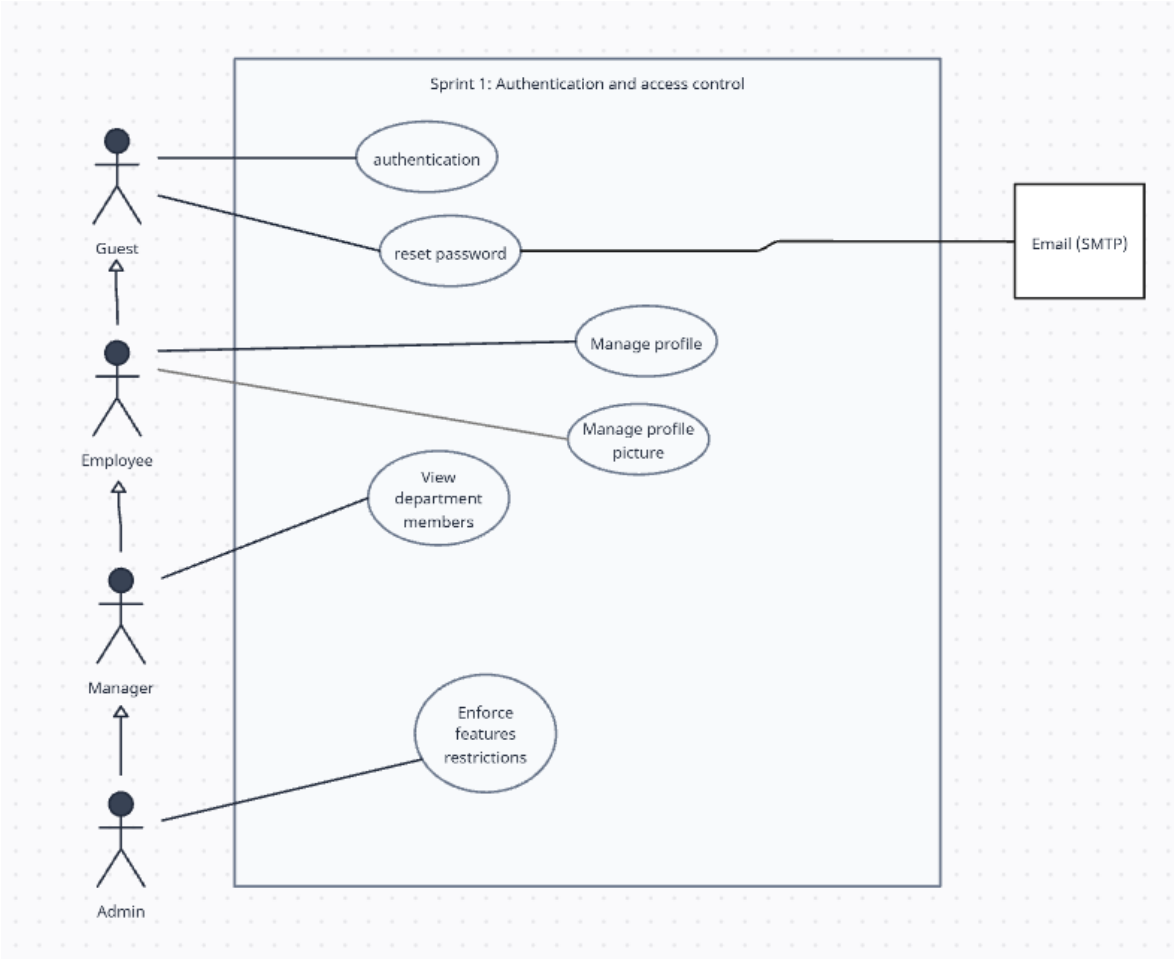

Figure 3.3 – Sprint 1 use-case diagram — Authentication and access control

Sprint 1 domain class diagram.

Figure 3.4 focuses on the persisted entities required for identity and access: user accounts, sessions or refresh families, roles, feature restrictions, and profile-related attributes. It is intended to show which domain objects underwrite the Sprint 1 use cases without repeating the full global class model.

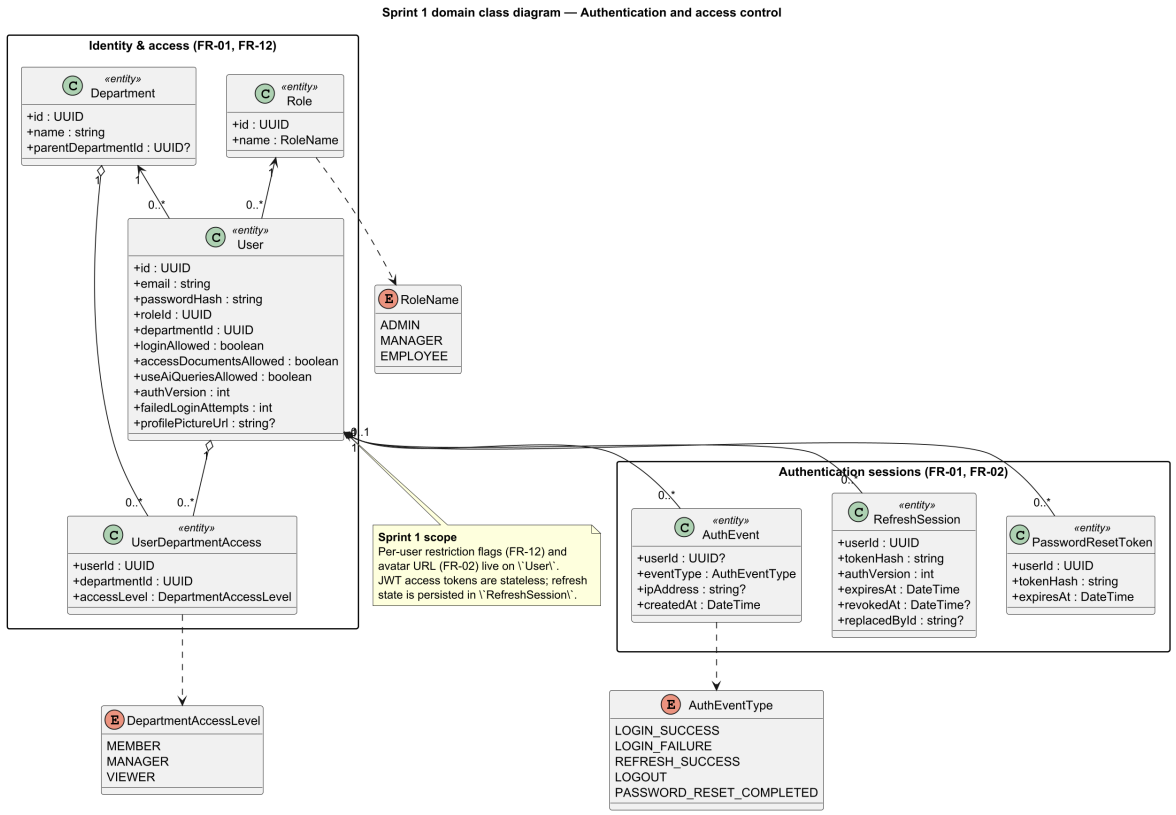

Figure 3.4 – Sprint 1 domain class diagram — Authentication and access control

3.5.1.2 Sprint 1 interaction sequences: session and profile flows

Signing in and managing the session

User sign-in — sequence diagram.

A user enters email and password; the platform verifies the credentials, applies account lockout after repeated failures, and opens a secure session with a short-lived access token and a protected refresh cookie.

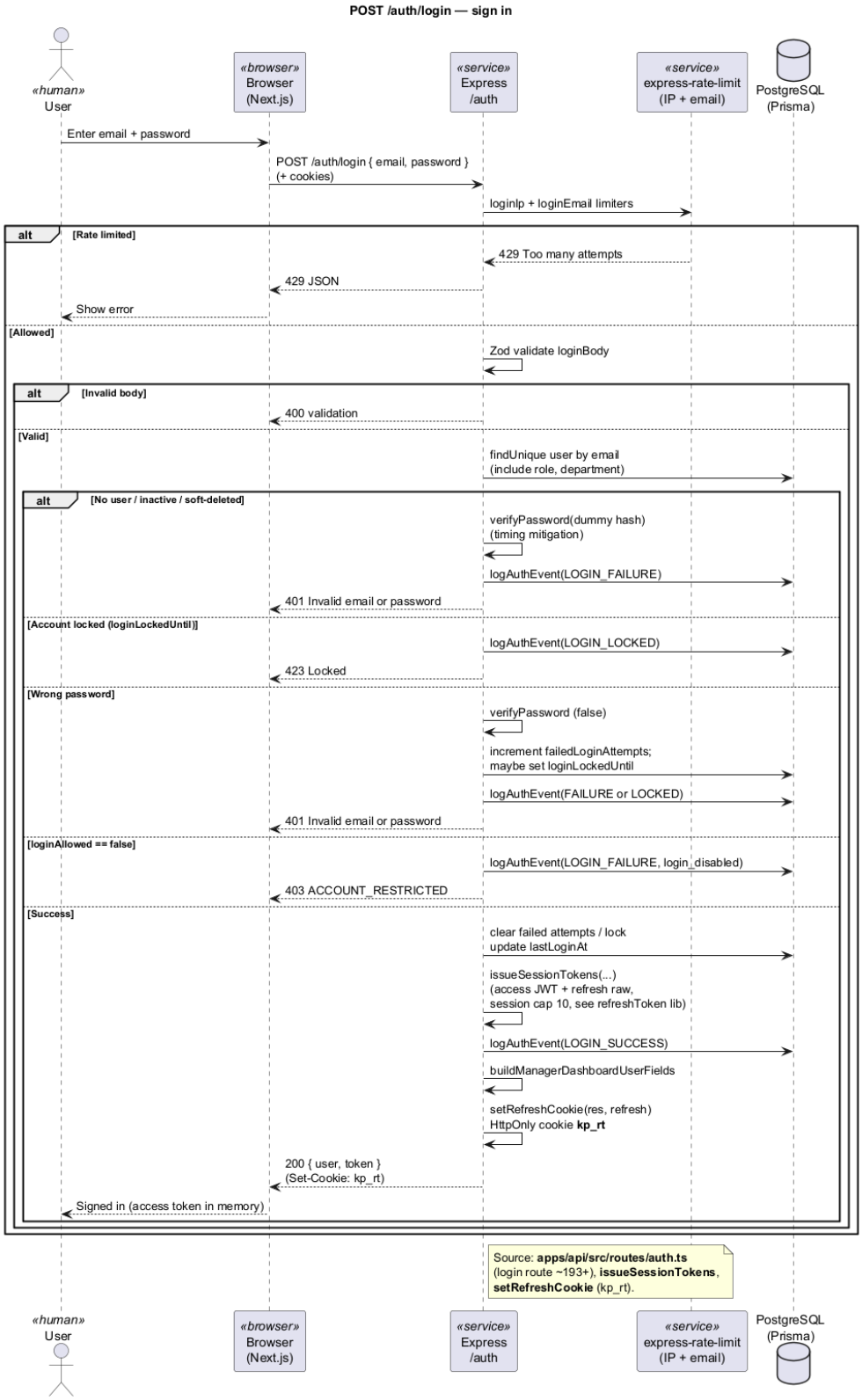

Figure 3.5 – User sign-in — sequence diagram

User sign-out — sequence diagram.

Signing out ends the active session on the server and clears the refresh cookie from the browser, so the account cannot be reused from that device without a new sign-in.

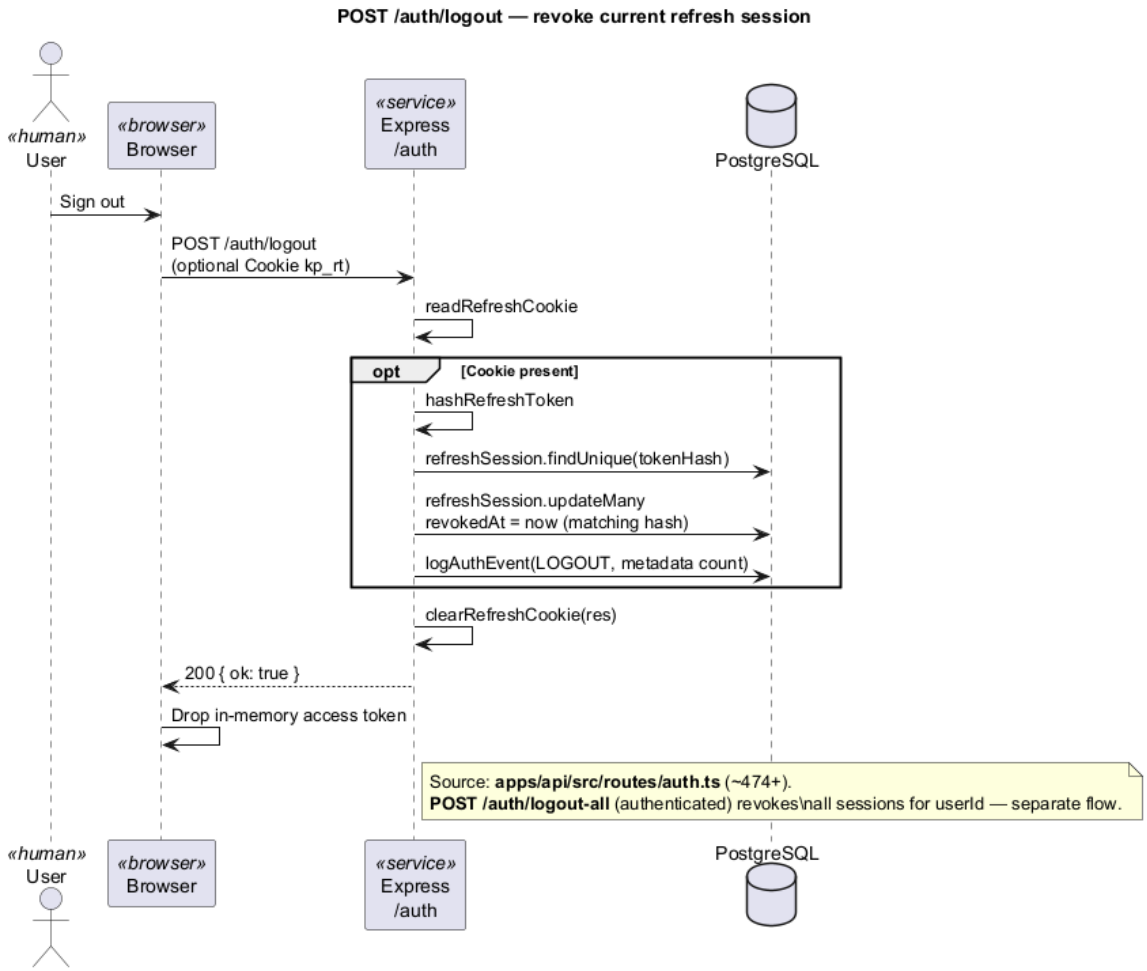

Figure 3.6 – User sign-out — sequence diagram

Forgotten password recovery — sequence diagram.

A user who forgot their password requests a reset link by email, then sets a new password through a time-limited link. Existing sessions are closed once the password changes.

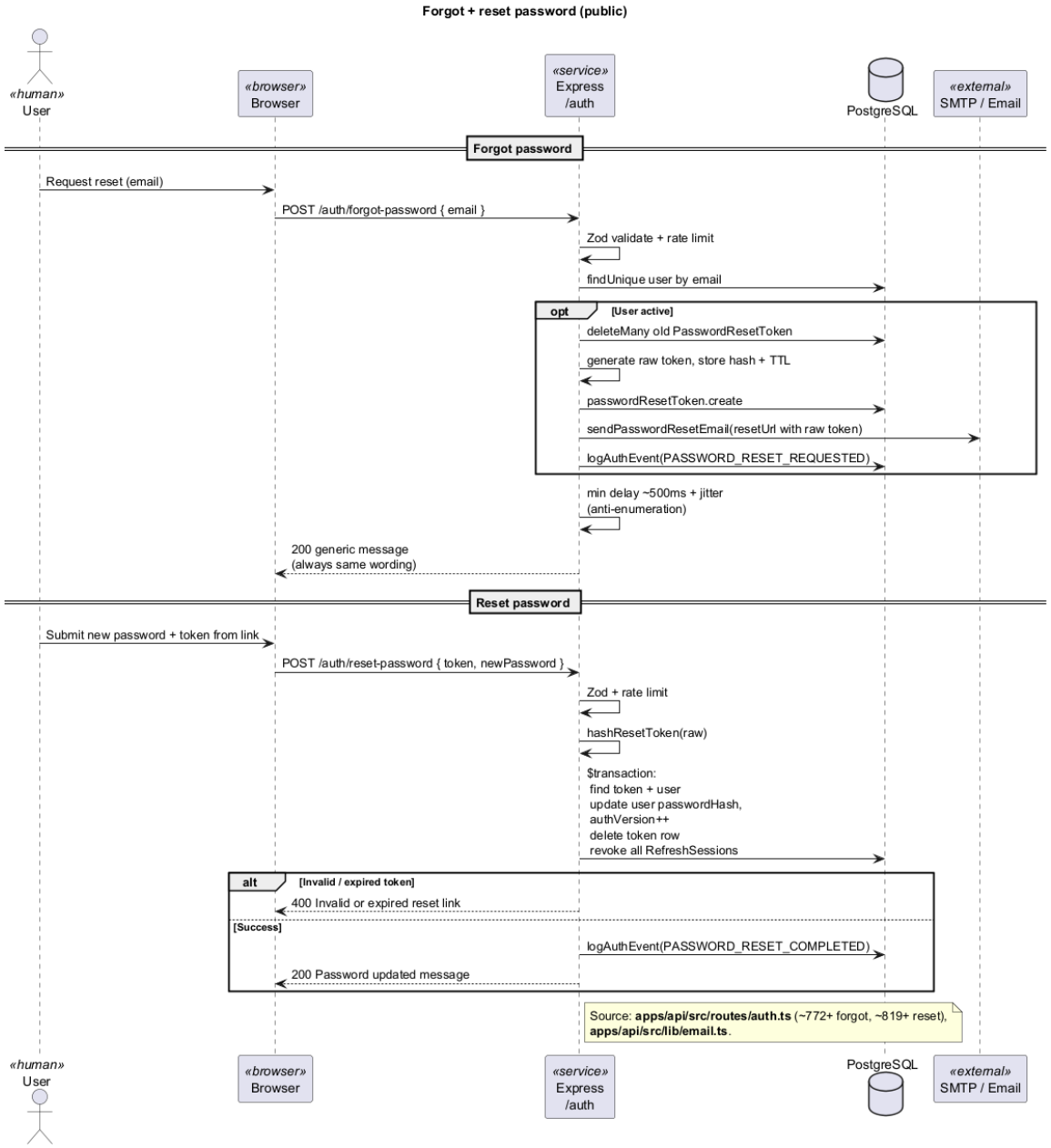

Figure 3.7 – Forgotten password recovery — sequence diagram

Changing password while signed in — sequence diagram.

A signed-in user can change their password after confirming the current one. A successful change closes other active sessions for safety.

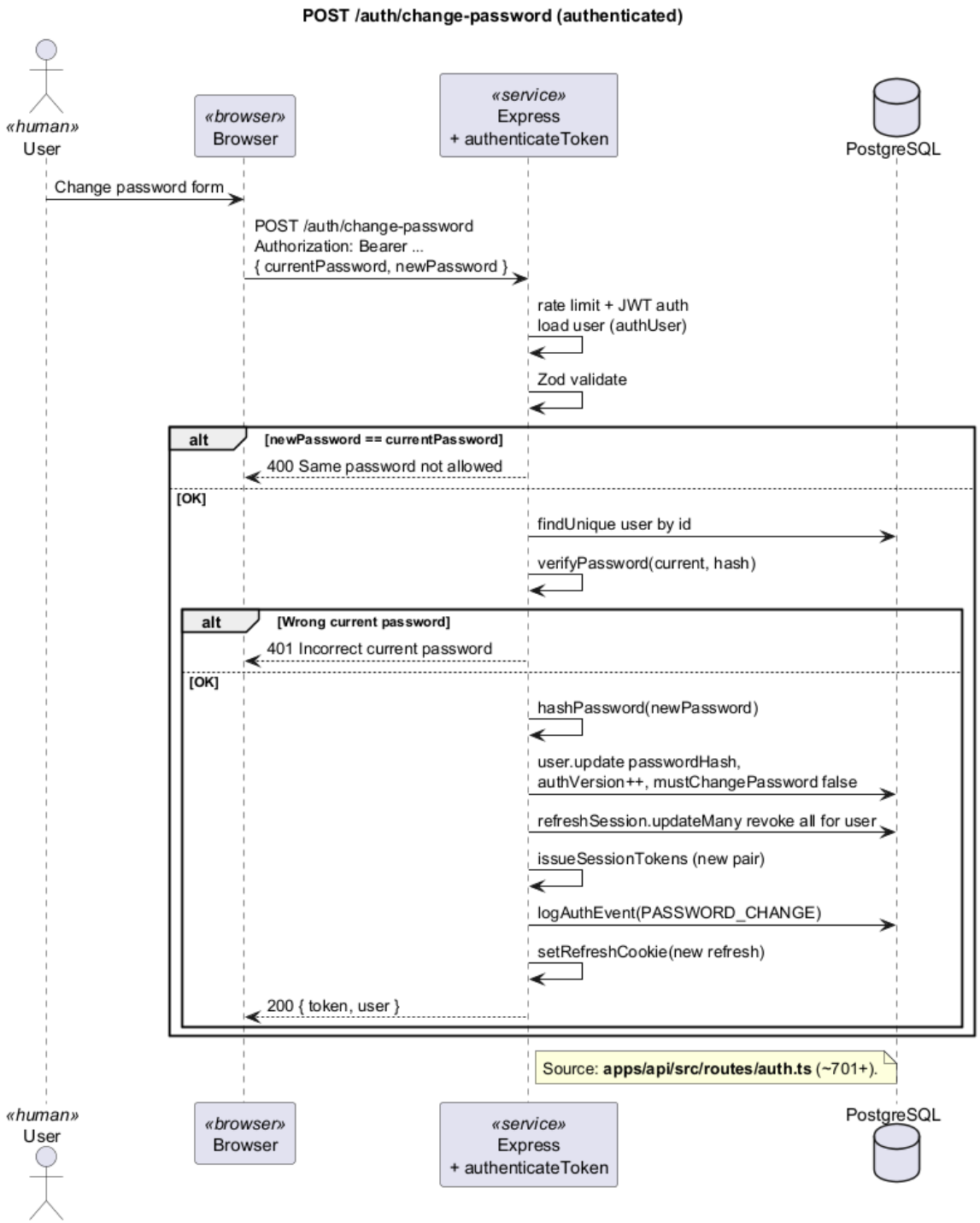

Figure 3.8 – Changing password while signed in — sequence diagram

Profile, permissions and session context

Updating the user profile — sequence diagram.

Users can update personal details such as display name and contact information. Some changes, such as email updates, require the session to be renewed for security.

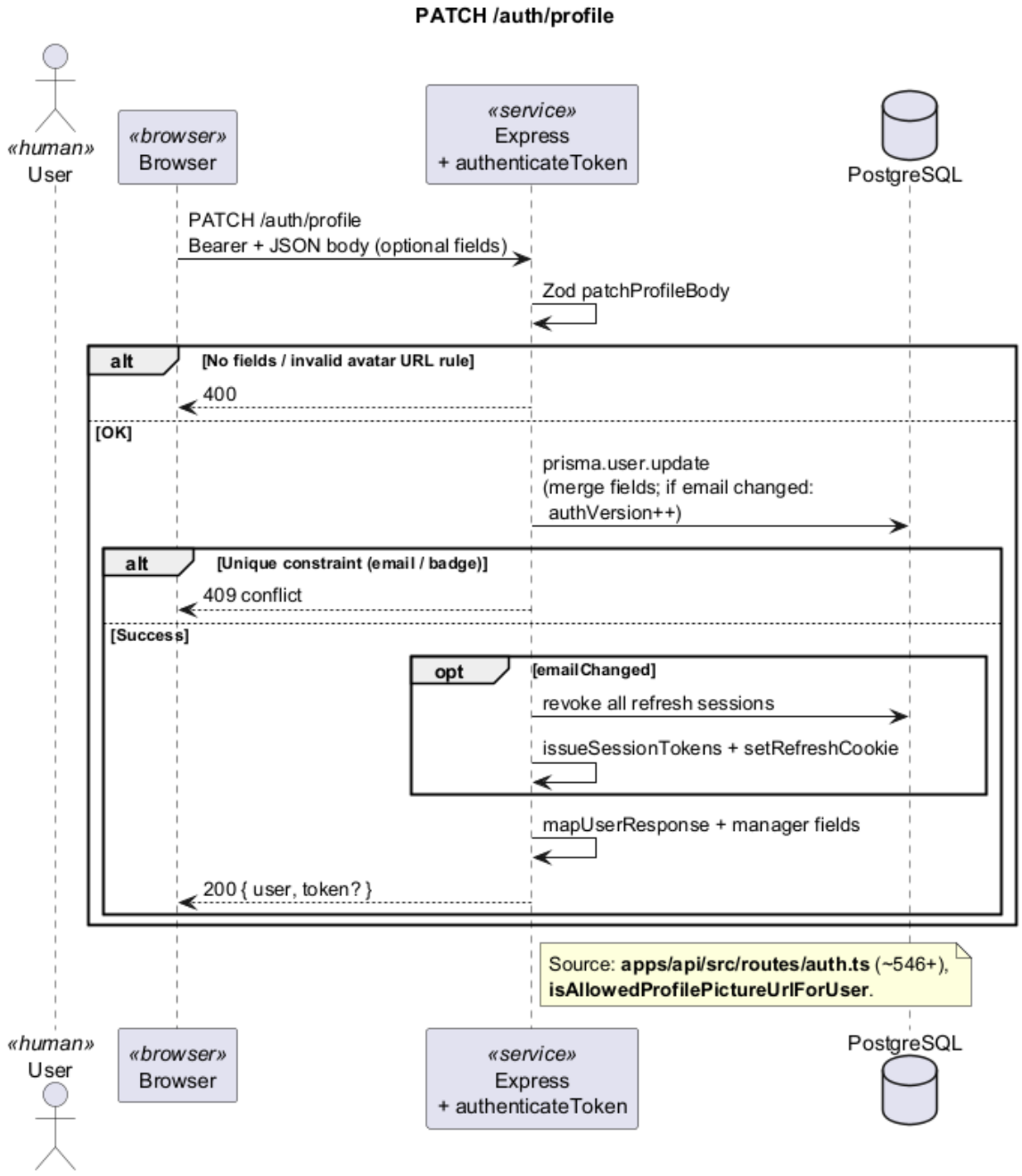

Figure 3.9 – Updating the user profile — sequence diagram

Blocking access to a restricted feature — sequence diagram.

Before sensitive actions run, the server checks whether the user may use the document library or AI-powered search. If not, the request is rejected with a clear permission error.

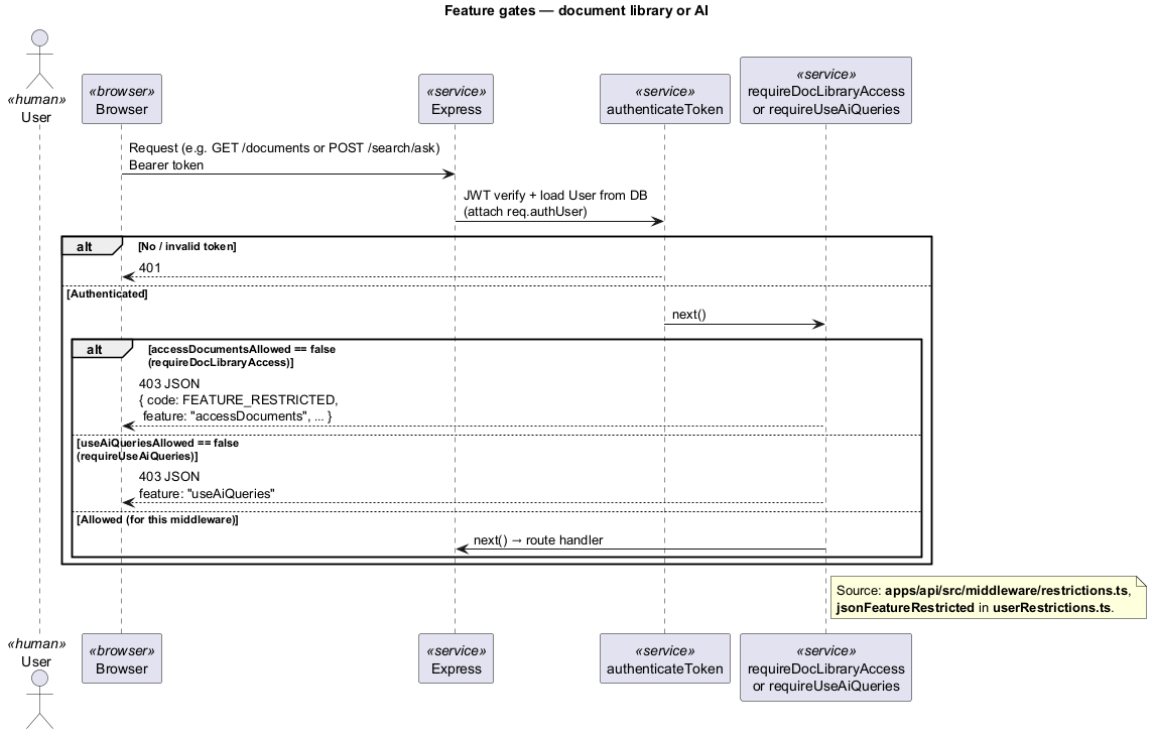

Figure 3.10 – Blocking access to a restricted feature — sequence diagram

3.5.2 Sprint 2: Document library and ingest

Sprint 2 introduces the document library: uploading files, preparing them for search, browsing and downloading content, and managing versions and metadata within department visibility rules.

3.5.2.1 Sprint 2 structural models: document library and ingest

Sprint 2 use-case diagram.

Figure 3.11 presents the Sprint 2 functional boundary for the document library. Employee, Manager, and Admin actors inherit capabilities for browsing, downloading, favourites, uploads, versioning, metadata, archive, reprocess, and bulk delete. External

Document processing is linked to upload and reprocess. The diagram aims to separate consumption from stewardship roles before the ingest and library sequence flows.

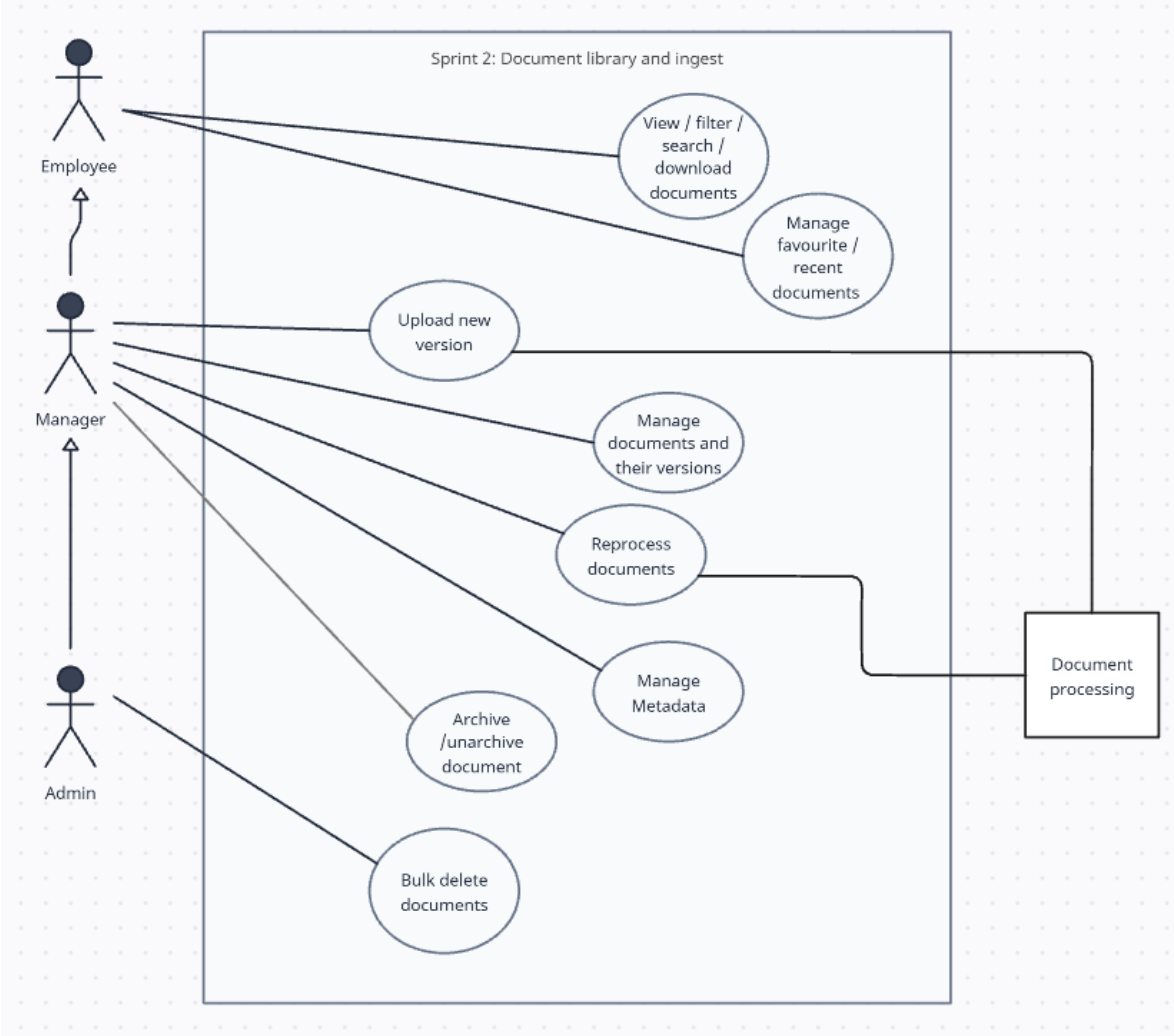

Figure 3.11 – Sprint 2 use-case diagram — Document library and ingest

Sprint 2 domain class diagram.

Figure 3.12 isolates the document-centric domain objects: documents, versions, metadata and tags, department visibility, favourites or recents, and processing status. It shows how library state is modelled so that asynchronous ingest can update search readiness without changing the user-facing document identity.

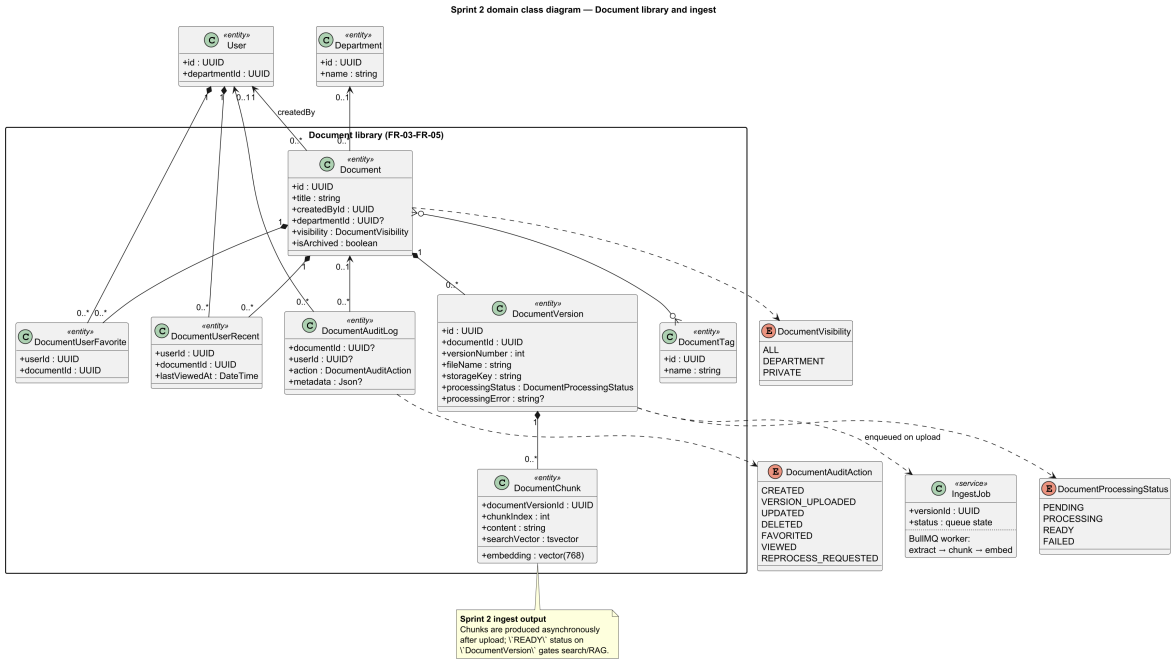

Figure 3.12 – Sprint 2 domain class diagram — Document library and ingest

3.5.2.2 Sprint 2 interaction sequences: upload, ingest and library flows

Uploading and preparing documents

Uploading a document — sequence diagram.

A user uploads a file with basic metadata. The platform stores the file, records the document in the library, and queues background processing so the user does not have to wait for indexing to finish.

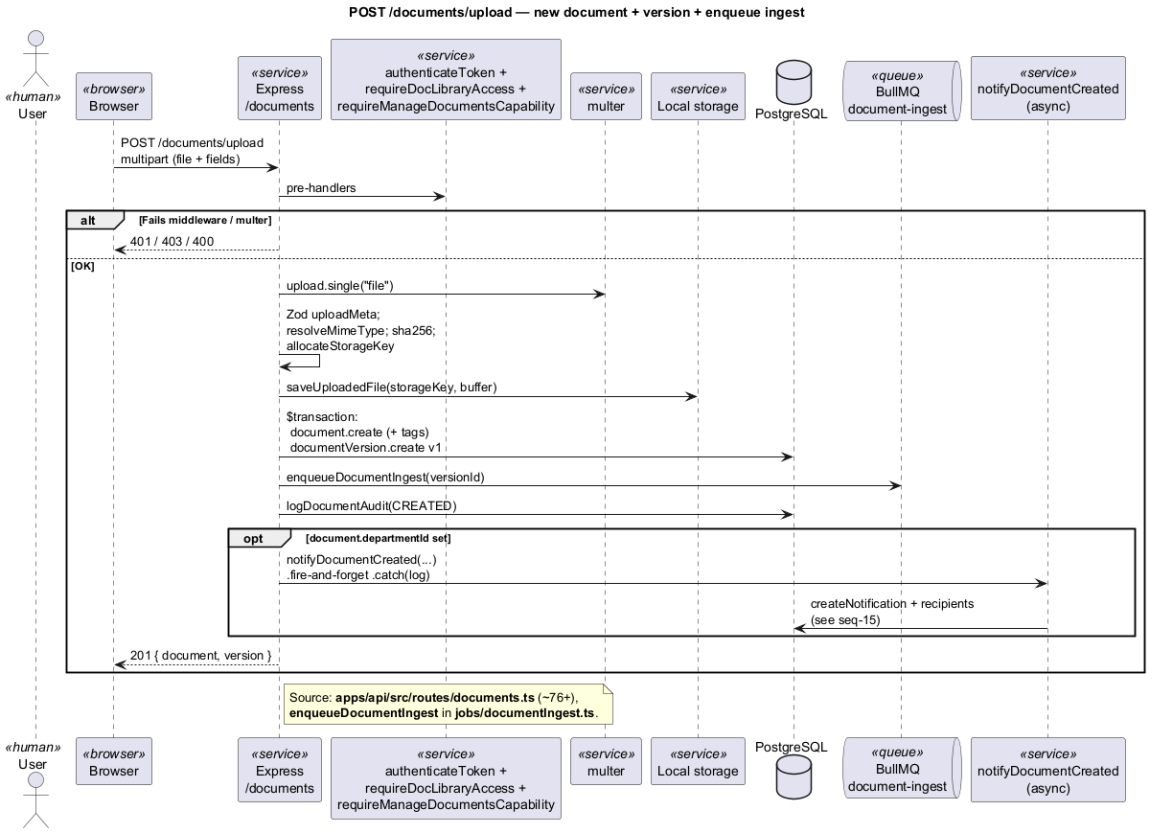

Figure 3.13 – Uploading a document — sequence diagram

Background document processing — sequence diagram.

After upload, a background worker extracts text, splits it into meaningful passages, creates searchable embeddings, and stores them so the document can later appear in semantic search and AI answers.

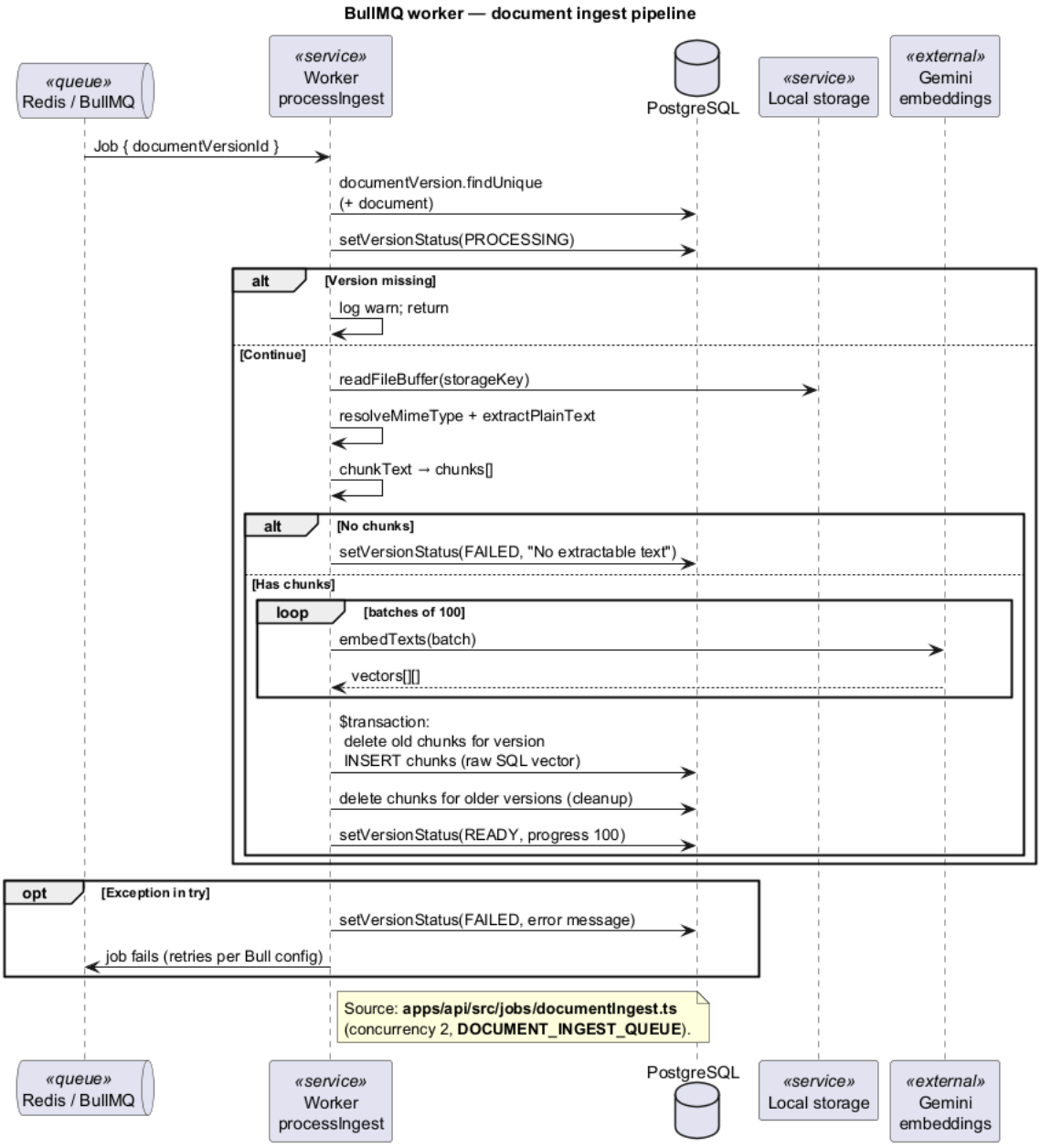

Figure 3.14 – Background document processing — sequence diagram

Reprocessing an existing version — sequence diagram.

An administrator can trigger reprocessing of a document version, for example after improving the indexing pipeline, without uploading the file again.

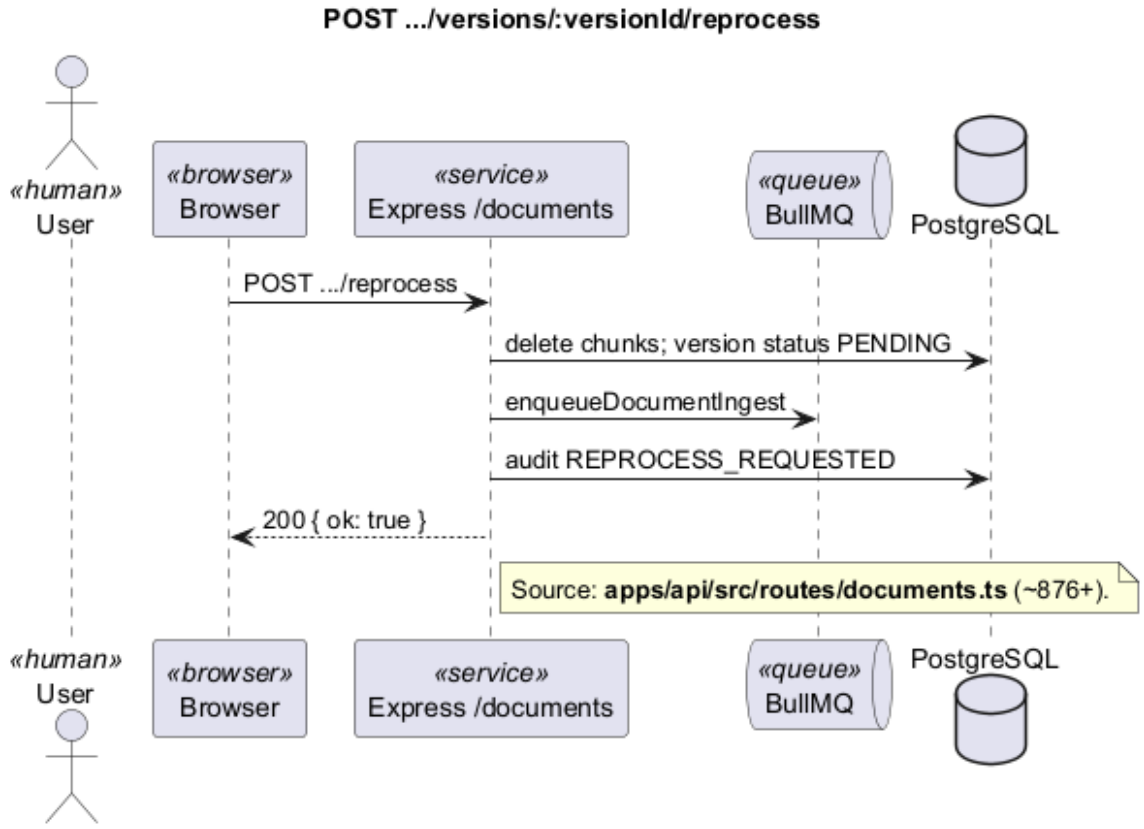

Figure 3.15 – Reprocessing an existing version — sequence diagram

Browsing and downloading

Downloading a document file — sequence diagram.

An authorised user downloads the original file. The platform verifies department access before sending the file.

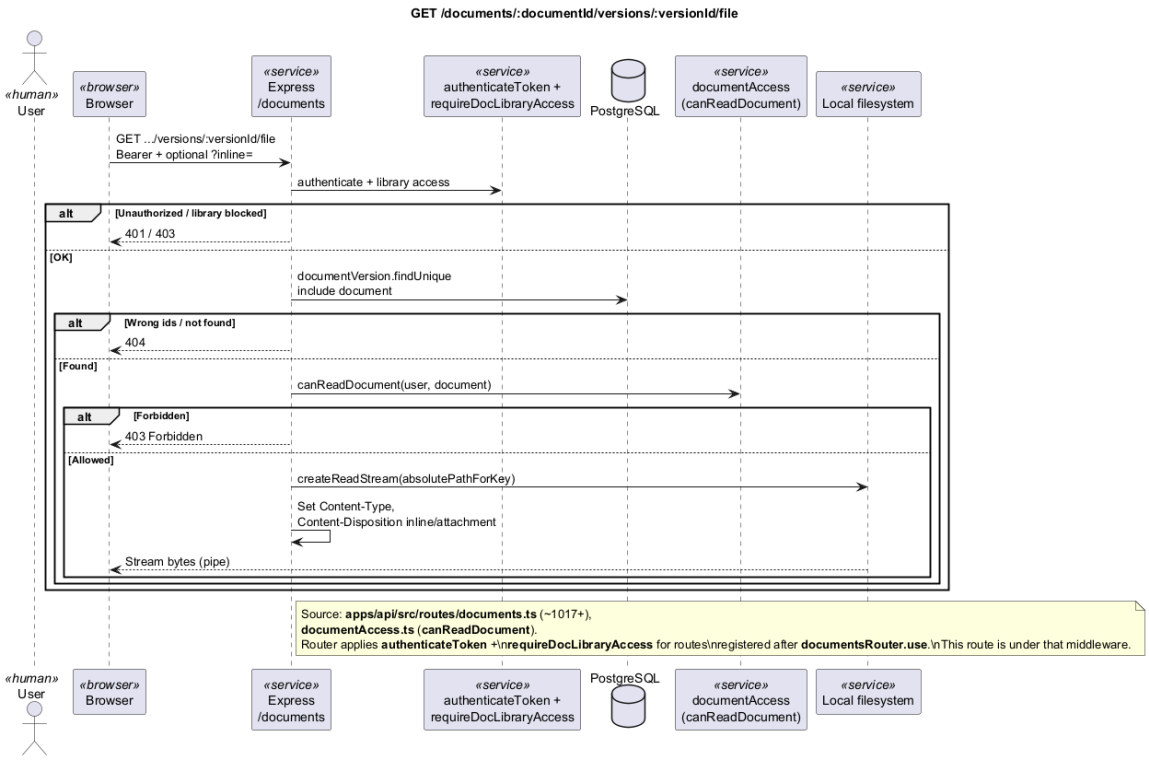

Figure 3.16 – Downloading a document file — sequence diagram

Browsing the document library — sequence diagram.

Users browse the library with filters and pagination. Results show summary information suitable for the catalogue view without loading full text content.

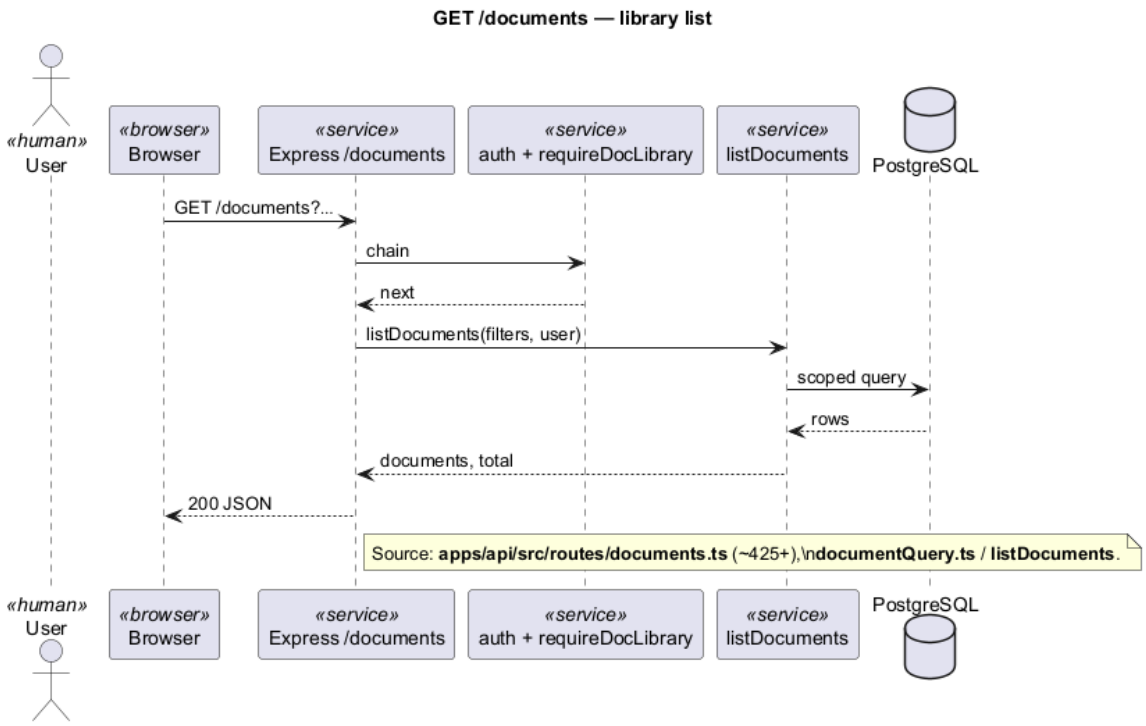

Figure 3.17 – Browsing the document library — sequence diagram

Opening a document detail view — sequence diagram.

Opening a document shows its metadata, current version, tags, processing status, and the information needed for preview or further actions.

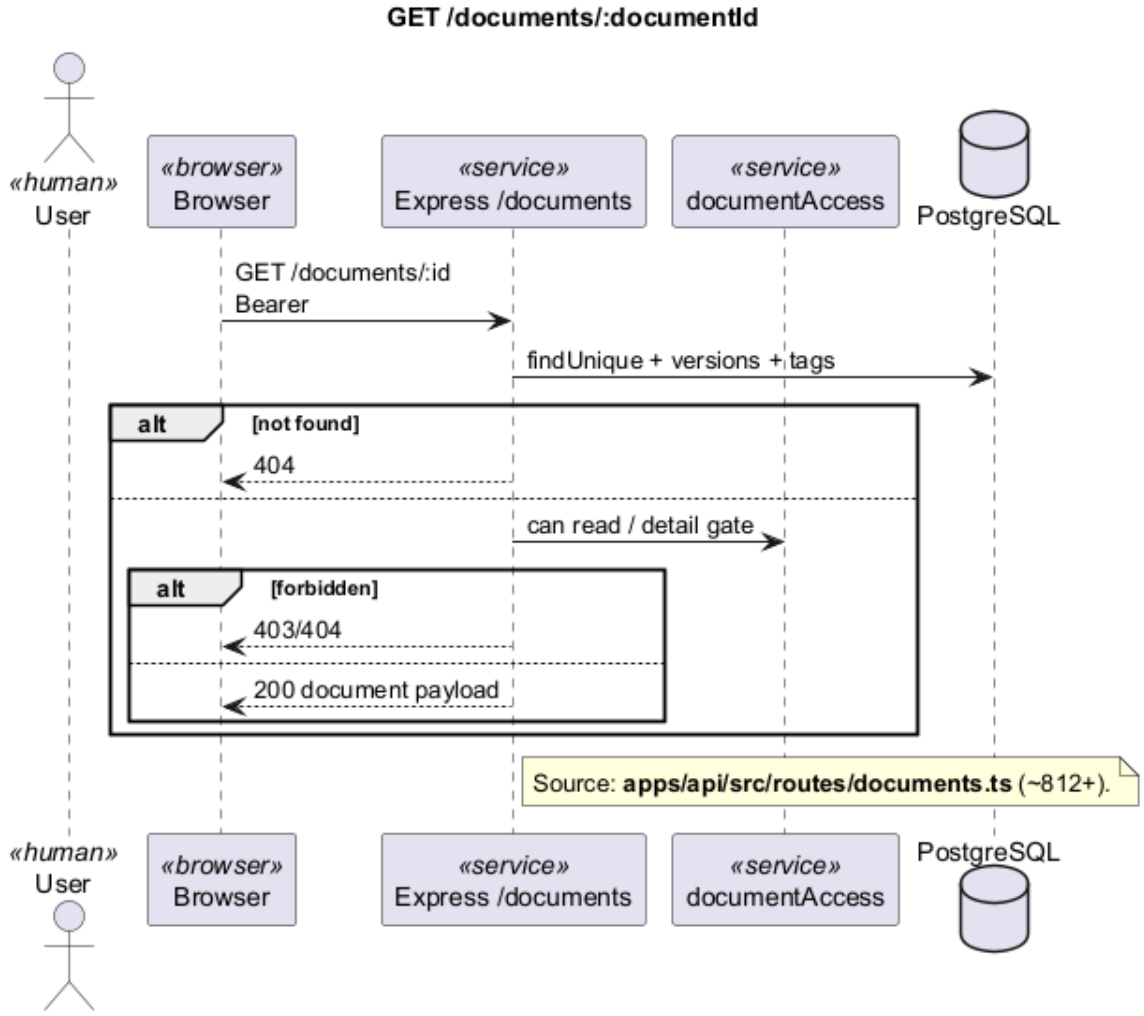

Figure 3.18 – Opening a document detail view — sequence diagram

Favourites and recently viewed documents — sequence diagram.

Users can mark documents as favourites and revisit recently opened items from the library home view.

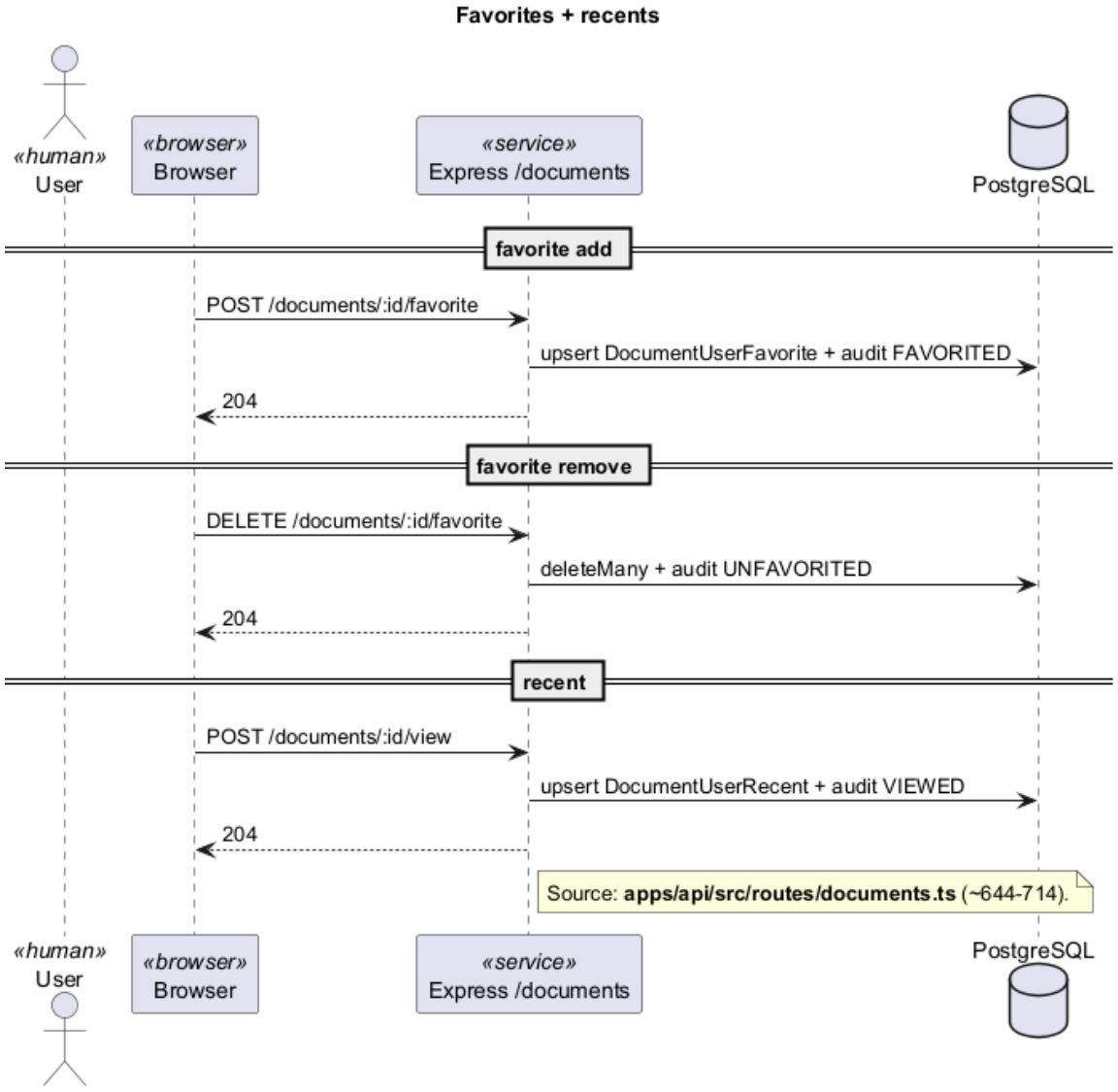

Figure 3.19 – Favourites and recently viewed documents — sequence diagram

Editing metadata and versions

Updating document metadata — sequence diagram.

Authorised users can edit title, description, tags, and visibility. Changes are recorded for traceability.

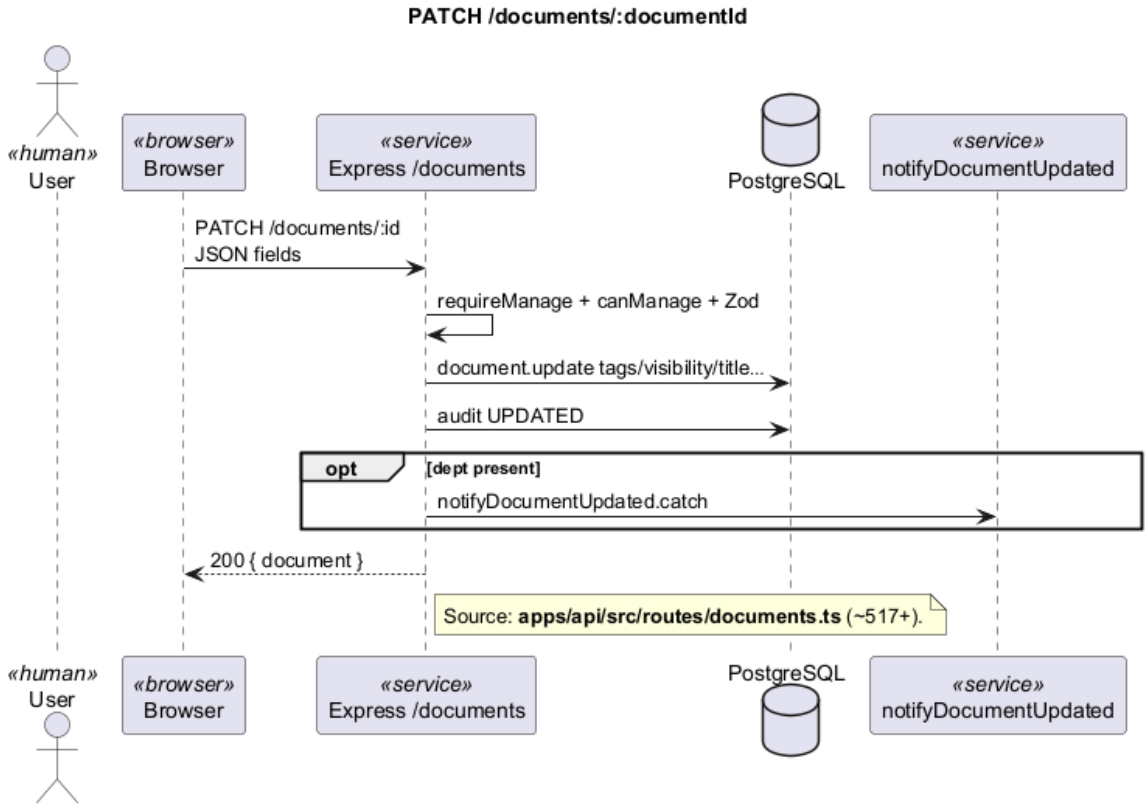

Figure 3.20 – Updating document metadata — sequence diagram

Publishing a new document version — sequence diagram.

Uploading a replacement file creates a new version, starts fresh indexing, and keeps earlier versions available for audit and rollback.

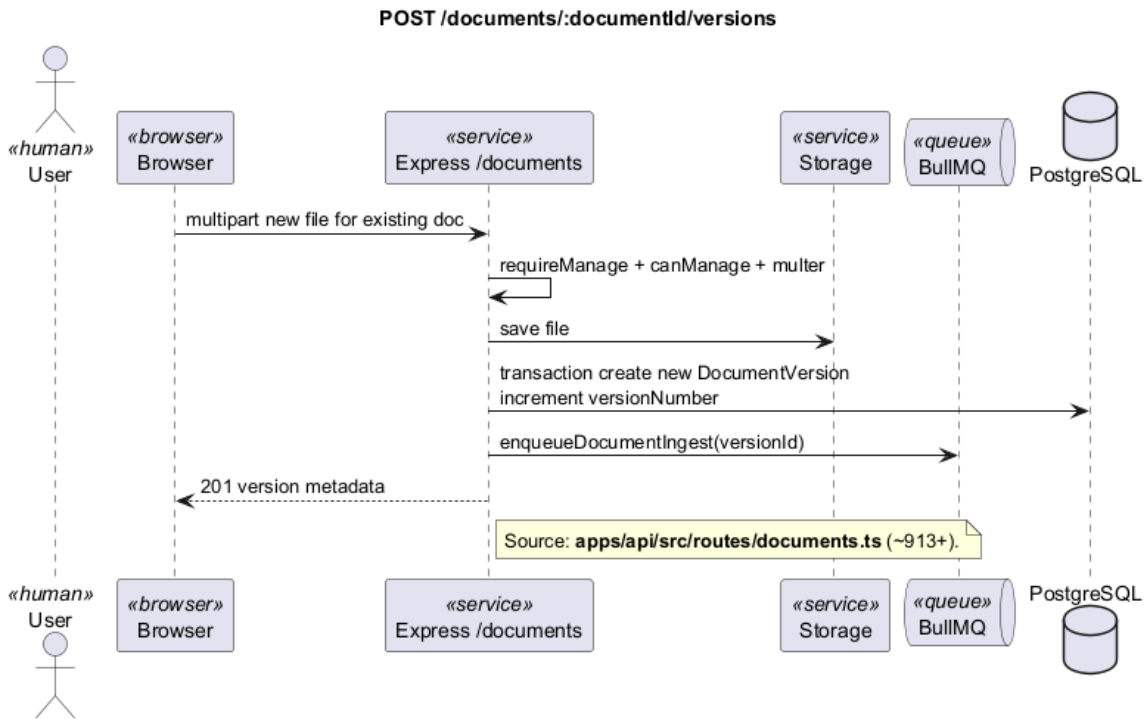

Figure 3.21 – Publishing a new document version — sequence diagram

3.5.3 Sprint 3: Search, RAG and conversations

Sprint 3 adds intelligent search over the document base and a conversational workspace where users can ask questions and receive answers grounded in organisational content.

3.5.3.1 Sprint 3 structural models: search, RAG and conversations

Sprint 3 use-case diagram.

Figure 3.22 scopes Sprint 3 around conversations, RAG questions, semantic search, answer feedback, and administrator feedback analytics. Semantic search is included by the Ask flow and depends on the external AI provider (Gemini). The diagram aims to show how retrieval and dialogue capabilities sit with employee users while analytics remain an administrative concern.

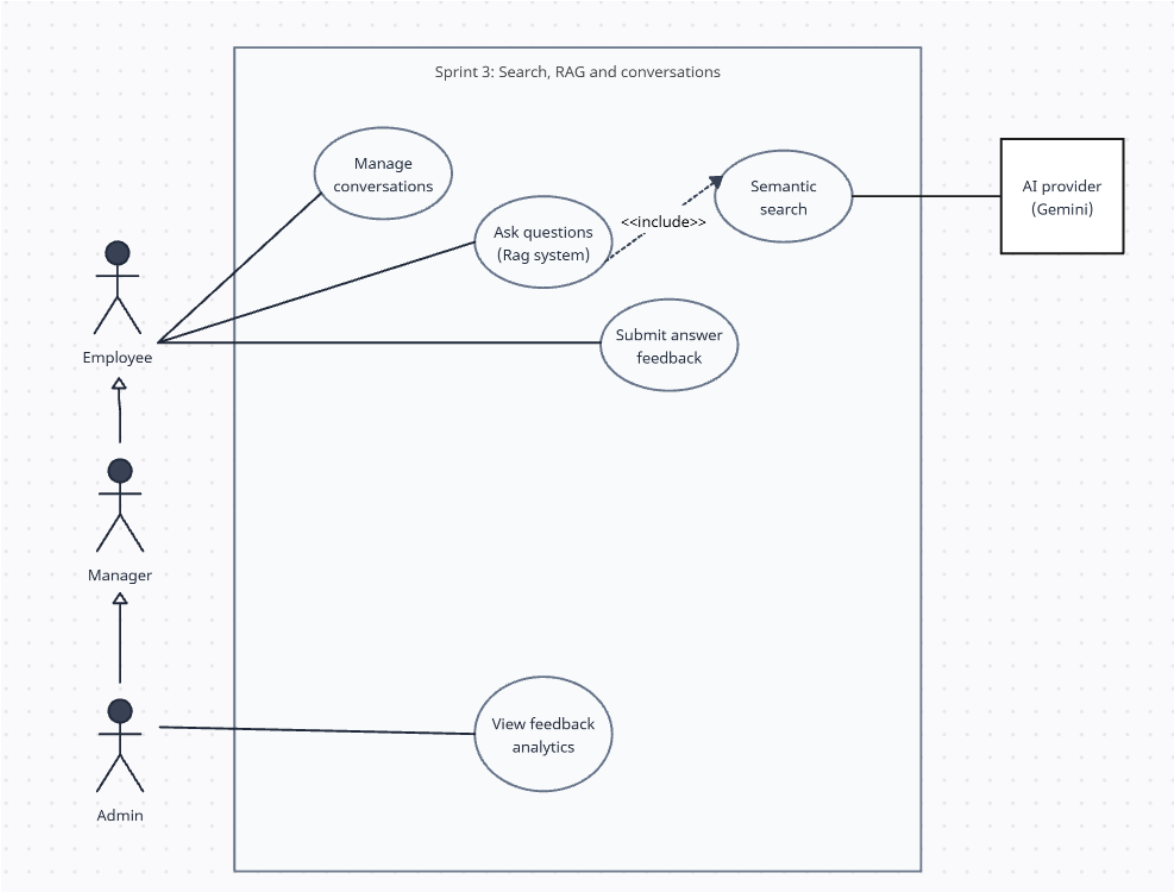

Figure 3.22 – Sprint 3 use-case diagram — Search, RAG and conversations

Sprint 3 domain class diagram.

Figure 3.23 highlights conversation threads, messages, citations or retrieved chunks, and answer-feedback records linked to users and documents. It is meant to clarify the data that backs search and Ask so that the later sequence diagrams can refer to a stable conversation and feedback model.

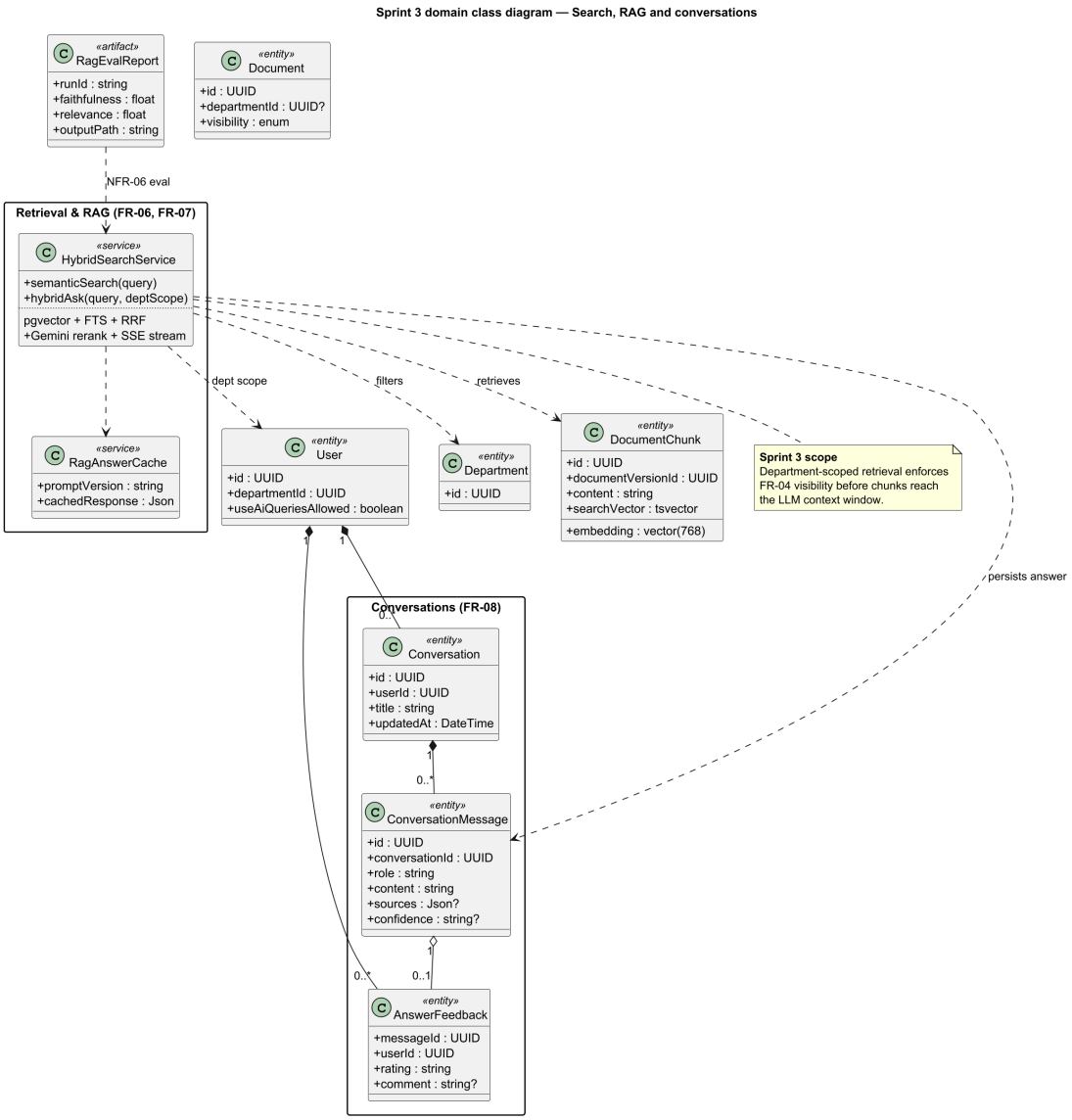

Figure 3.23 – Sprint 3 domain class diagram — Search, RAG and conversations

3.5.3.2 Sprint 3 interaction sequences: search, Ask and conversation flows

Search and AI-assisted answers

Semantic search — sequence diagram.

A user enters a natural-language query; the platform finds the most relevant document passages and returns ranked excerpts for direct exploration.

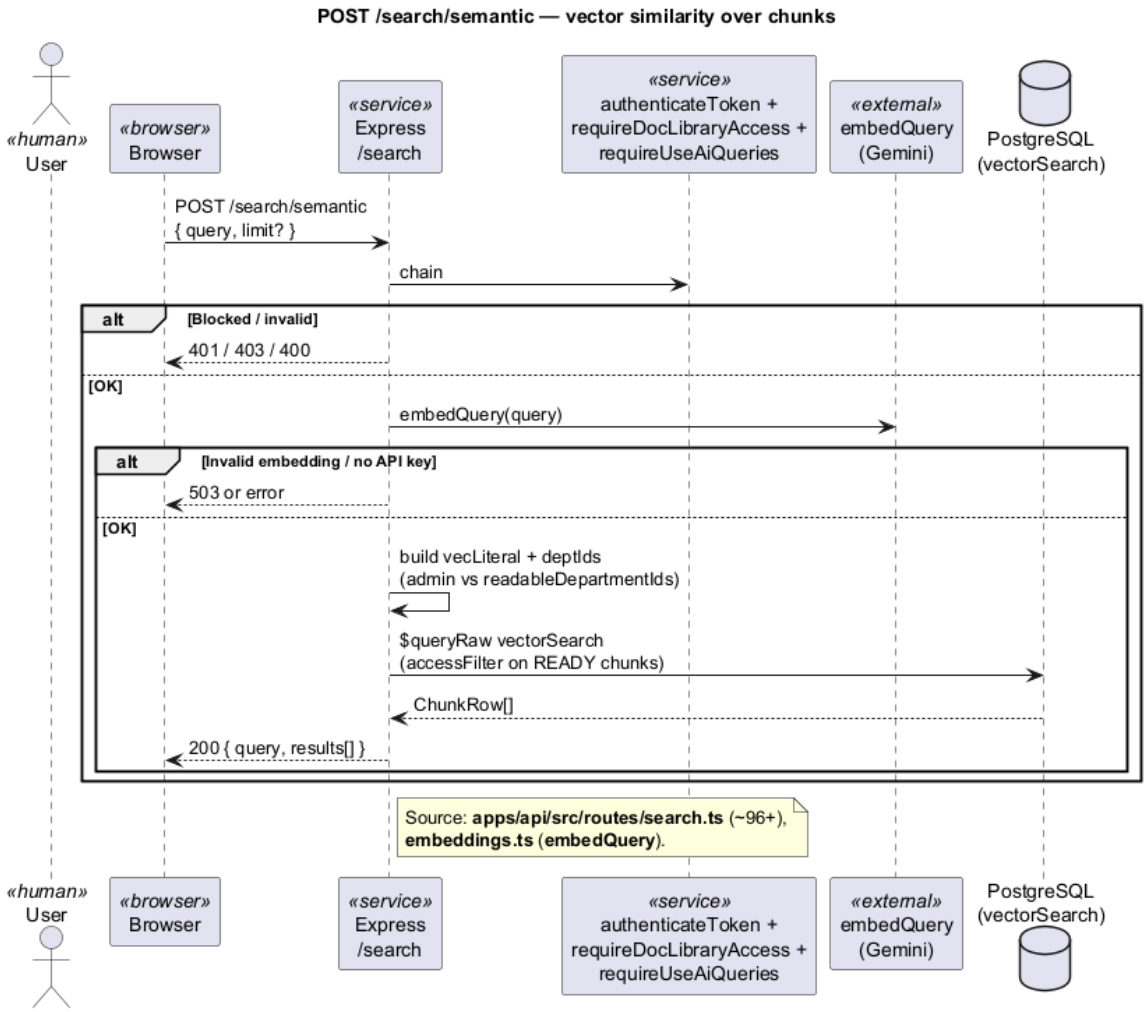

Figure 3.24 – Semantic search — sequence diagram

Asking a question with AI assistance — sequence diagram.

In the Ask workspace, the platform retrieves relevant passages, combines keyword and semantic matching, and streams a structured answer with source references to the user in real time.

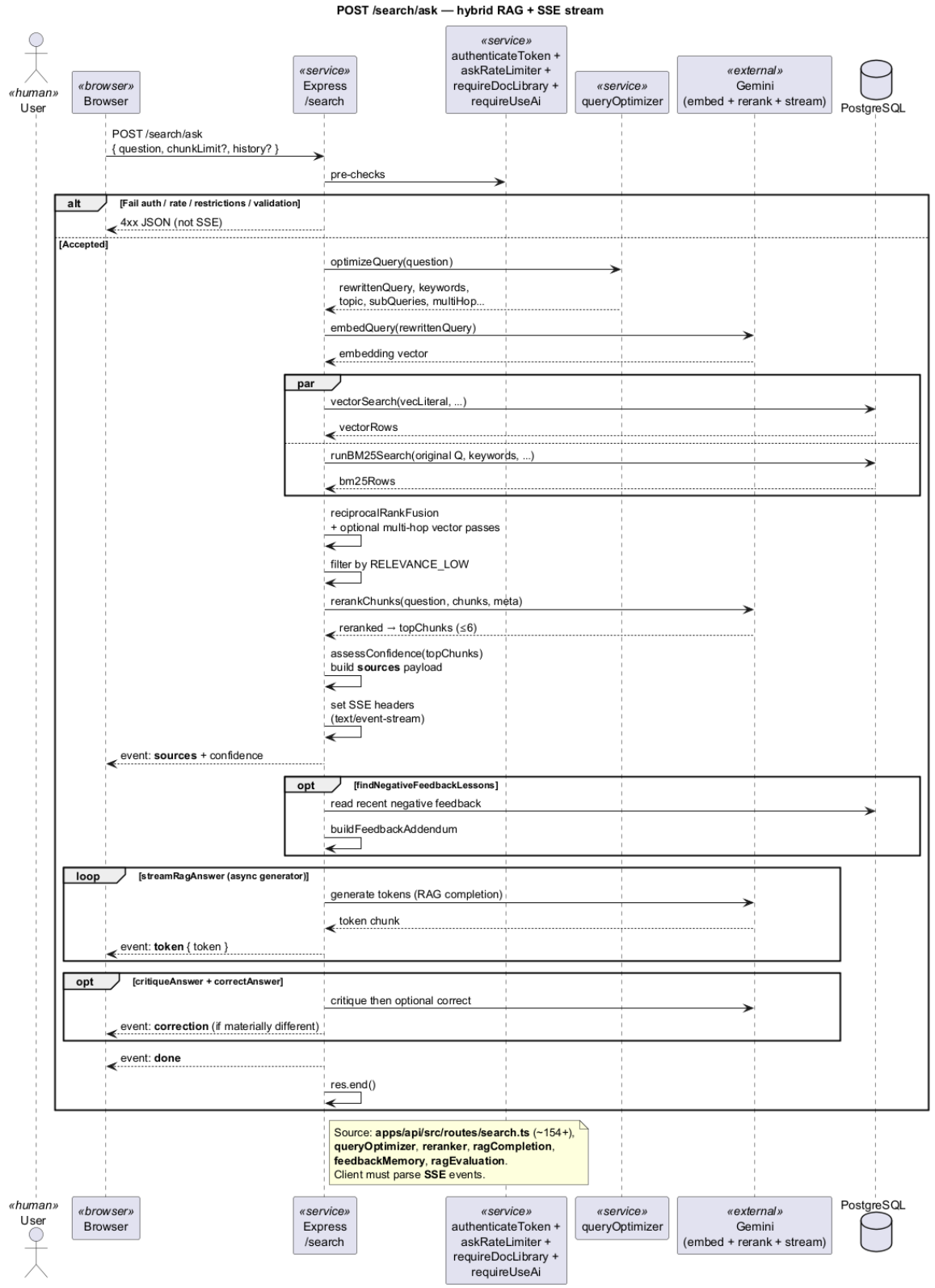

Figure 3.25 – Asking a question with AI assistance — sequence diagram

Conversations and answer quality

Creating conversations and adding messages — sequence diagram.

Users open a new conversation, ask questions, receive answers, and can automatically generate a short title from the first question. Opening an existing thread loads its full history (see Figure 3.28).

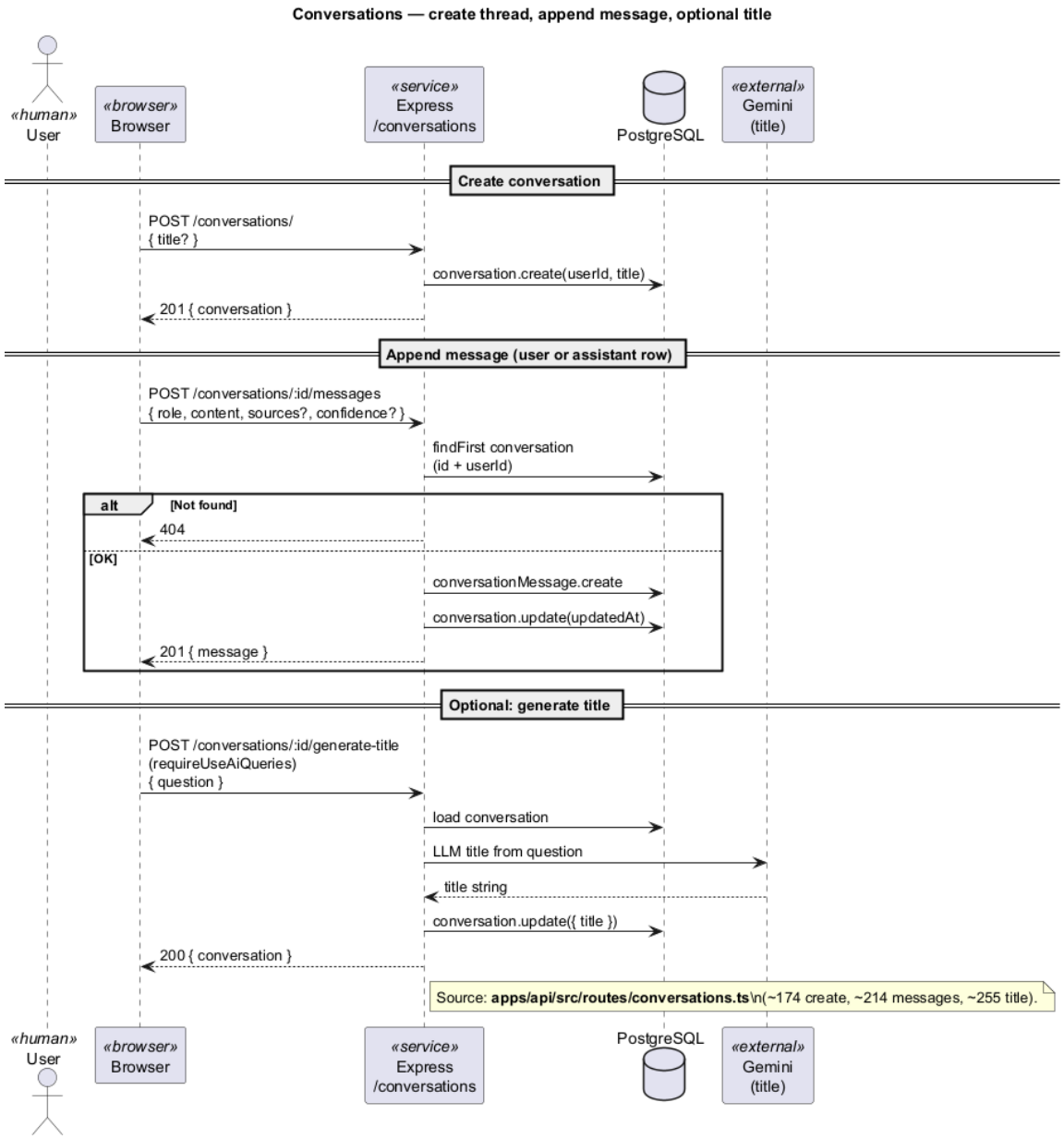

Figure 3.26 – Creating conversations and adding messages — sequence diagram

Rating an answer — sequence diagram.

After receiving an answer, users can submit positive or negative feedback with an optional comment, helping administrators monitor answer quality over time.

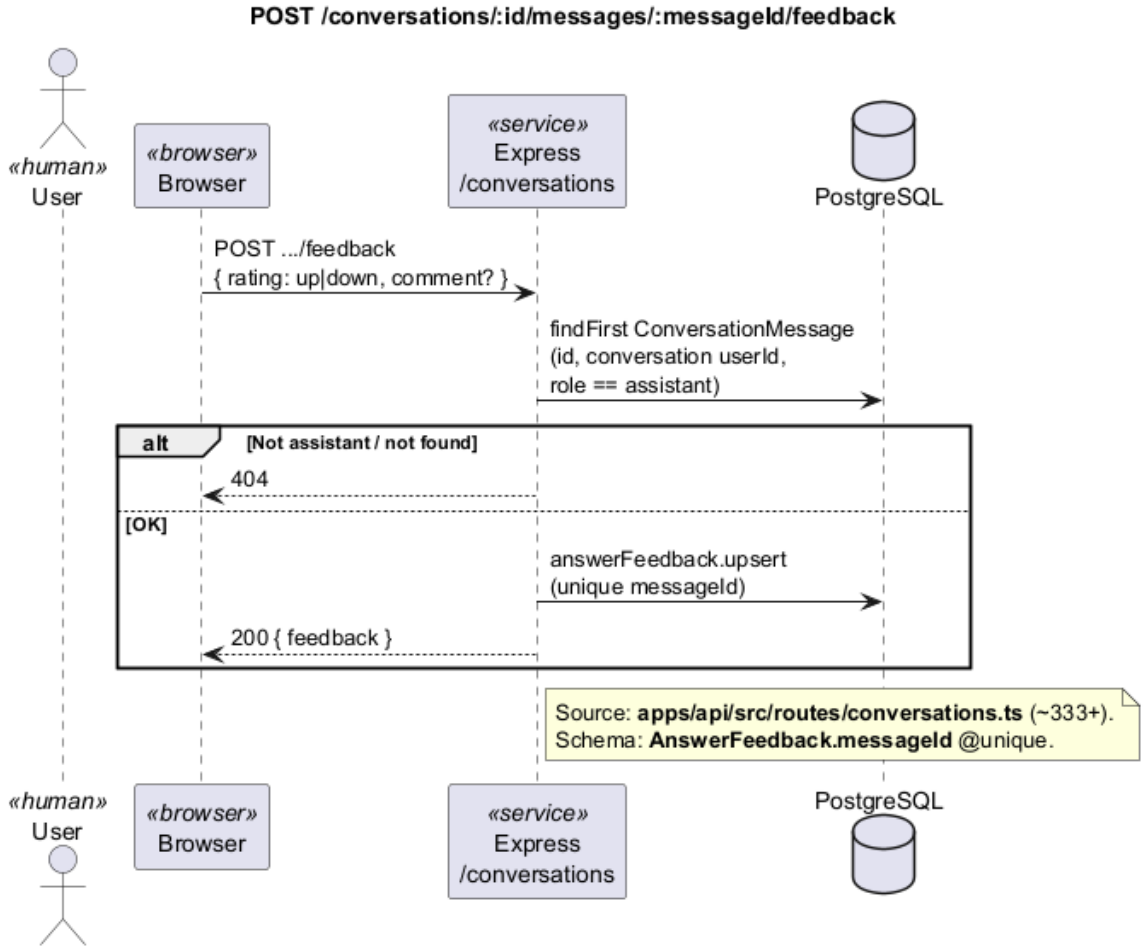

Figure 3.27 – Rating an answer — sequence diagram

Managing conversation threads — sequence diagram.

Users list past conversations, open a thread with its message history, rename a thread, or delete it when it is no longer needed.

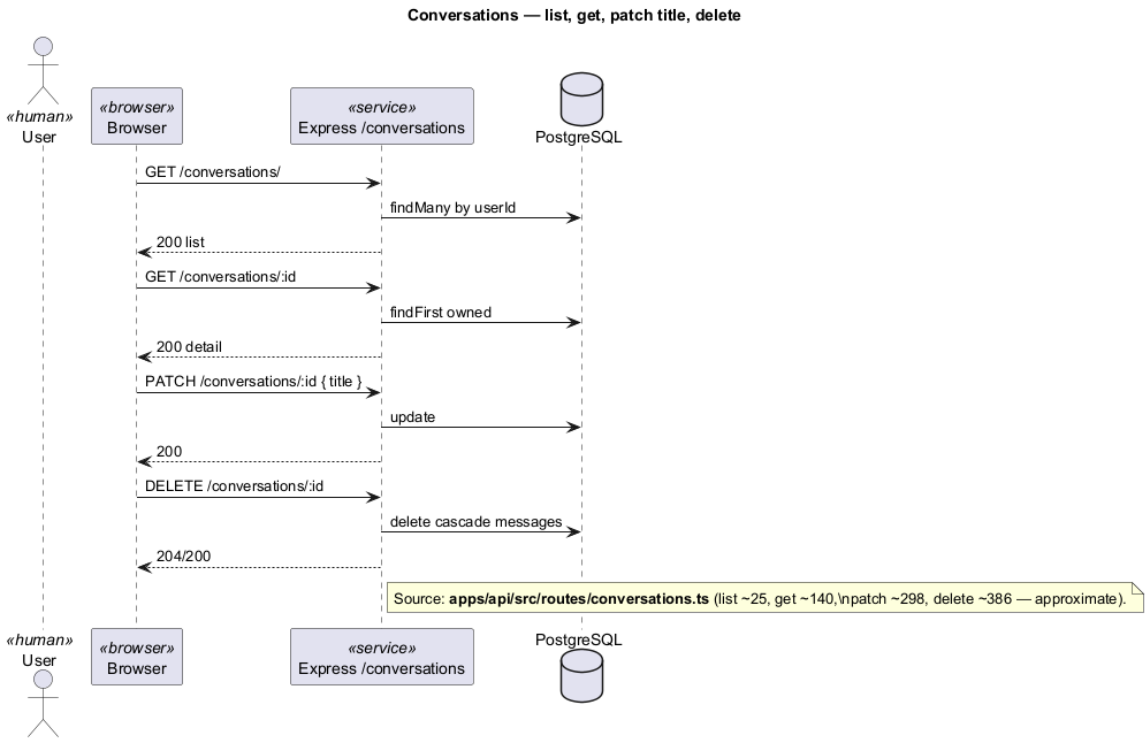

Figure 3.28 – Managing conversation threads — sequence diagram

Administrator feedback overview — sequence diagram.

Administrators consult aggregated feedback metrics (volume, satisfaction ratio, and recent comments) to track how well AI answers are received across the organisation.

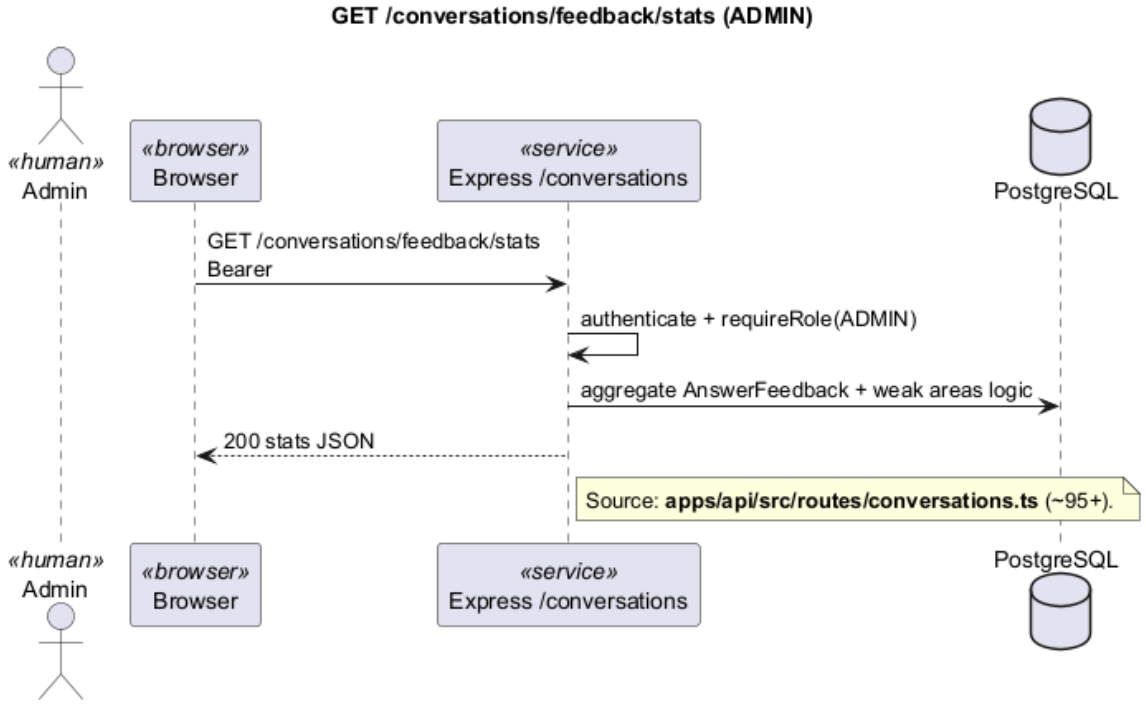

Figure 3.29 – Administrator feedback overview — sequence diagram

3.5.4 Sprint 4: Governance and notifications

Sprint 4 supports day-to-day governance: keeping users informed through notifications, giving managers visibility over their teams, and providing administrators with tools to manage users, departments, and documents at scale.

3.5.4.1 Sprint 4 structural models: governance and notifications

Sprint 4 use-case diagram.

Figure 3.30 groups Sprint 4 capabilities into employee notification browsing and downloads, manager send and attachment flows, and administrator user, document, export, and audit views. System events trigger automatic notifications outside the interactive actor set. The aim is to show how operational communication and governance tools share one sprint boundary.

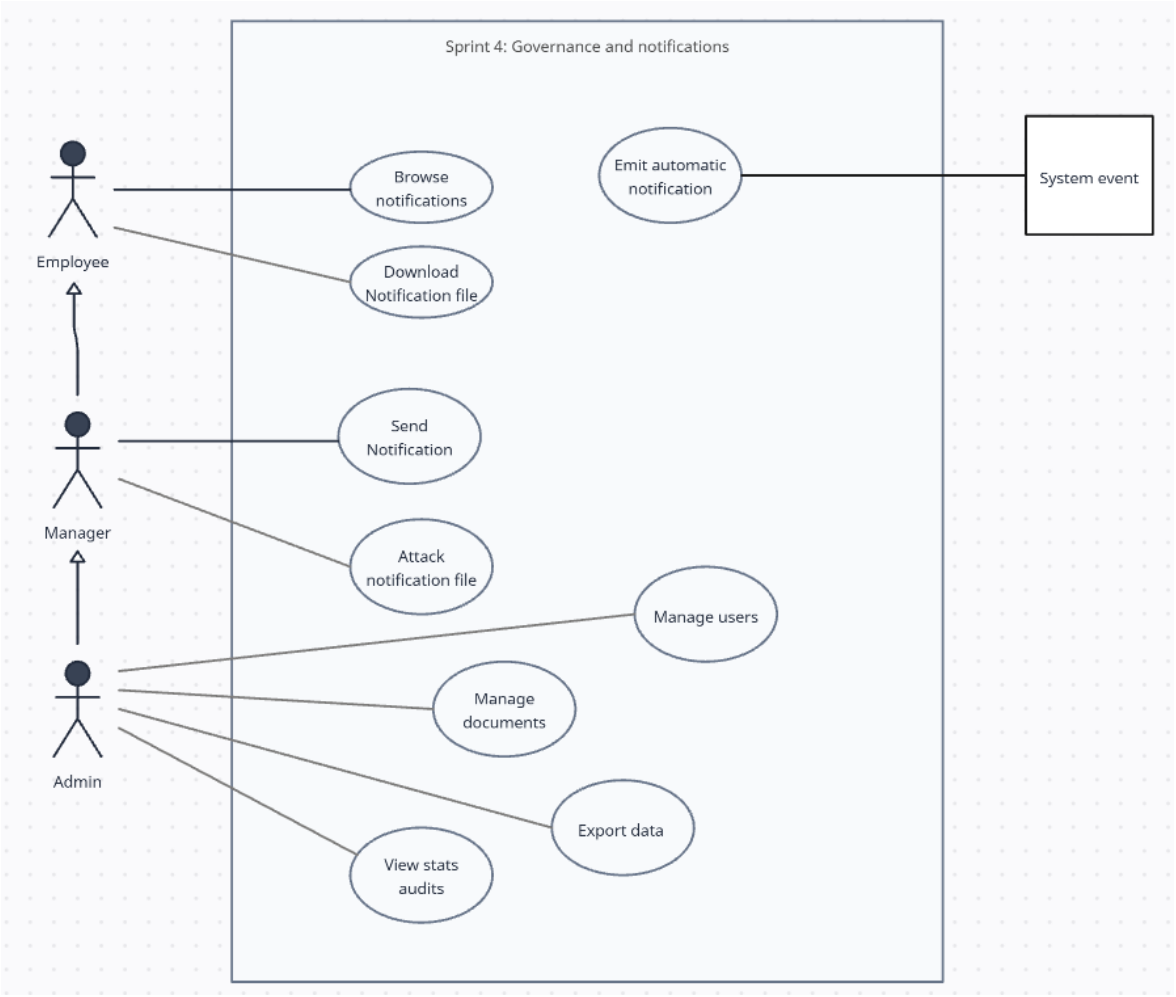

Figure 3.30 – Sprint 4 use-case diagram — Governance and notifications

Sprint 4 domain class diagram.

Figure 3.31 covers notification messages and attachments, department membership views used by managers, and the administrative entities needed for users, departments, access grants, activity logs, and exports. It shows which persisted structures support inbox delivery and governance operations in this sprint.

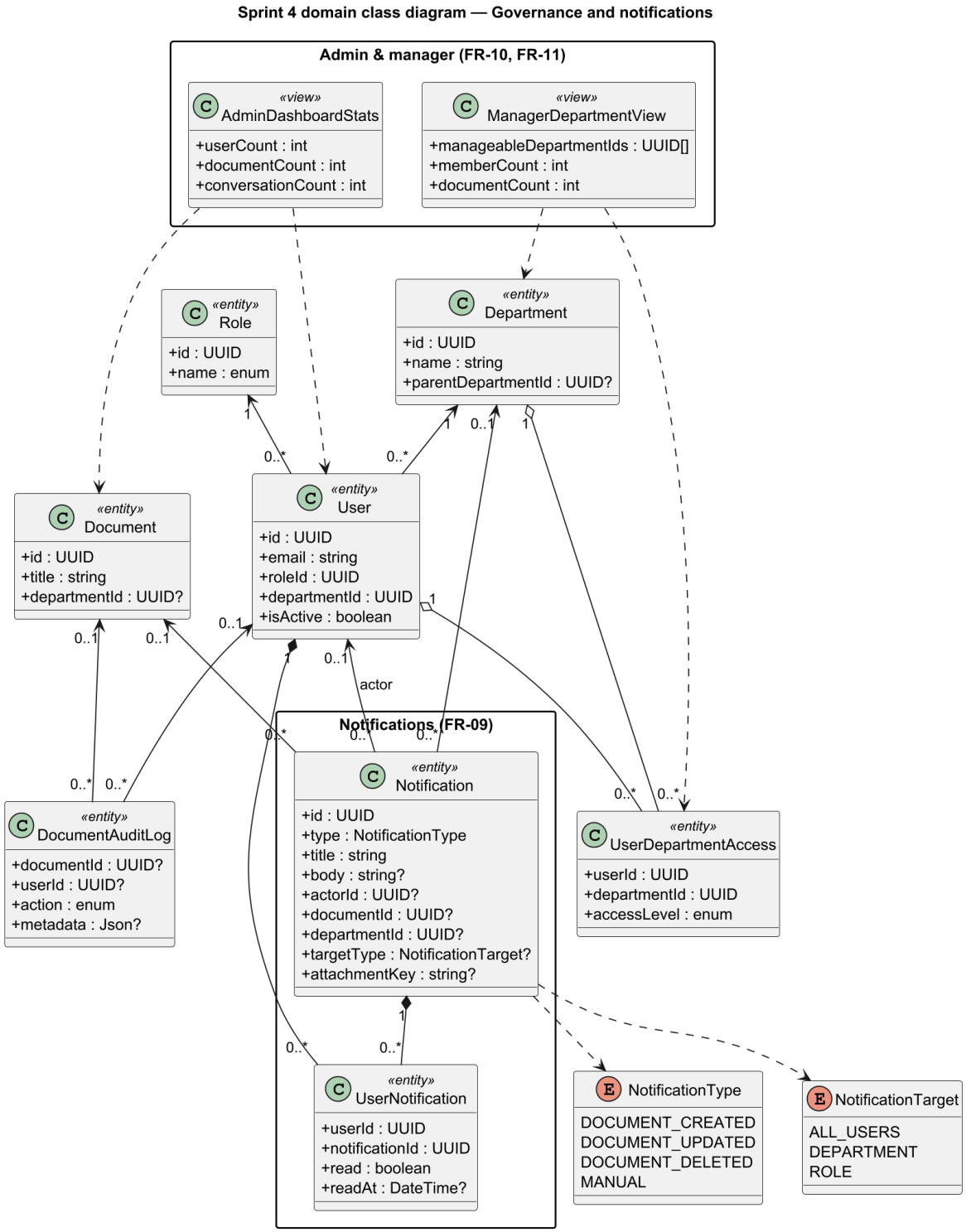

Figure 3.31 – Sprint 4 domain class diagram — Governance and notifications

3.5.4.2 Sprint 4 interaction sequences: notifications and governance flows

Notifications

Automatic notification when a document is added — sequence diagram.

When a new document is published, subscribed department members receive an inbox notification without delaying the upload experience.

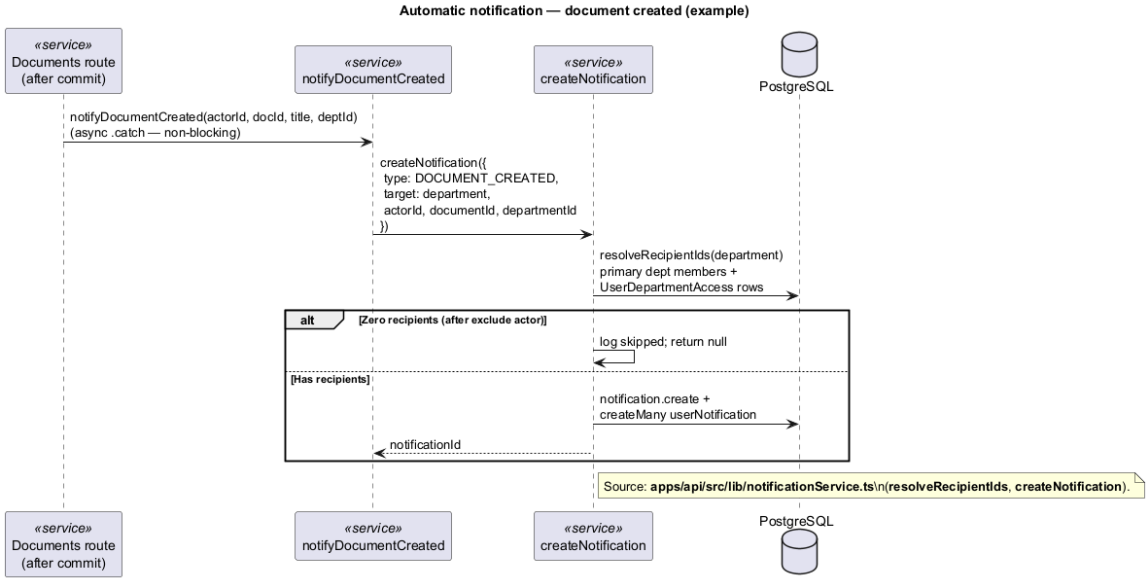

Figure 3.32 – Automatic notification when a document is added — sequence diagram

Sending a manual announcement — sequence diagram.

Administrators and managers can send targeted announcements, optionally with attachments, to selected users or departments.

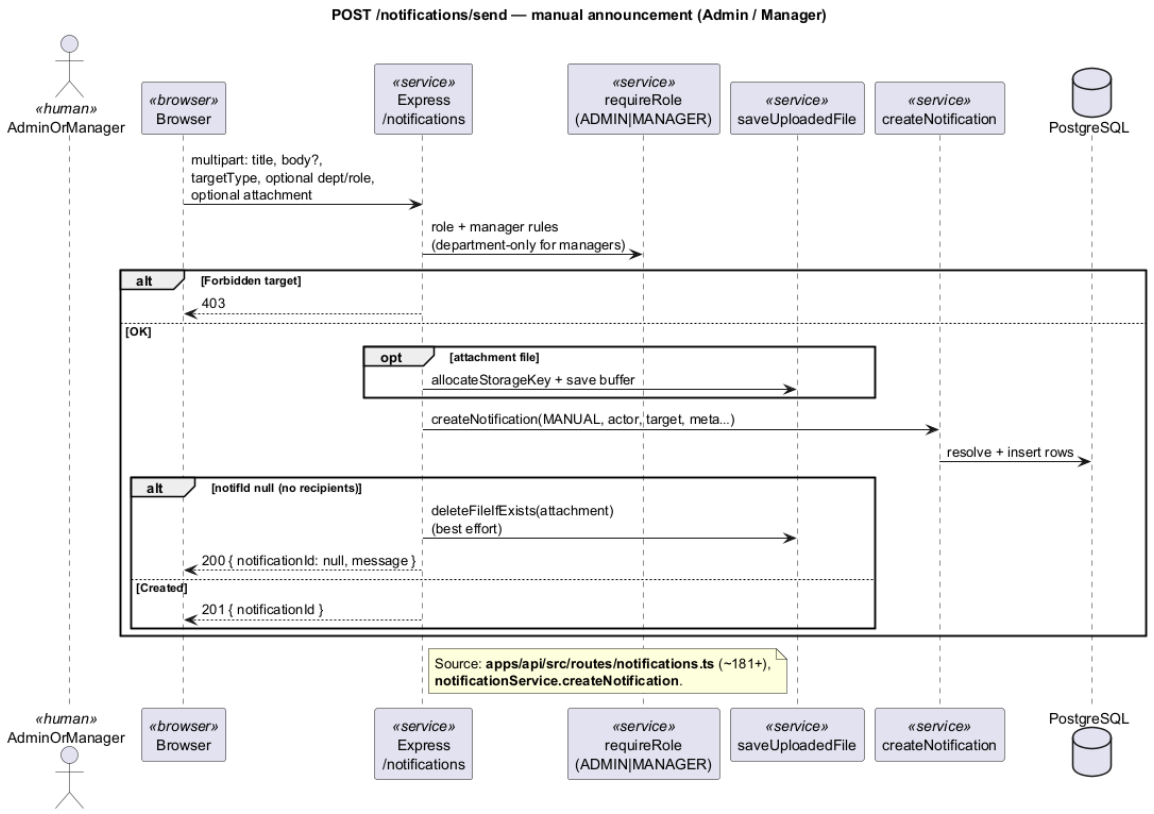

Figure 3.33 – Sending a manual announcement — sequence diagram

Reading and managing notifications — sequence diagram.

Users open their notification inbox, see unread items, mark messages as read, and access any attached content.

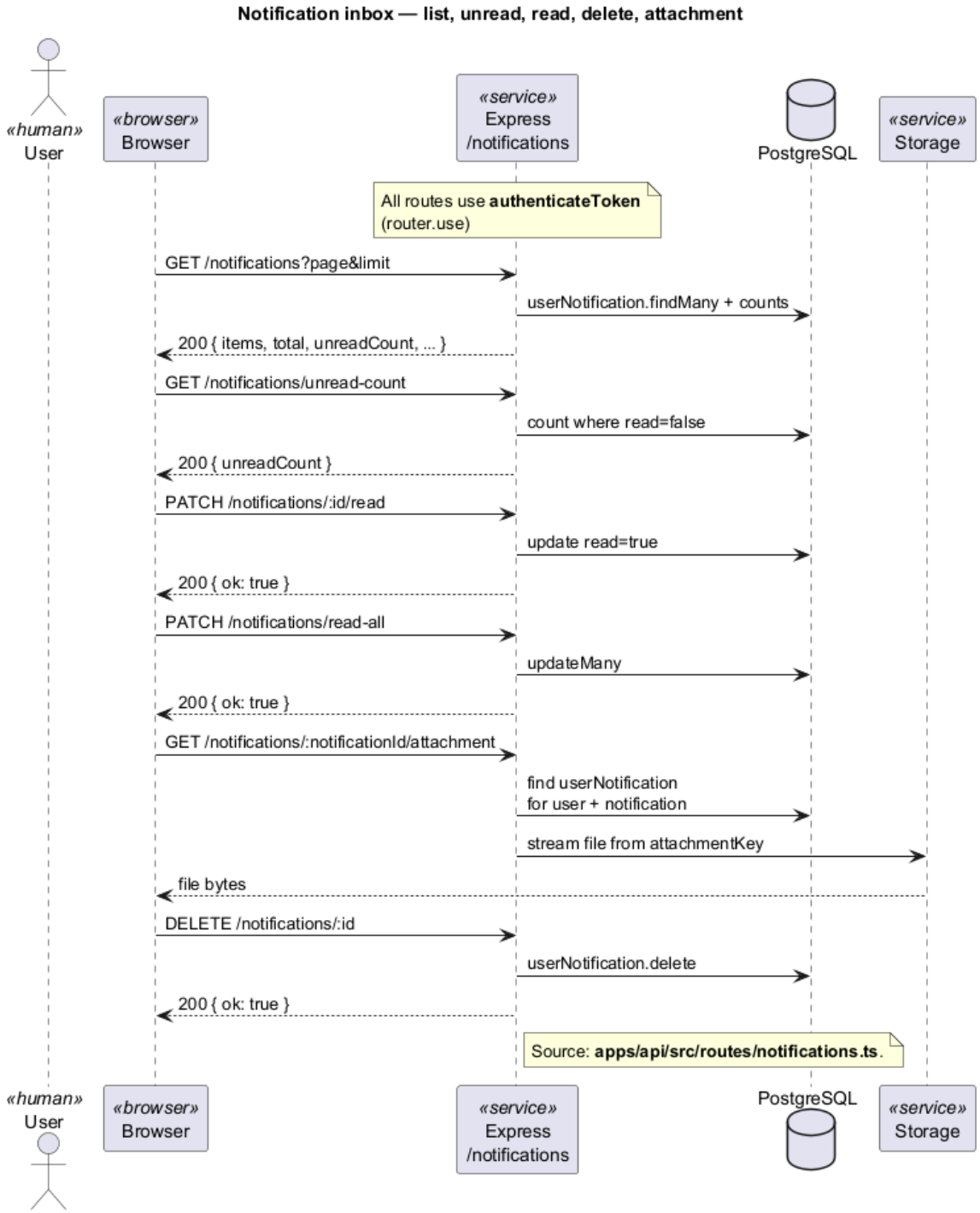

Figure 3.34 – Reading and managing notifications — sequence diagram

Manager workspace

Manager department overview — sequence diagram.

Managers view the departments they oversee and inspect each department's member list to understand team composition.

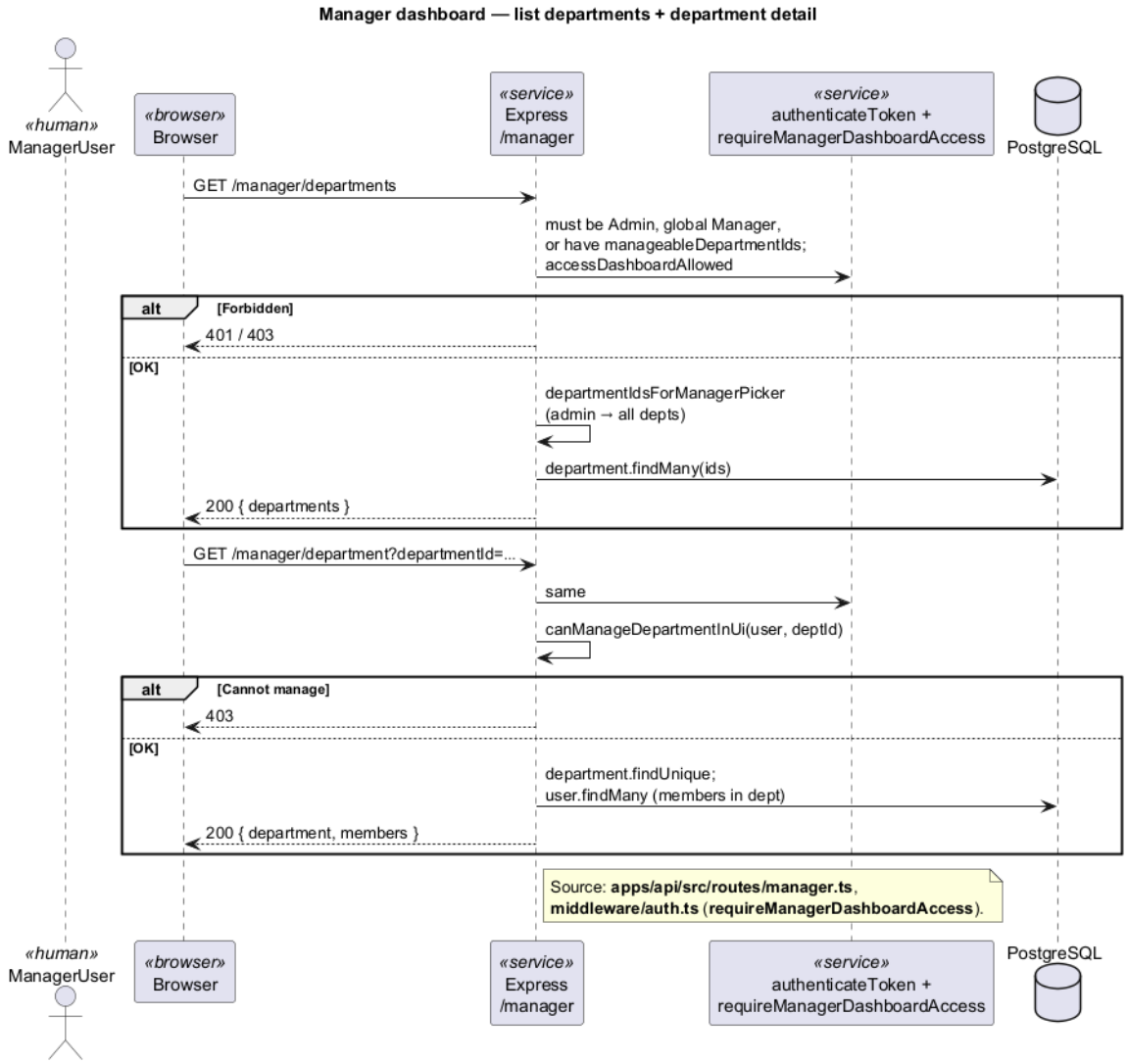

Figure 3.35 – Manager department overview — sequence diagram

Administrator governance

Creating a user account — sequence diagram.

Administrators create accounts with an initial role, department access, and optional feature restrictions. Credentials are shared through organisational channels.

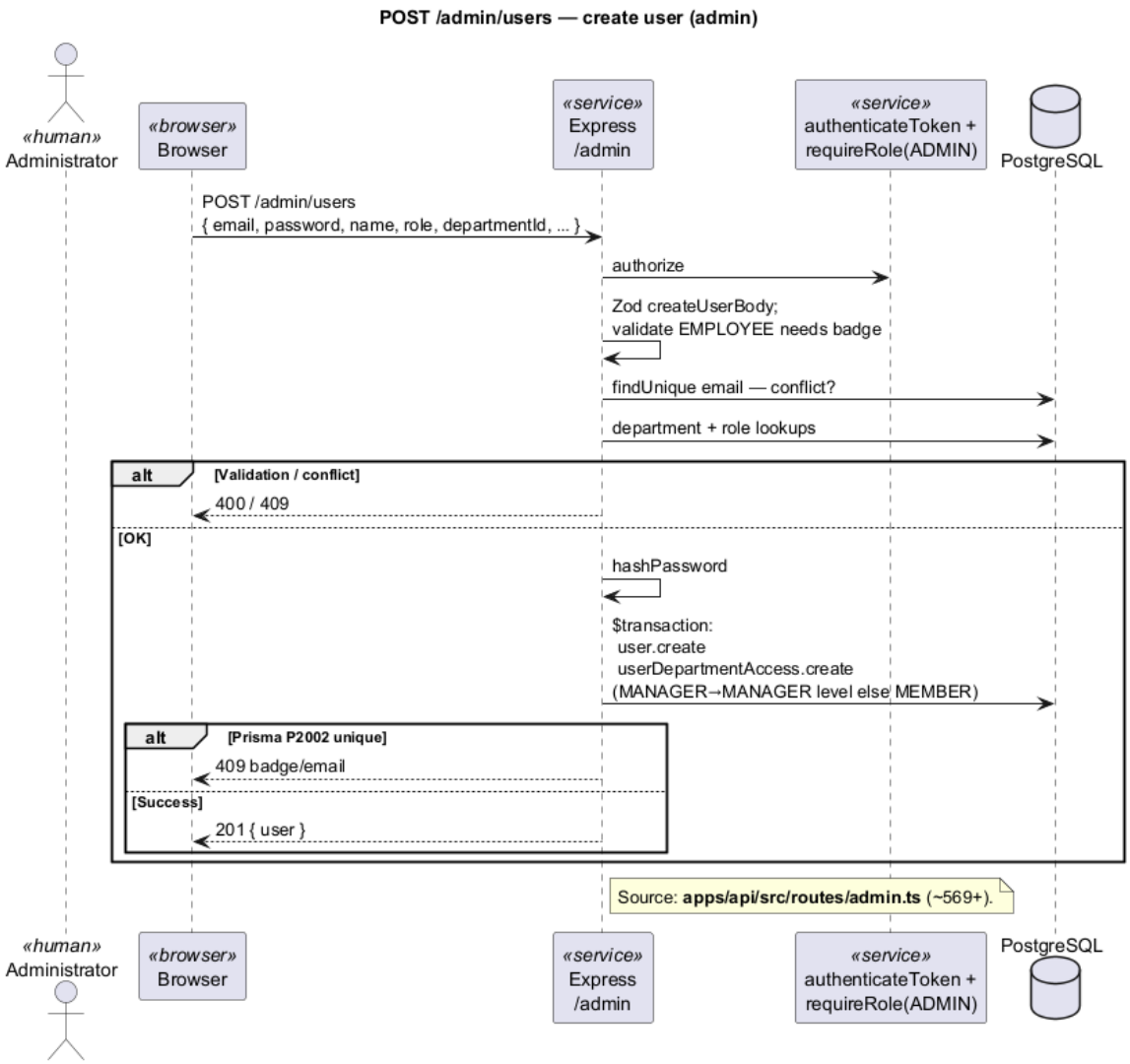

Figure 3.36 – Creating a user account — sequence diagram

Merging two departments — sequence diagram.

Department merge moves documents, access rights, and memberships from one department into another while preserving data consistency.

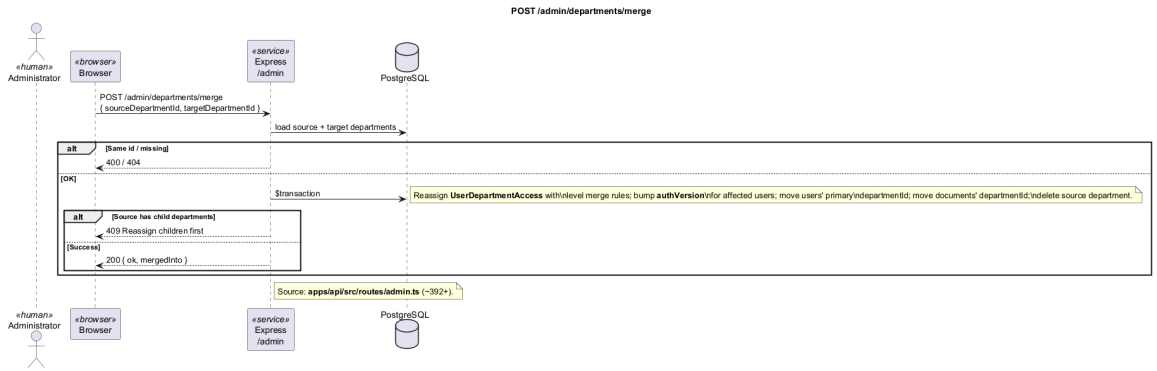

Figure 3.37 – Merging two departments — sequence diagram

Replacing a user's department access in bulk — sequence diagram.

Administrators can replace all department-access entries for one user in a single operation, which simplifies onboarding and access reviews.

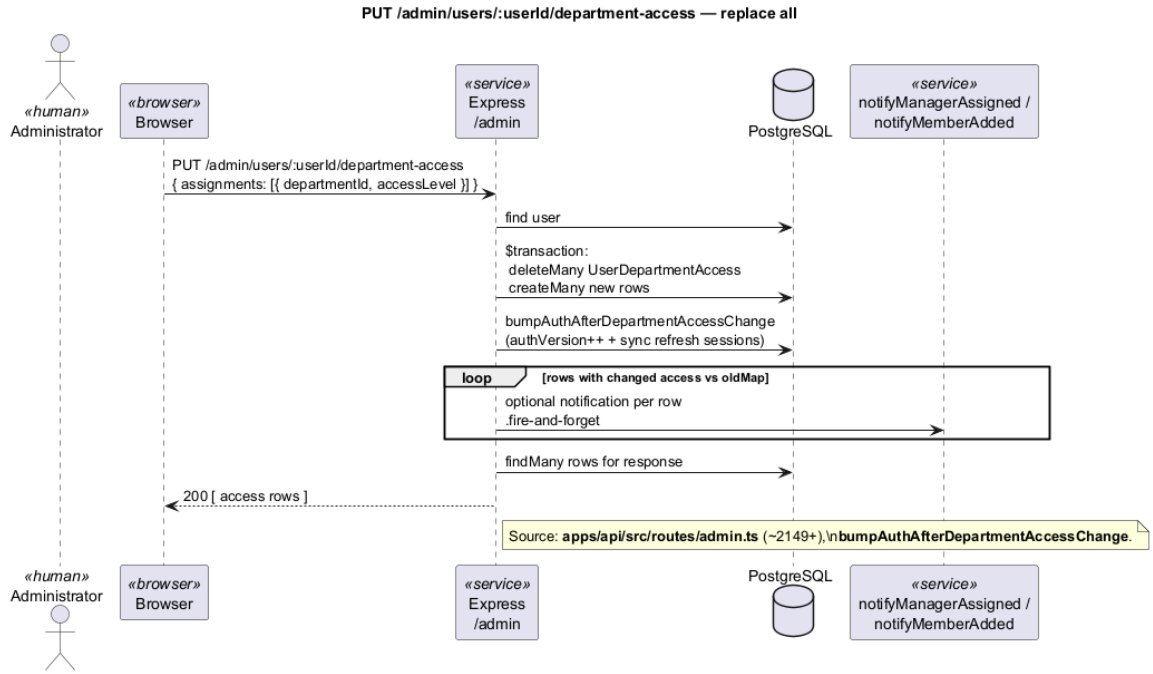

Figure 3.38 – Replacing a user's department access in bulk — sequence diagram

Creating a department — sequence diagram.

Administrators add a new organisational unit with a unique identifier. The operation is logged for audit purposes.

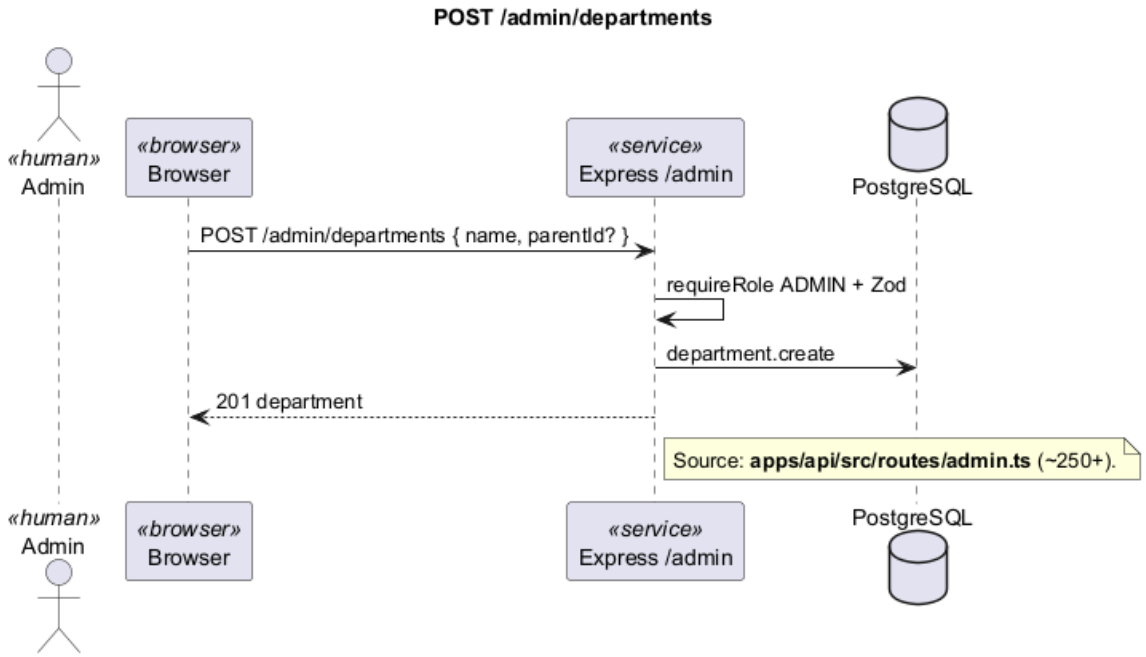

Figure 3.39 – Creating a department — sequence diagram

Granting or revoking single department access — sequence diagram.

Fine-grained actions add or remove access to one department for one user without rewriting the entire access matrix.

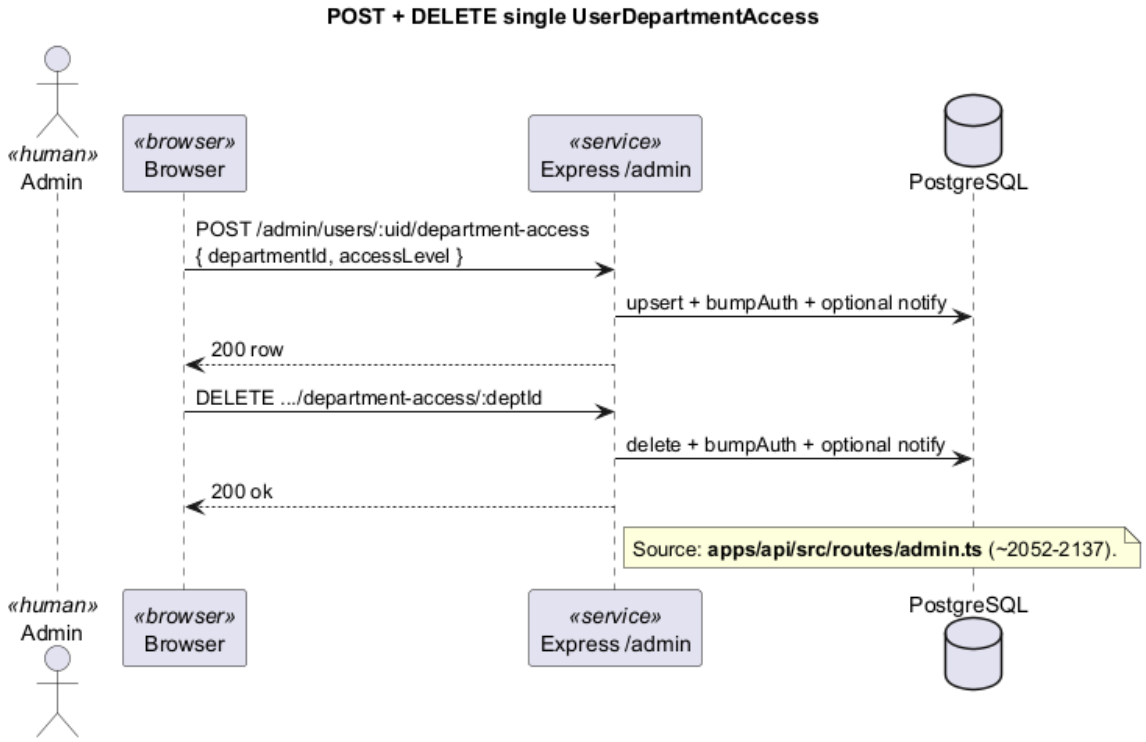

Figure 3.40 – Granting or revoking single department access — sequence diagram

Platform-wide statistics — sequence diagram.

Administrators view high-level indicators such as user count, document volume, conversation activity, and indexing backlog.

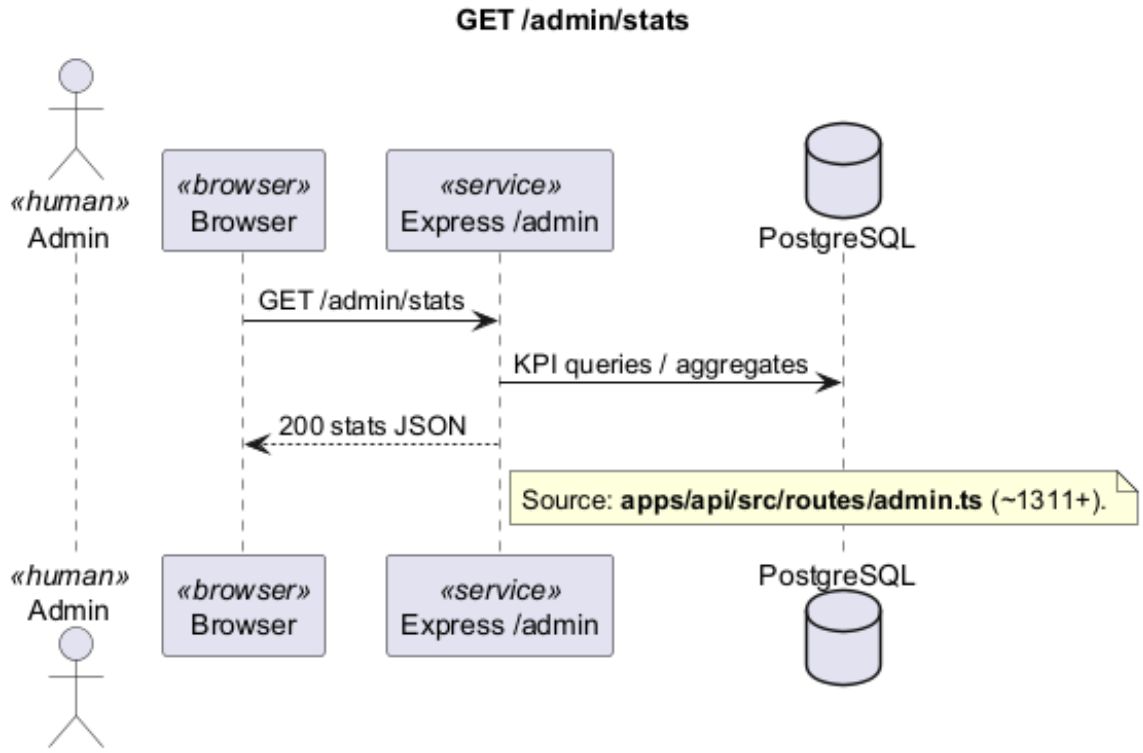

Figure 3.41 – Platform-wide statistics — sequence diagram

Activity log and export — sequence diagram.

Administrators browse recent platform activity and export filtered events to spreadsheet format for offline review.

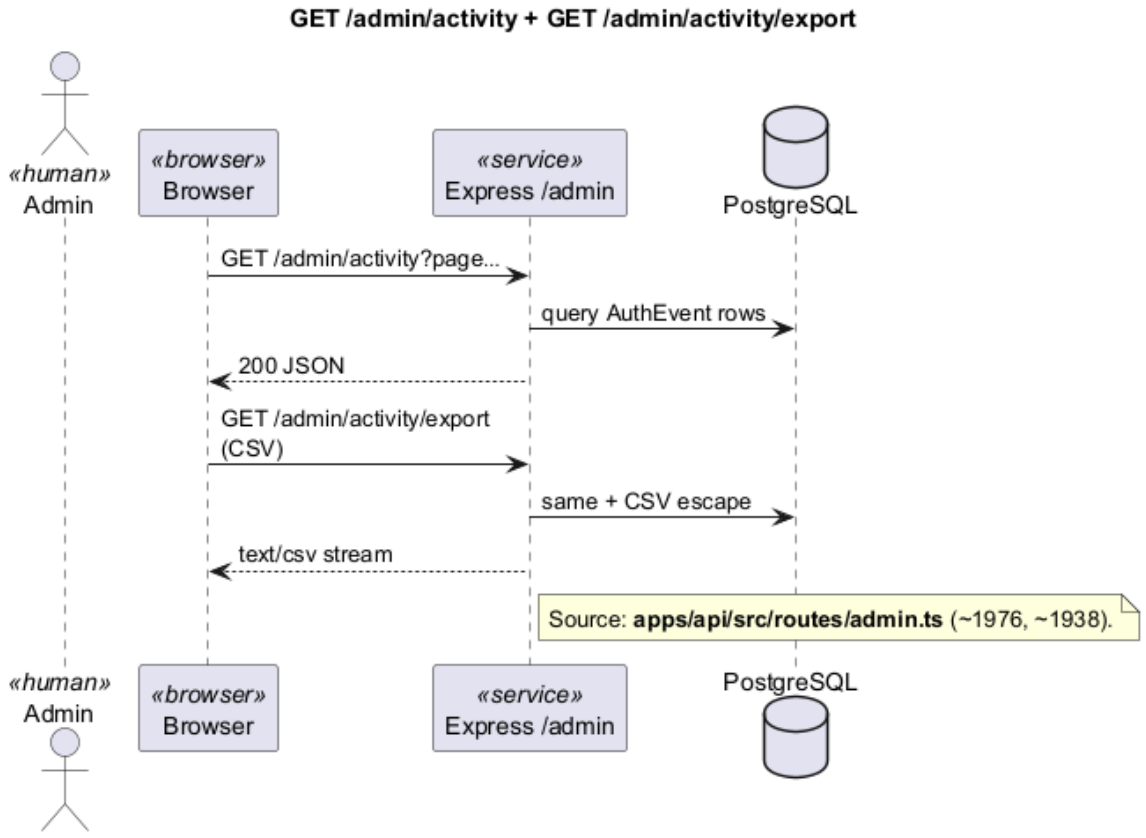

Figure 3.42 – Activity log and export — sequence diagram

Document change audit log — sequence diagram.

Administrators review and export a platform-wide log of document changes (metadata, visibility, and version updates) to support compliance and access reviews.

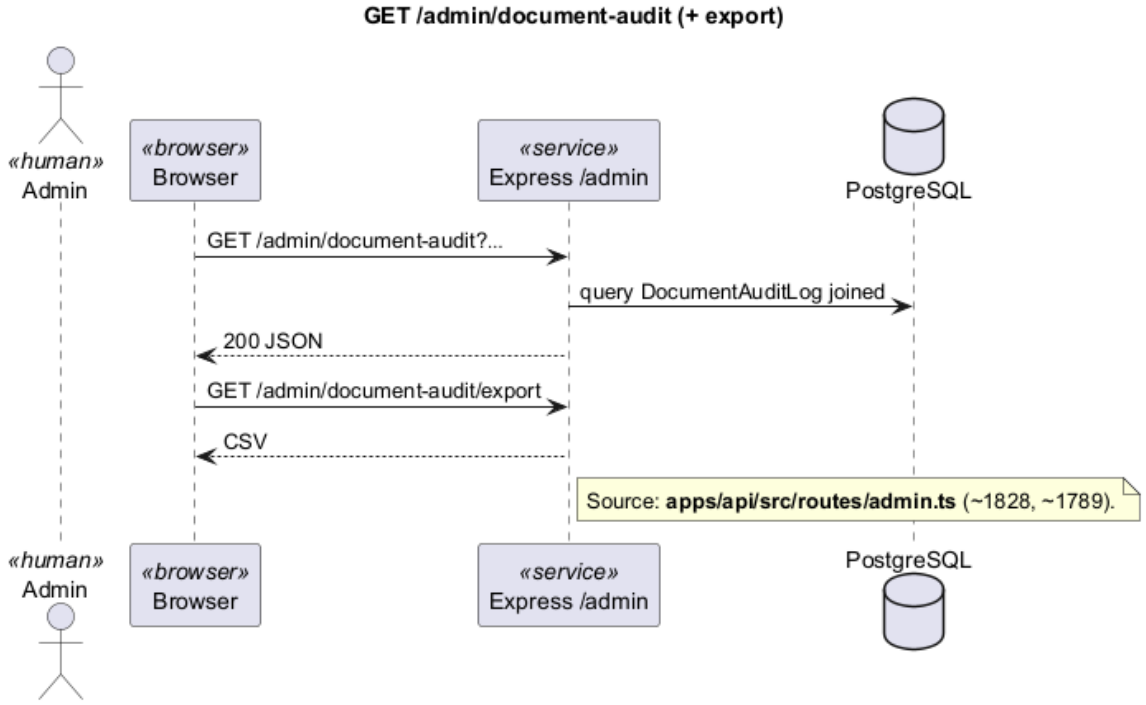

Figure 3.43 – Document change audit log — sequence diagram

Exporting the document catalogue — sequence diagram.

Administrators export the document inventory (titles, departments, tags, and processing status) for reconciliation or reporting.

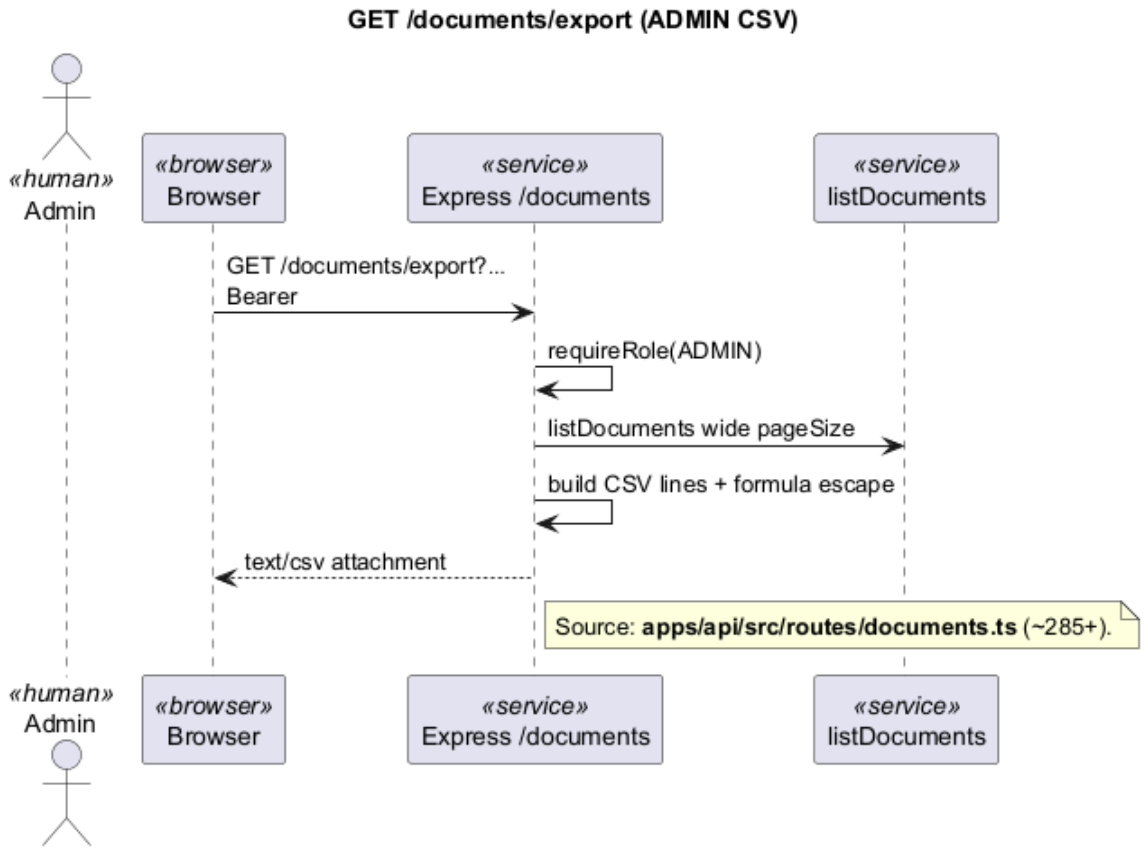

Figure 3.44 – Exporting the document catalogue — sequence diagram

Deleting multiple documents — sequence diagram.

Administrators remove a selected set of documents in one guarded operation, including stored files and related records.

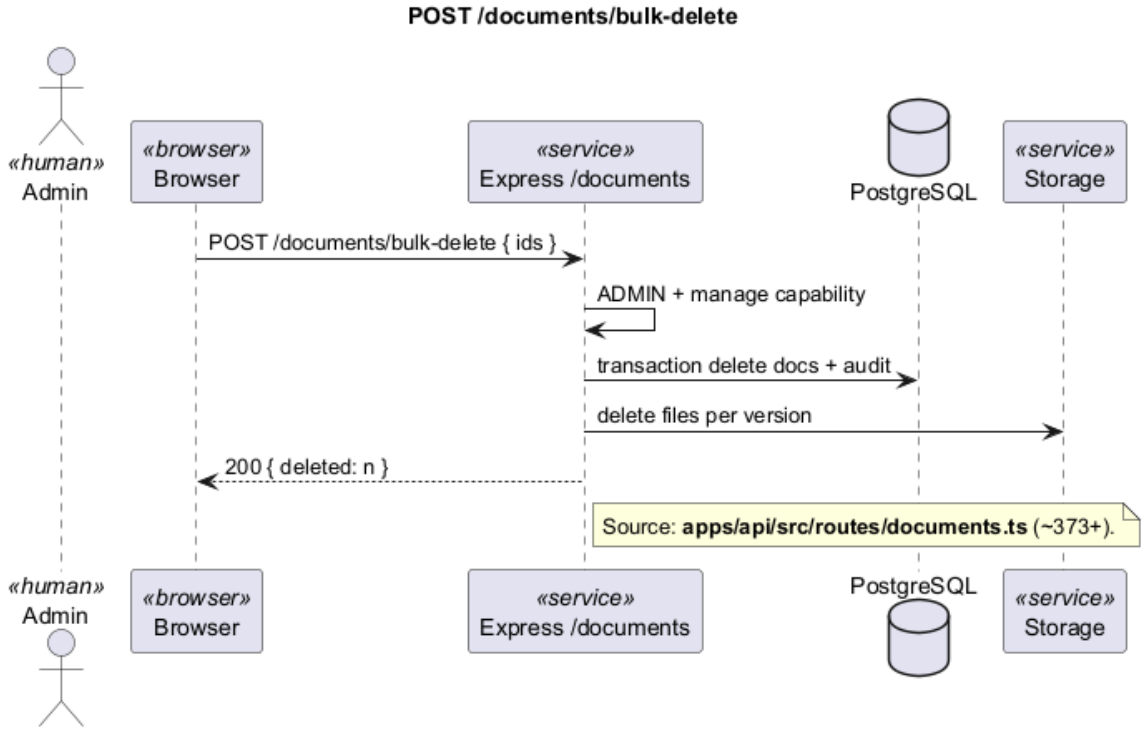

Figure 3.45 – Deleting multiple documents — sequence diagram

Updating or locking a user account — sequence diagram.

Administrators adjust roles, restrictions, or active status. A sign-in lock prevents further authentication attempts for a defined period without affecting unrelated accounts.

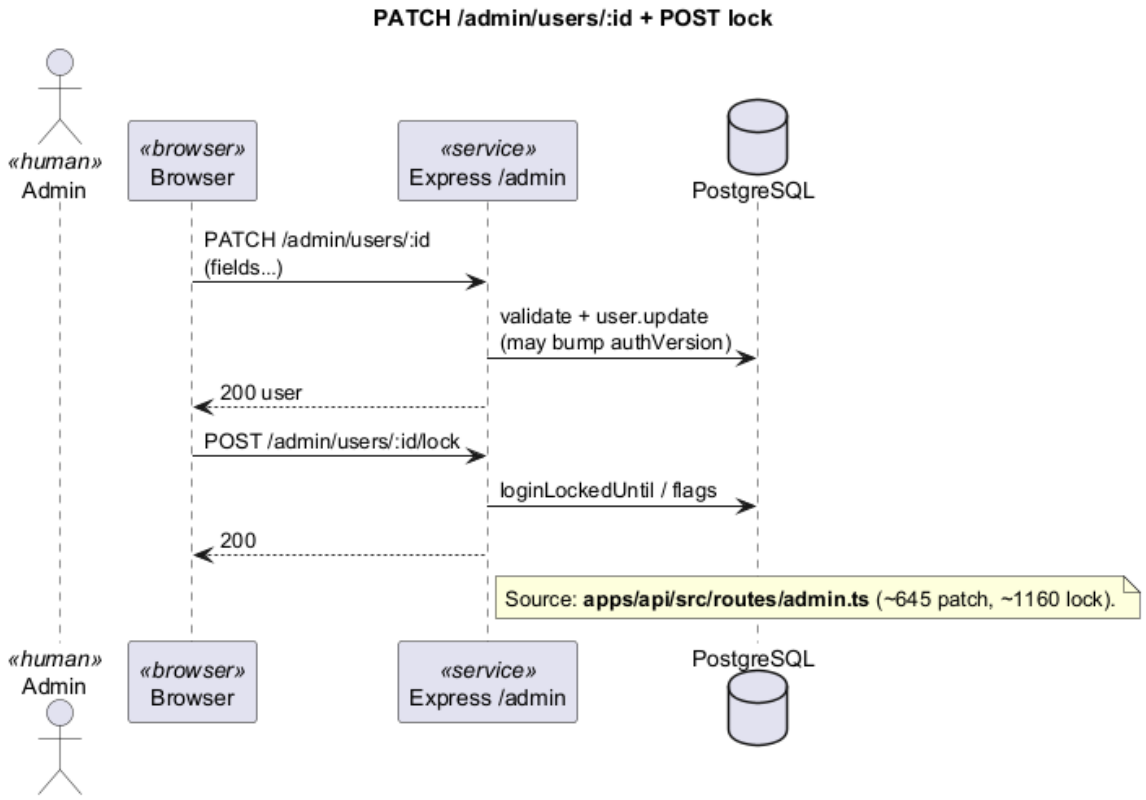

Figure 3.46 – Updating or locking a user account — sequence diagram

3.5.5 Sprint 5: Operations, testing and deployment

Sprint 5 focuses on operability: confirming that the platform is healthy, packaging it for reproducible deployment, and validating behaviour through automated tests and AI quality checks.

3.5.5.1 Sprint 5 structural models: operations, testing and deployment

Sprint 5 use-case diagram.

Figure 3.47 frames Sprint 5 around deployment, health probes, integration testing, and RAG evaluation. Administrator and Developer actors share operational responsibilities, while Operations / CI drives health checks. The diagram aims to show that delivery readiness is a designed capability set, not only an implementation afterthought.

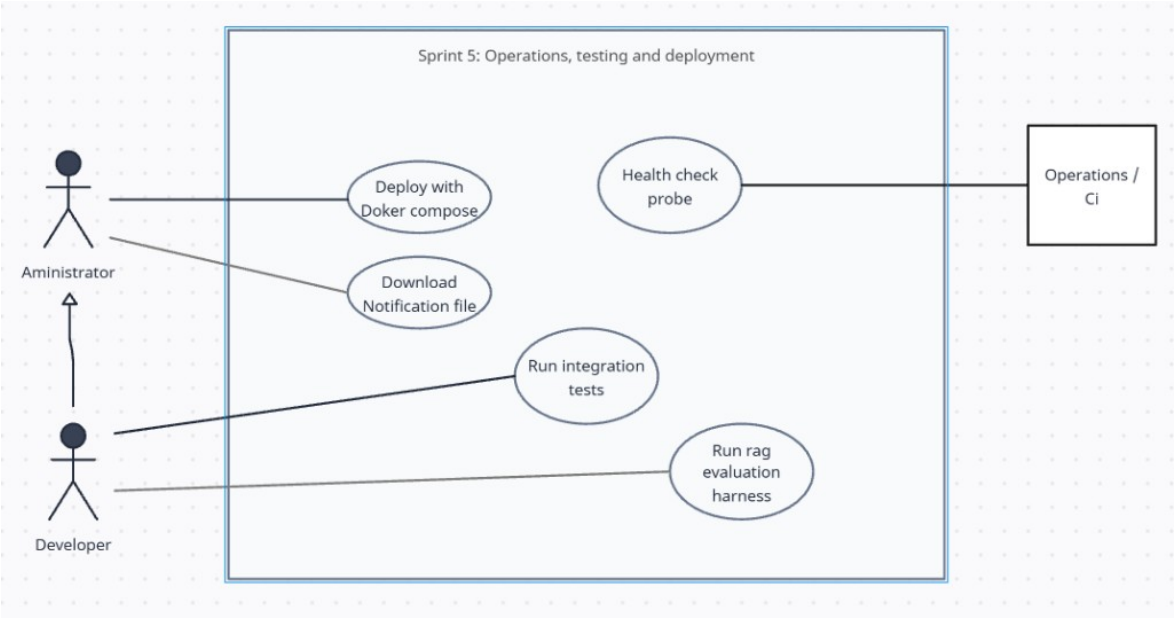

Figure 3.47 – Sprint 5 use-case diagram — Operations, testing and deployment

Sprint 5 domain class diagram.

Figure 3.48 isolates the lightweight operational artefacts used in this sprint—health and readiness indicators, deployment composition, and evaluation or test run metadata where modelled. It is intended to connect operability concerns to the same UML vocabulary used in earlier sprints, even when most behaviour is exercised outside the interactive product UI.

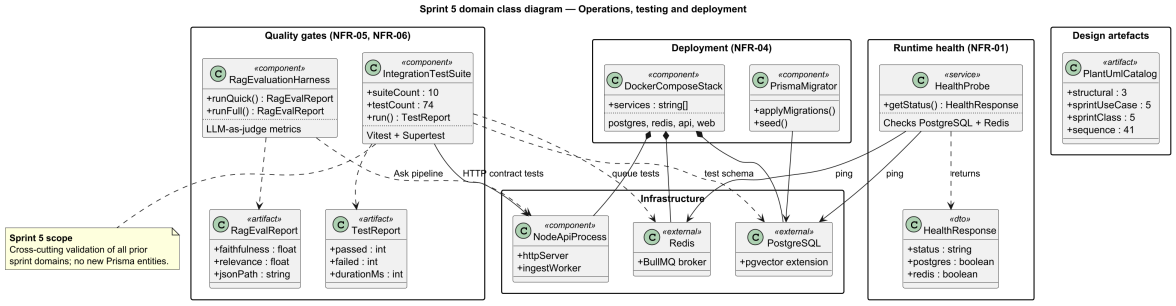

Figure 3.48 – Sprint 5 domain class diagram — Operations, testing and deployment

3.5.5.2 Sprint 5 interaction sequences: health and operational flows

Service health check — sequence diagram.

Monitoring tools call a public health endpoint to verify that the application is running and can reach its database and cache services.

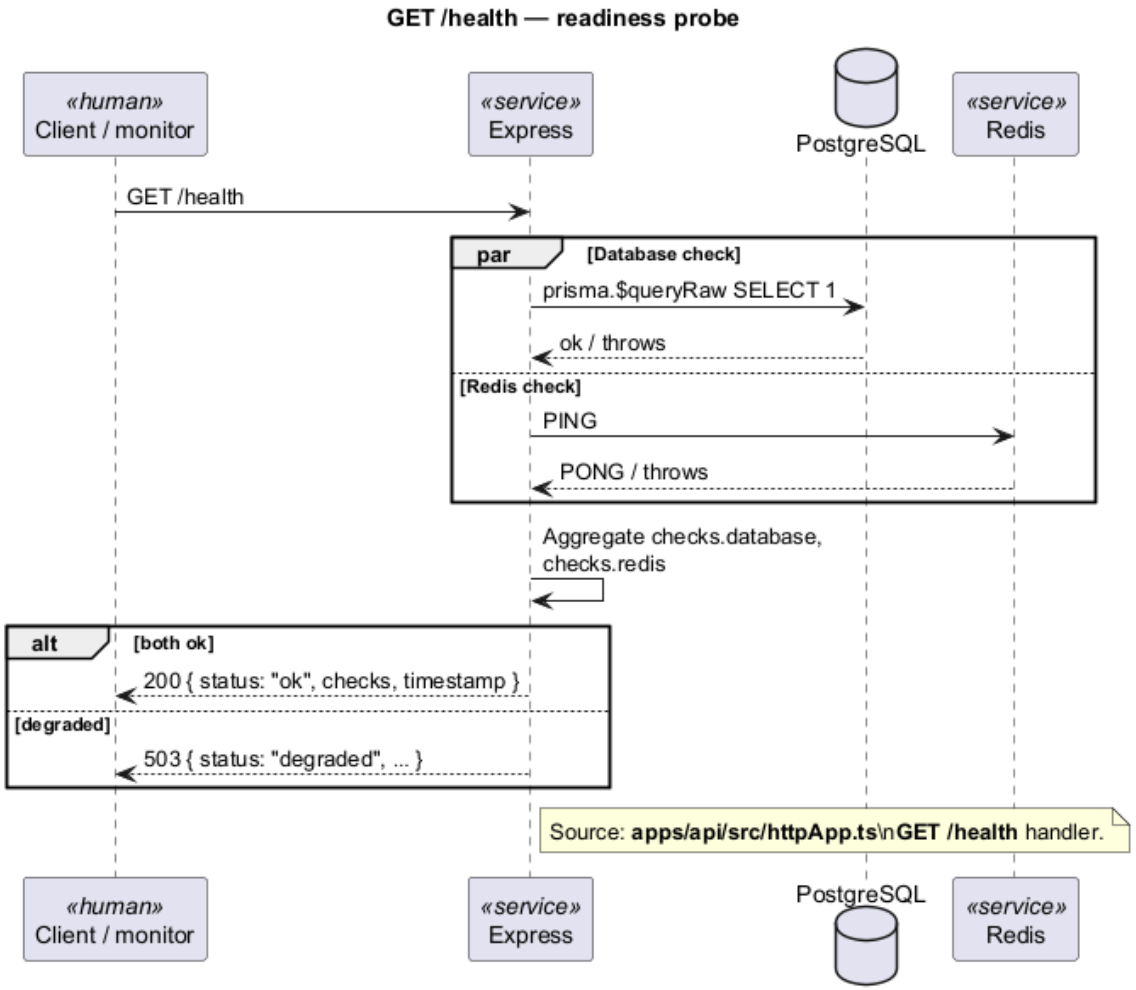

Figure 3.49 – Service health check — sequence diagram

3.5.5.3 Operational deliverables

Integration test suite.

An automated test suite exercises the main user journeys (authentication, documents, search, conversations, administration, and manager views) so regressions are detected before release.

Containerised deployment.

The platform is packaged with its database, cache, API, and web application so the full environment can be started consistently on any workstation or in continuous integration.

RAG quality evaluation.

A repeatable evaluation workflow runs predefined questions through the Ask pipeline and scores answer quality, producing reports that can be compared across development iterations.

3.6 Chapter Conclusion

This chapter set out the structure of the Knowledge Platform and how its parts interact at design time. The three-tier layout separates user-facing workflows from business coordination and from persistence and AI processing, while subsystem boundaries make ownership clear for authentication, documents, search, conversations, and governance. The stack choices favour a pragmatic TypeScript setup that supports streaming answers, hybrid retrieval, and manageable deployment for a prototype-scale enterprise system. Sprint-scoped interaction diagrams show how concrete user journeys run across the web application, API services, and background workers. Those design artefacts form an implementable blueprint whose adequacy is checked in Chapters 4 and 5.

CHAPTER 4

Implementation

This chapter explains how the sprint-aligned design of Section 3.5 (Chapter 3) became working software. Delivery followed the five increments of Table 2.5 in order, each building on the previous one. For every sprint the work tracked the matching design subsection, validated the principal flows, and embedded representative UI screenshots where they help. The result is a monorepo with a secured API, a document library with asynchronous ingest, hybrid RAG, governance consoles, and the operational tooling described below.

4.1 Hardware and Software Environment

Development ran on a Windows 10/11 workstation with Node.js 20+ as the primary runtime. The application is organised as a monorepo with separate API and web packages, orchestrated from the repository root for local development, database migration, seeding, linting, and RAG evaluation. Table 4.1 summarises the languages, frameworks, data services, and tooling used in implementation.

Table 4.1 – Development and deployment stack (detailed)

Category	**Technology and role**	**Version**
Workstation	Windows 10/11; Git; Docker Desktop; VS Code / Cursor IDE	—
Languages	**TypeScript** (strict typing across API and web); **JavaScript** (ES modules, Node runtime); **SQL** (PostgreSQL migrations and analytics queries)	5.9
Web frameworks	**Next.js** 15 (App Router, SSR/CSR hybrid, route handlers); **React** 19 (client components, hooks); **CSS Modules** + design tokens (light/dark themes)	15 / 19
API layer	**Express** 4 REST API; **tsx** dev runner; **JWT** + HttpOnly refresh cookies; **Multer** uploads; **Helmet**/**CORS**/rate limiting	4.21
Data access	**Prisma** 5 ORM (29 migrations, seed script); **Zod** request validation; **bcryptjs** password hashing	5.22
Persistence	**PostgreSQL** 16 with **pgvector** (768-d embeddings); full-text search indexes for hybrid retrieval	16
Async processing	**Redis** 7 job broker; **BullMQ** 5 ingest worker queue (text extraction, chunking, embedding)	7 / 5
AI services	**Google Gemini** — `gemini-2.5-flash` (generation/rerank), `gemini-embedding-001` (vectorisation); `@google/generative-ai` SDK	API
Deployment	**Docker Compose** (dev + production compose files); multi-stage Node 22 Alpine images for API and web	Compose
Quality assurance	**Vitest** 3 + **Supertest** integration tests (74 cases); **ESLint** + **Prettier**; RAG evaluation CLI	3.2

4.2 Implementation Steps by Sprint

4.2.1 Sprint 1: Authentication and access control

Sprint 1 had to establish a secure perimeter before any document or AI feature could be exposed. Following Subsection 3.5.1, the web client keeps short-lived access tokens in memory (never in `localStorage`), stores refresh tokens in HttpOnly cookies, allows only one in-flight refresh to avoid parallel rotations, and relies on middleware that enforces roles plus per-user restriction flags for the document library and Ask. Profile avatars are served from a CORP-aware public route so browsers can embed them across origins. The sprint delivers a complete sign-in, recovery, profile, and role-based navigation shell, with integration tests confirming flows 3.5–3.10.

4.2.1.1 Implemented web interfaces

The Sprint 1 web client covers credential-based sign-in, password recovery, profile management, and role-based home hubs. Error and restriction states are validated manually in Section 5.3.4.

Authentication pages.

Sign-in and password recovery are implemented as static Next.js routes. Figure 4.1 shows the production sign-in layout; Figure 4.2 shows the recovery workflow.

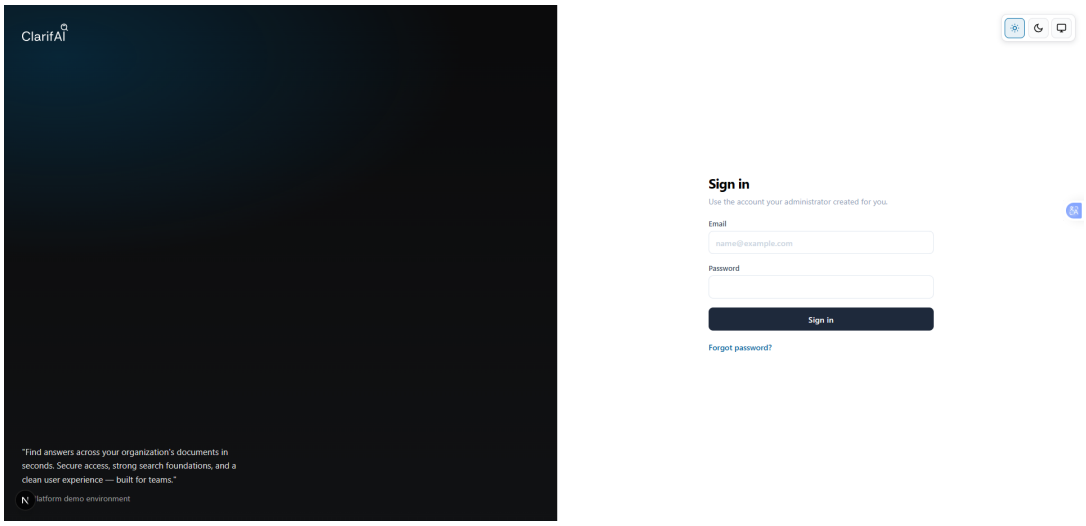

Figure 4.1 – Knowledge Platform sign-in page with email, password, and theme toggle

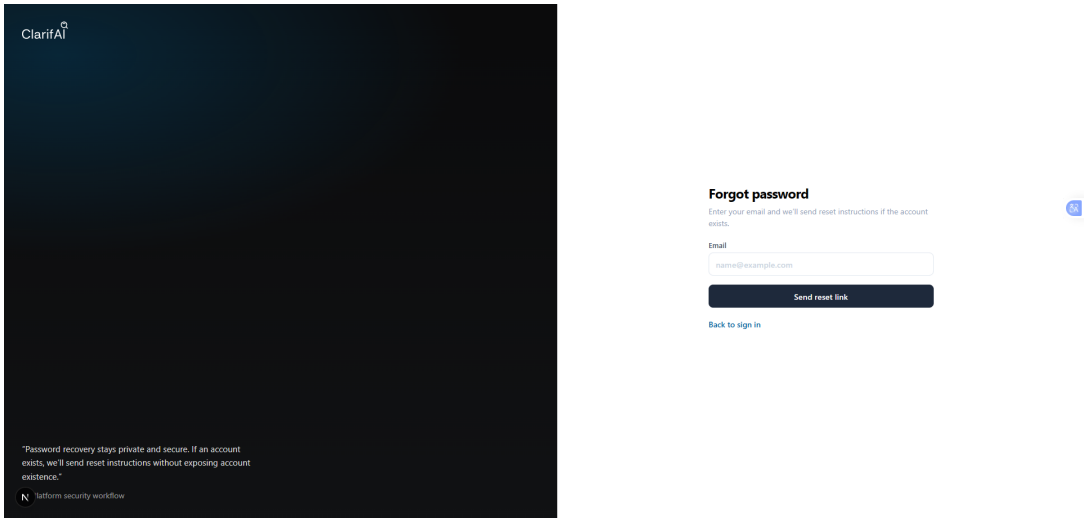

Figure 4.2 – Password recovery request form with email field and submit action

Profile management.

Figure 4.3 shows the profile settings page: avatar upload, editable contact fields, read-only role and department (set by administrators), and inline change-password controls.

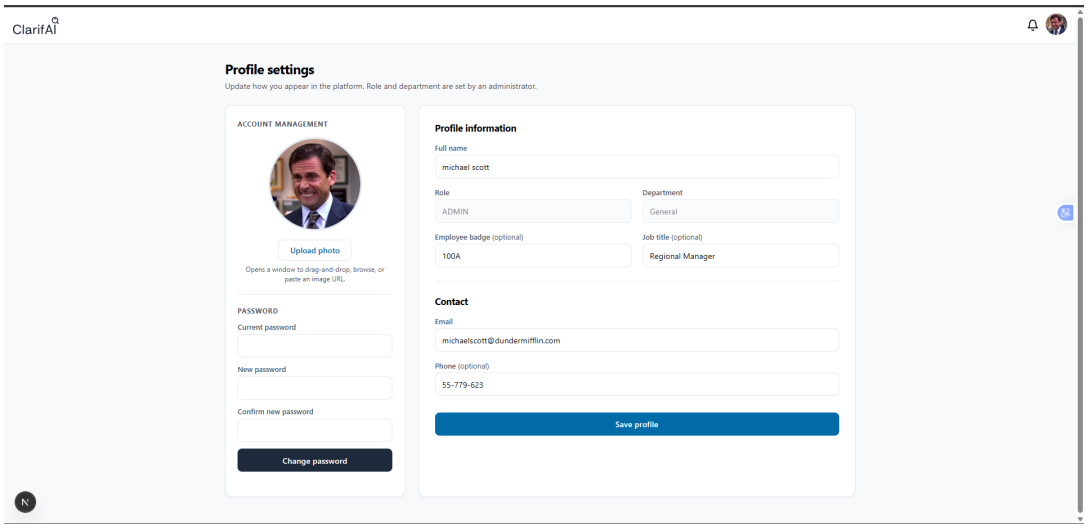

Figure 4.3 – Profile settings with avatar upload, read-only role and department, and change-password form

Role-based home hubs.

After authentication, users land on a tile-based hub whose options depend on platform role. Employees see Documents and Ask (Figure 4.4); managers also see Department

overview (Figure 4.5); administrators see the full four-tile hub including Administration (Figure 4.6).

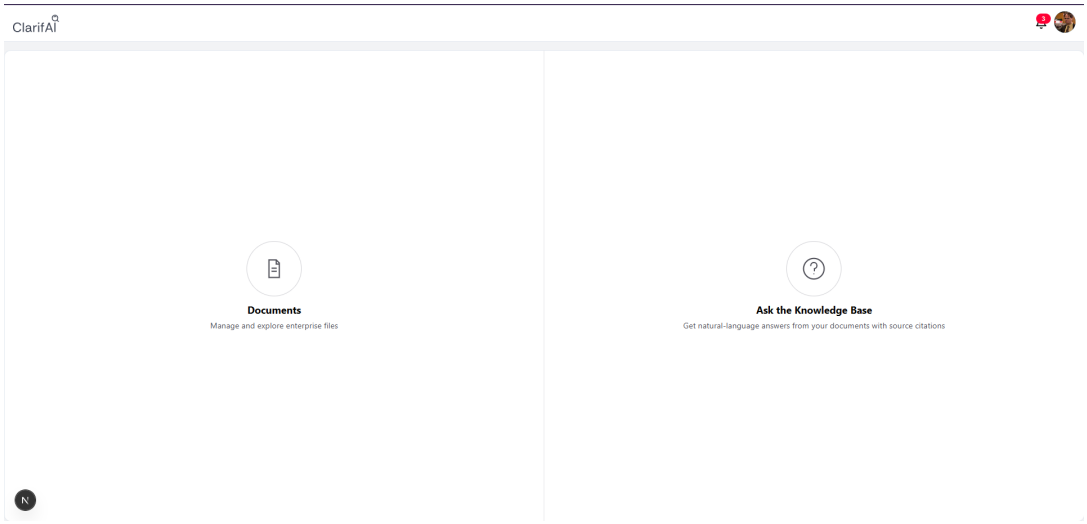

Figure 4.4 – Employee landing page with Documents and Ask tiles

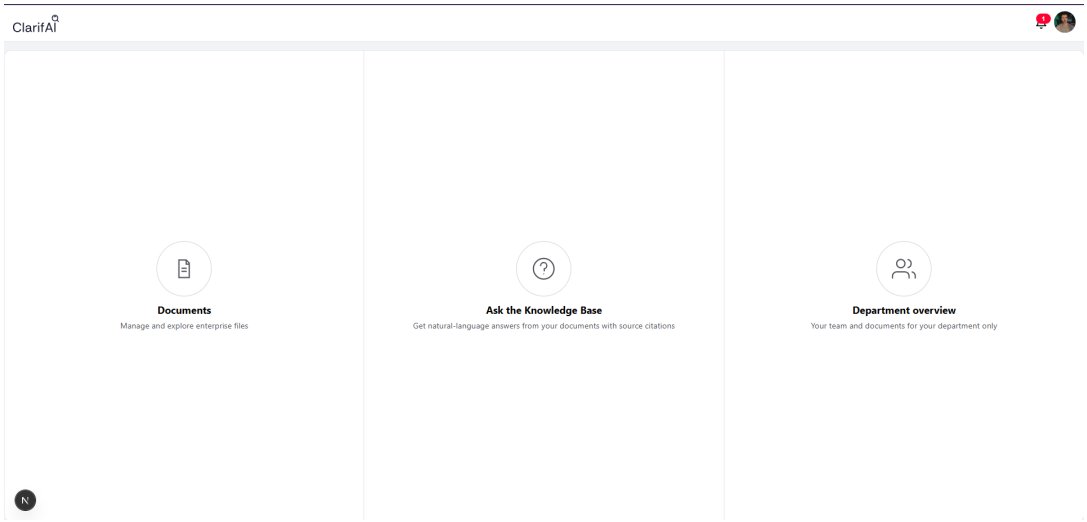

Figure 4.5 – Manager landing page with Department overview tile

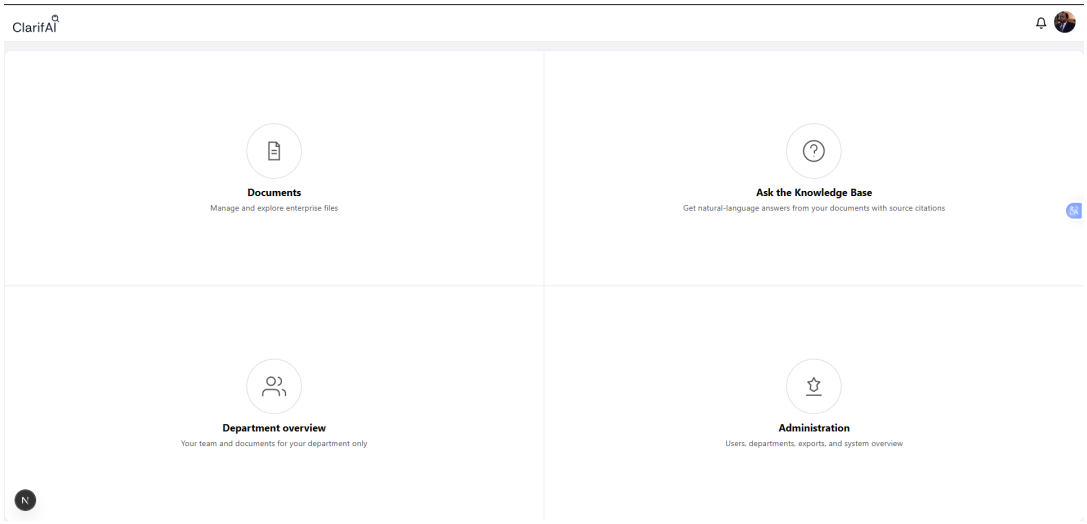

Figure 4.6 – Administrator landing page with Administration tile

Theme switching.

Figure 4.7 shows the same administrator hub in dark mode, confirming that the CSS theme tokens apply consistently across navigation tiles.

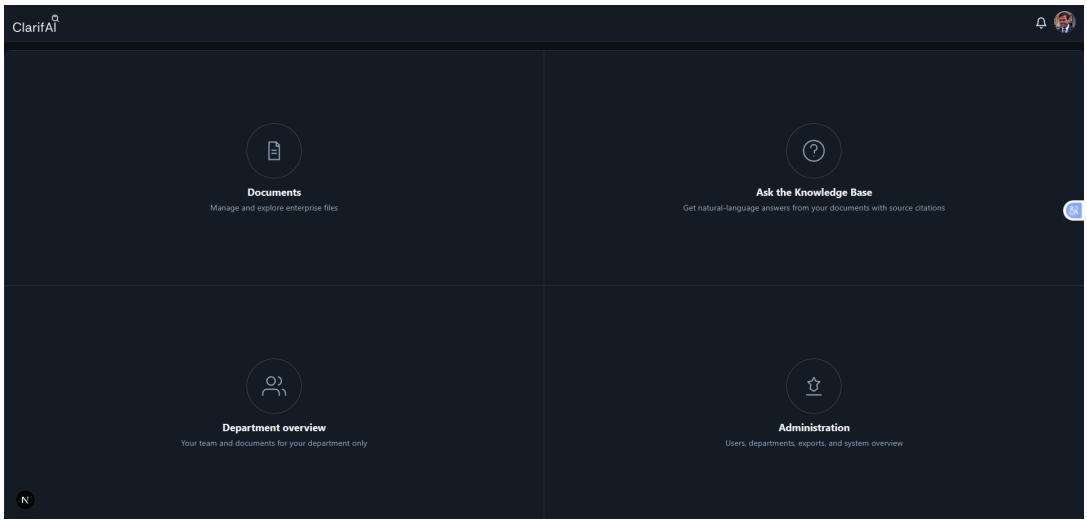

Figure 4.7 – Administrator home hub in dark mode showing consistent theme tokens

4.2.2 Sprint 2: Document library and ingest pipeline

Sprint 2 made organisational knowledge uploadable, searchable, and subject to visibility rules. Uploads pass MIME and size checks, are stored on disk, and are enqueued for asynchronous processing. The worker extracts text per format, chunks content with sentence-aware splitting and table preservation, computes 768-dimensional embeddings,

and persists indexed chunks. Visibility (`ALL`, `DEPARTMENT`, `PRIVATE`) is enforced so unauthorised callers receive a neutral 404 rather than revealing that a document exists. Tags, favourites, recents, and versioning complete the library. Integration tests cover document flows 3.13–3.16 and 3.17–3.15.

4.2.2.1 Implemented web interfaces

The document library lets users browse department-scoped collections, inspect file details, preview documents in the browser, and manage versions.

Library browse and file details.

Figure 4.8 shows the department card view with sidebar navigation (Recent, Favourites, Archived) and filter controls. Figure 4.9 shows the file detail side panel with ingest status, visibility, and action buttons (Open, Download, Favourite, Versions).

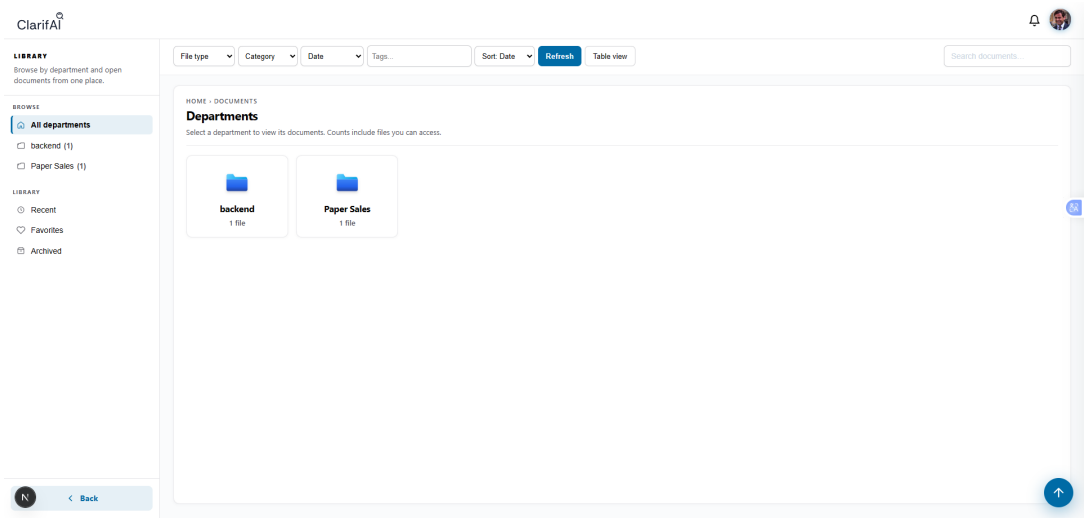

Figure 4.8 – Department library browse with sidebar navigation and filter controls

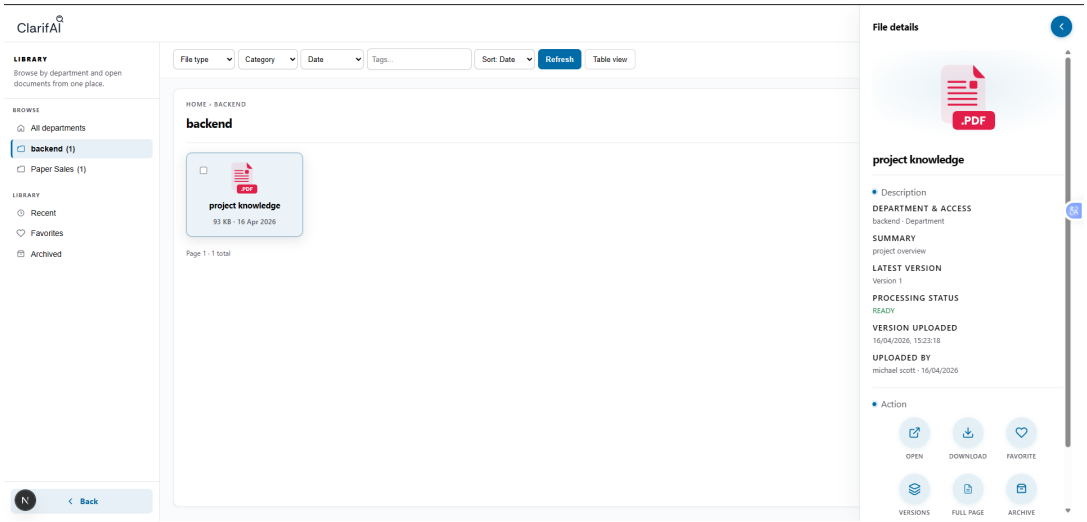

Figure 4.9 – Document detail side panel with metadata and action buttons

Upload workflow.

Managers and administrators open a multi-step upload modal (file, metadata, tags) from the library FAB (Figure 4.10), matching the upload sequence of Figure 3.13.

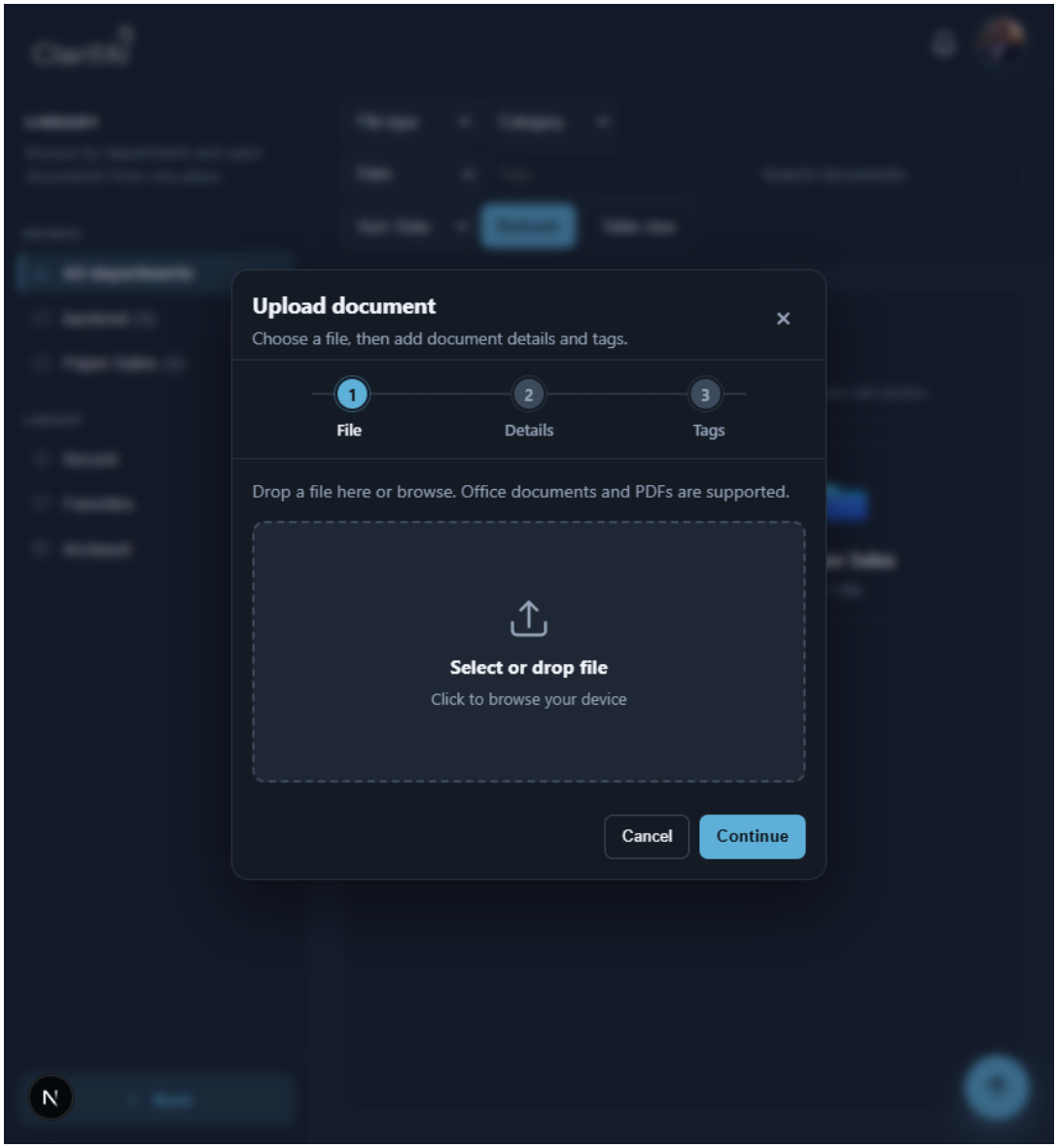

Figure 4.10 – Multi-step upload modal with file drop zone and metadata fields

Recent, favourites, and table view.

Sidebar scopes filter the library to recently opened files (Figure 4.11) or starred favourites (Figure 4.12). Users can switch between card and tabular layouts (Figure 4.13).

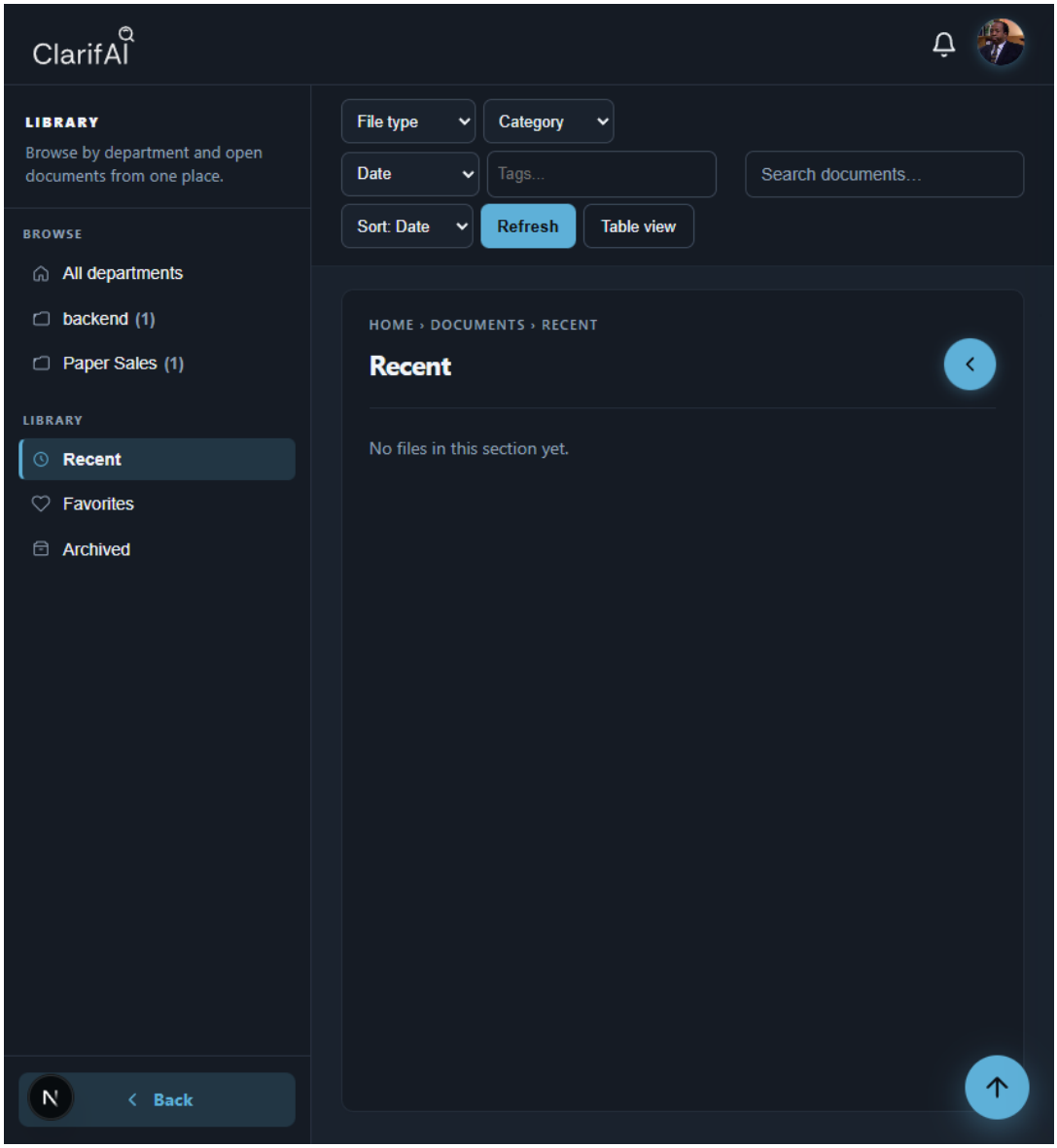

Figure 4.11 – Recent documents scope with sidebar highlight

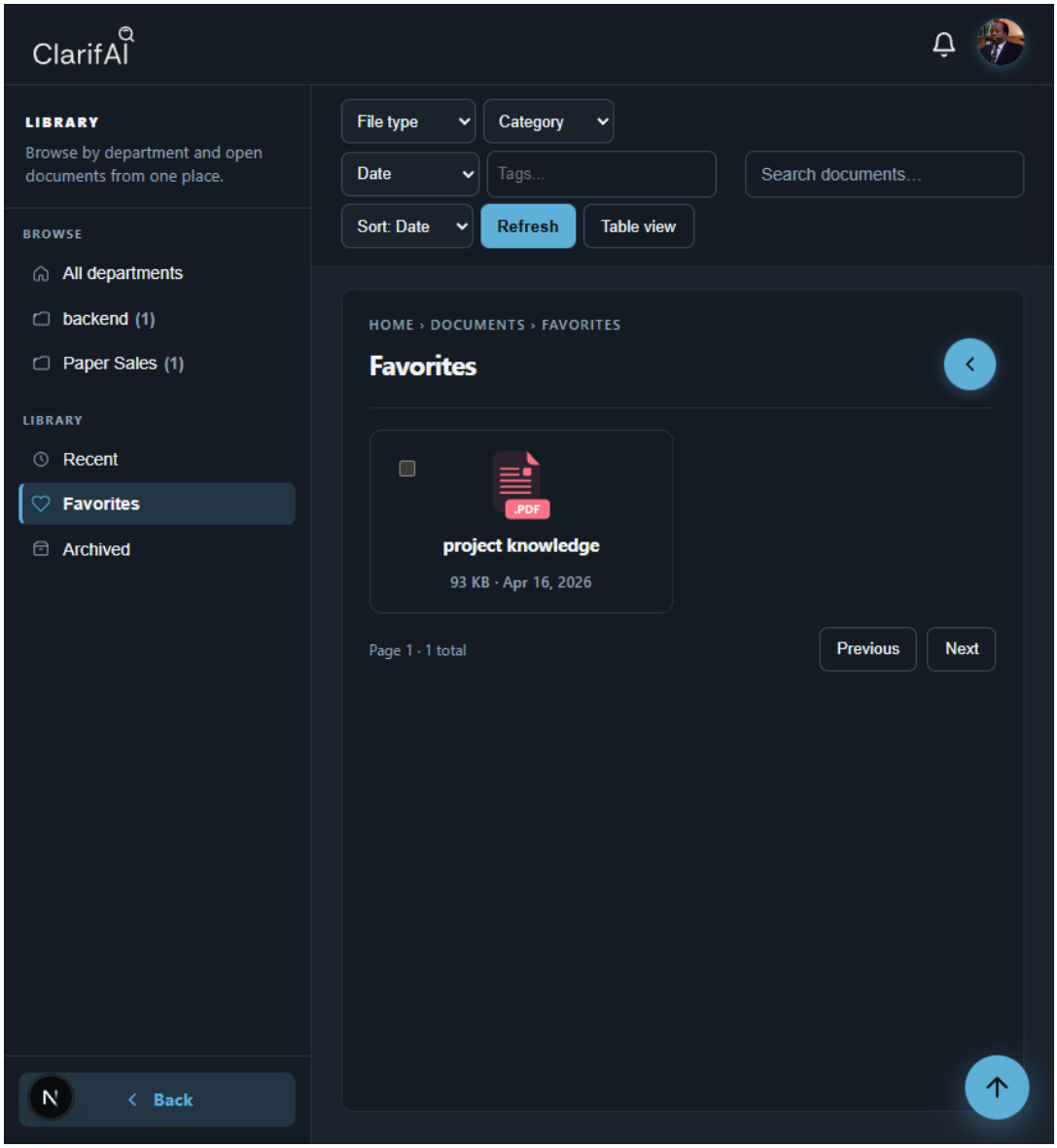

Figure 4.12 – Favourites scope listing starred documents

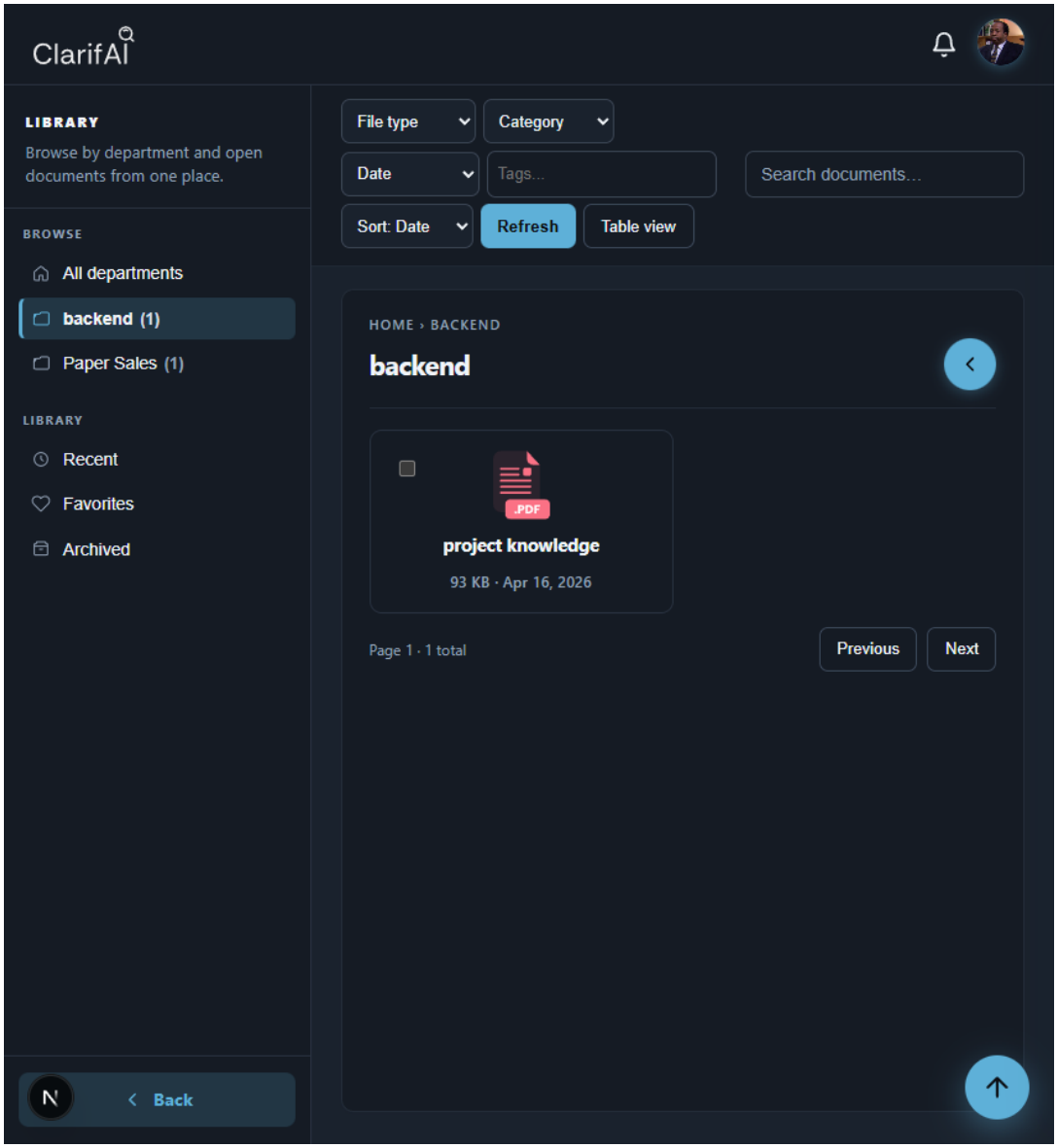

Figure 4.13 – Department library in table layout with column filters

Ingest processing status.

The detail panel shows asynchronous worker progress with a **Processing** badge and progress bar before the document reaches **READY** (validated end state in Chapter 5).

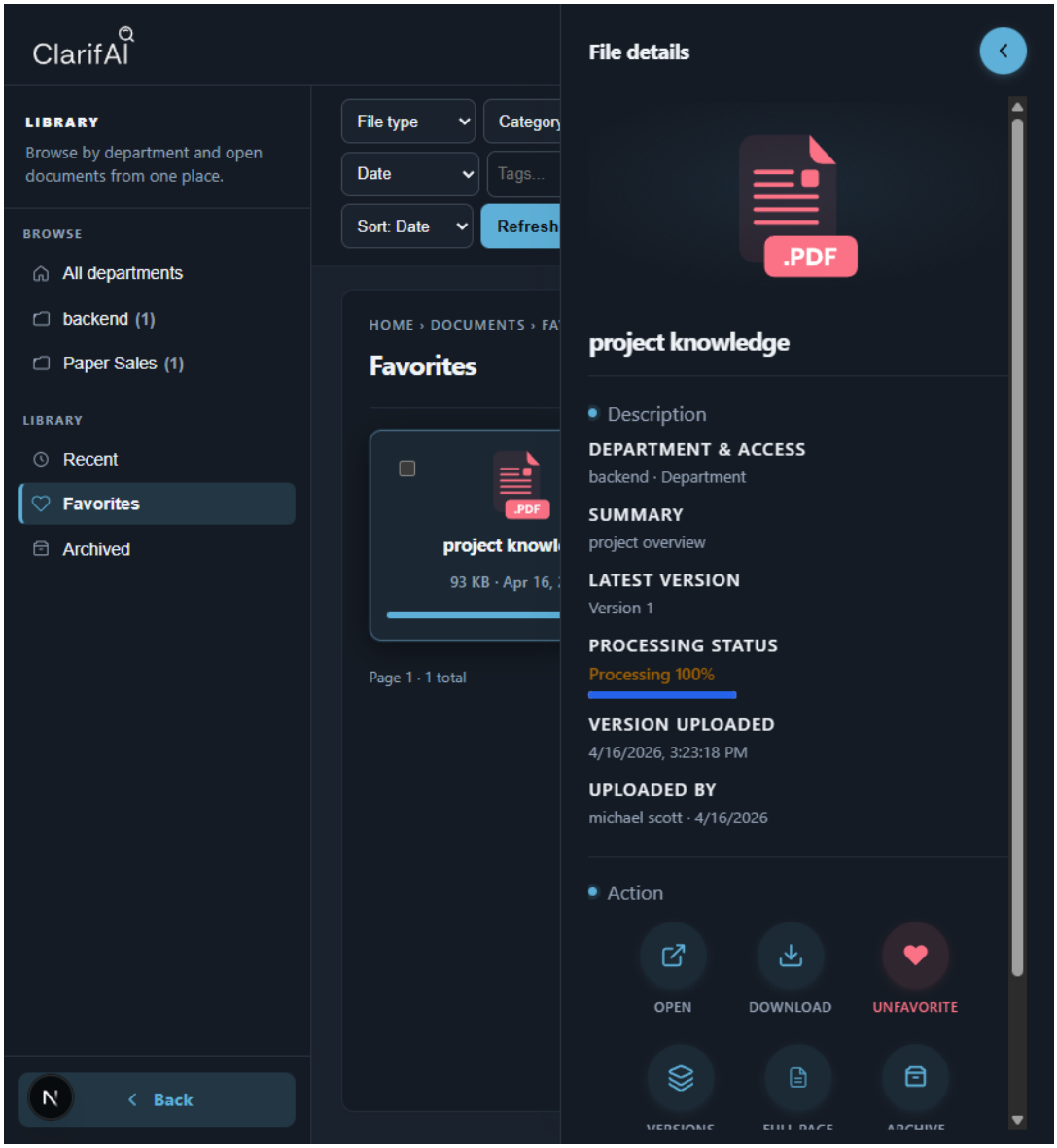

Figure 4.14 – File detail panel during ingest with processing badge and progress bar

Preview and versioning.

Authorised users open PDFs in an in-browser preview modal (Figure 4.15). Managers and administrators can upload new versions and inspect the version archive from the Versions action (Figure 4.16).

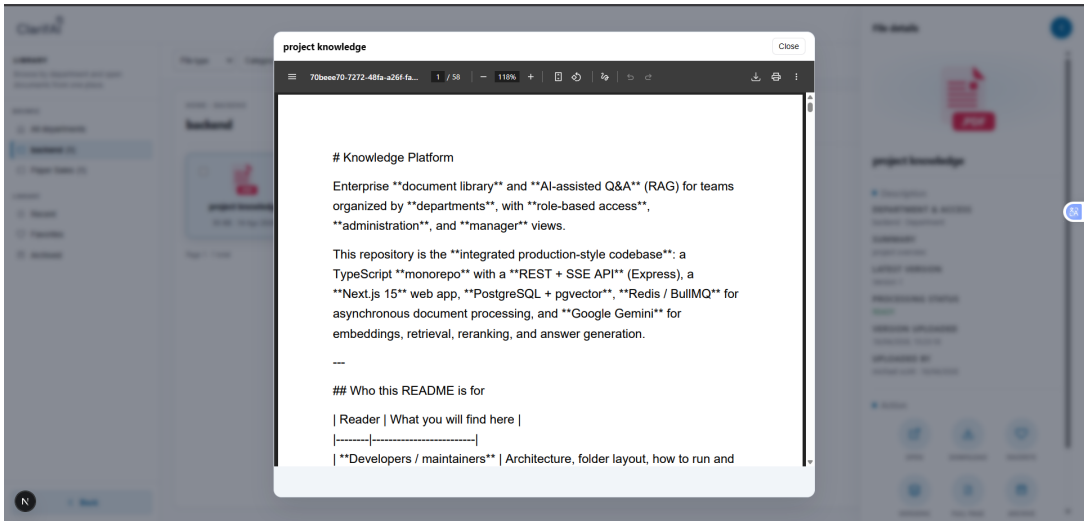

Figure 4.15 – In-app PDF preview modal with navigation controls

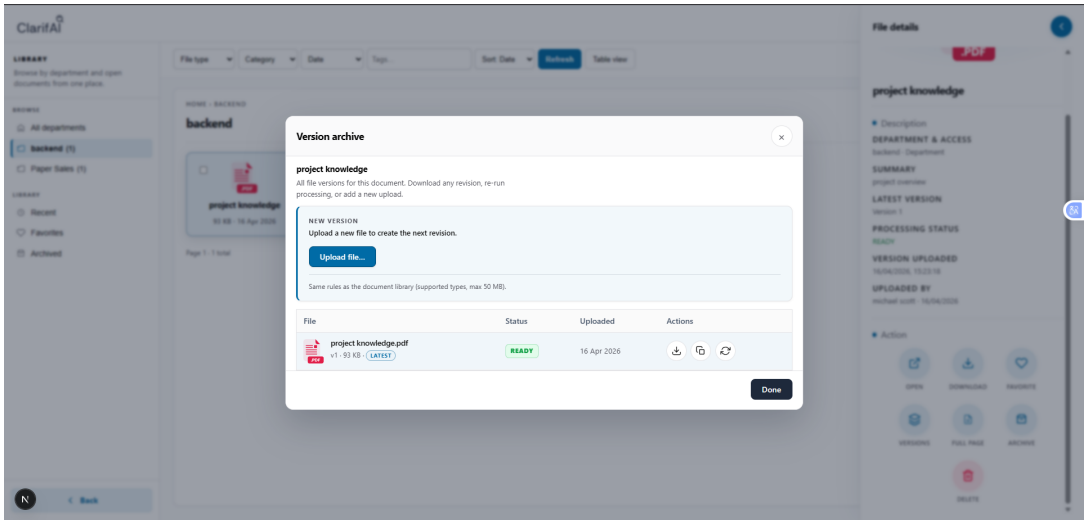

Figure 4.16 – Version archive dialog with upload and reprocess actions

4.2.3 Sprint 3: Search, RAG and conversations

Sprint 3 let authorised users find information semantically and receive cited, streaming answers grounded in their accessible corpus. Hybrid retrieval combines dense vectors and full-text search, fuses rankings, reranks with Gemini, and streams results over SSE. Algorithm 1 summarises the Ask endpoint logic.

Algorithm 1 Hybrid RAG Ask pipeline (simplified)

1: Authenticate user and resolve `readableDepartmentIds`
2: Optimise query (type, topic, rewrite, optional multi-hop)
3: Retrieve dense vectors (pgvector) and sparse FTS (BM25-style)
4: Fuse rankings with weighted reciprocal rank fusion (RRF)
5: Rerank top chunks with Gemini; optionally expand neighbor chunks
6: Stream SSE events: `sources`, `token`, optional `correction`
7: Persist conversation message with citations and confidence
8: Optionally cache answer keyed by prompt version and filters

The Ask UI consumes SSE streams and renders markdown with sanitised links. Conversation threads, message persistence, and thumbs-up/down feedback complete the assistant experience. Search, Ask, and conversation flows (3.24–3.27, 3.28, 3.29) were validated by automated integration tests.

4.2.3.1 Implemented web interfaces

Ask interface.

The Ask workspace streams structured answers with confidence indicators and clickable source citations grounded in retrieved chunks (Figure 3.24, Figure 3.25).

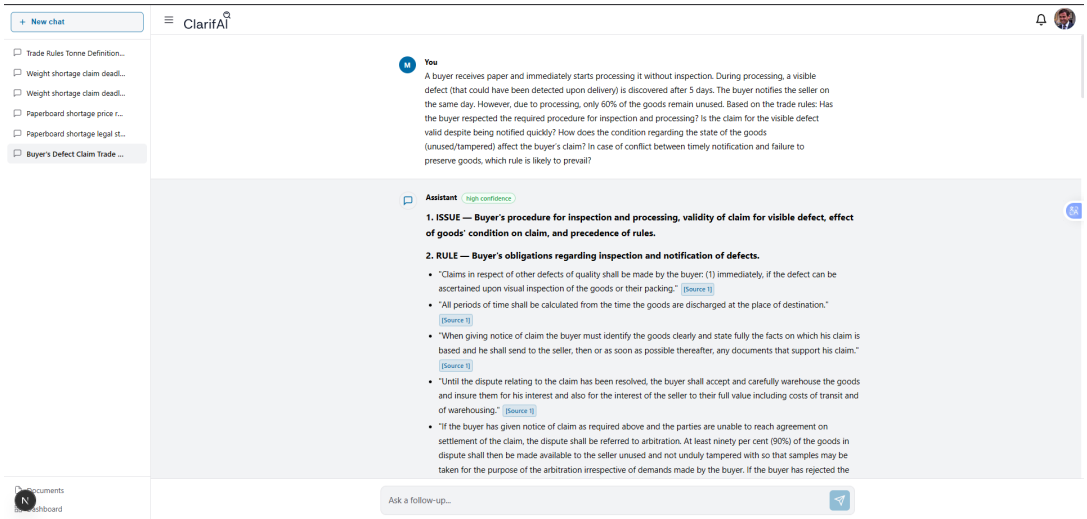

Figure 4.17 – Ask chat interface with structured answer, confidence indicator, and source citations

Answer feedback.

Each completed assistant message offers copy, thumbs-up, and thumbs-down controls; negative feedback can include a free-text comment for export and RAG evaluation (Figure 4.18, Figure 3.27).

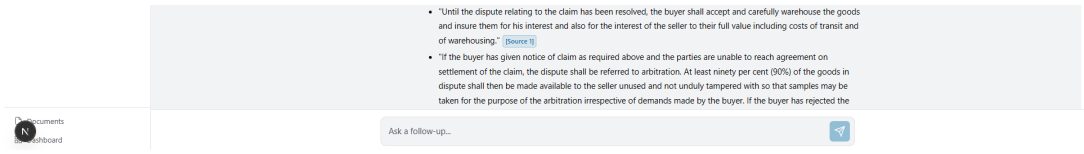

Figure 4.18 – Per-answer feedback controls with copy, thumbs up/down, and grounding badge

4.2.4 Sprint 4: Governance and notifications

Sprint 4 covered organisational governance. Administrators manage users, departments, and audit trails; managers oversee department-scoped activity; and all roles receive timely notifications. Department access logic computes readable and manageable scopes, including inherited manager and viewer rights. Administrators can bulk-restrict users, import CSV rosters with formula-injection escaping, merge departments, export KPIs, and review document audit logs. Managers use department dashboards; the notification service emits automatic events on document lifecycle changes and supports manual announcements with attachments. Document previews use blob URLs for PDFs and images; the inbox polls for unread badges. Admin, manager, and notification flows (3.32–3.38, 3.39–3.46) were confirmed by integration tests.

4.2.4.1 Implemented web interfaces

Governance consoles cover notifications, manager department overviews, administration modules, and audit dashboards. The screenshots below illustrate each area.

Notifications.

Users receive announcements in a slide-out inbox (Figure 4.19). Administrators and managers compose manual notifications through a modal form with recipient scope and optional attachments (Figure 4.20).

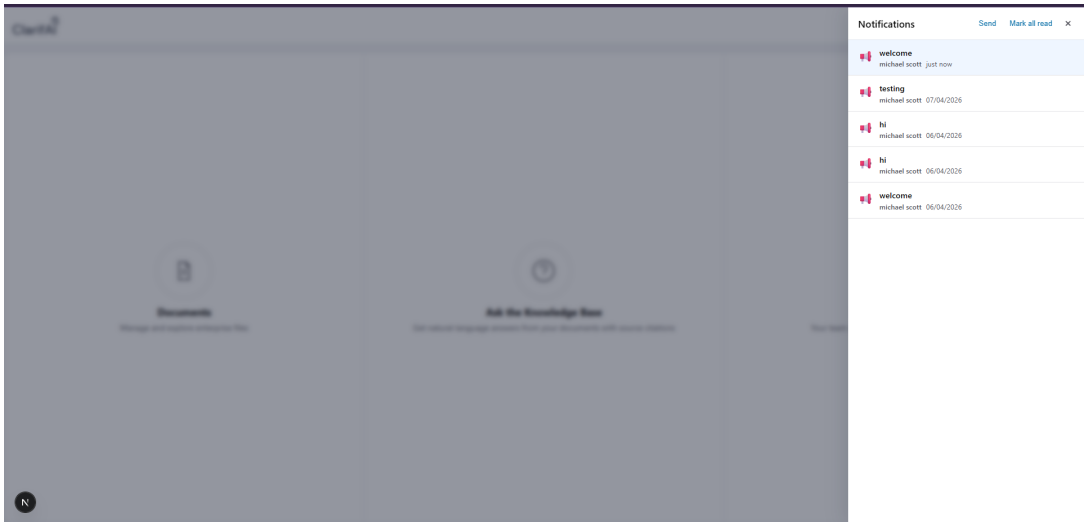

Figure 4.19 – Notifications inbox drawer listing unread announcements

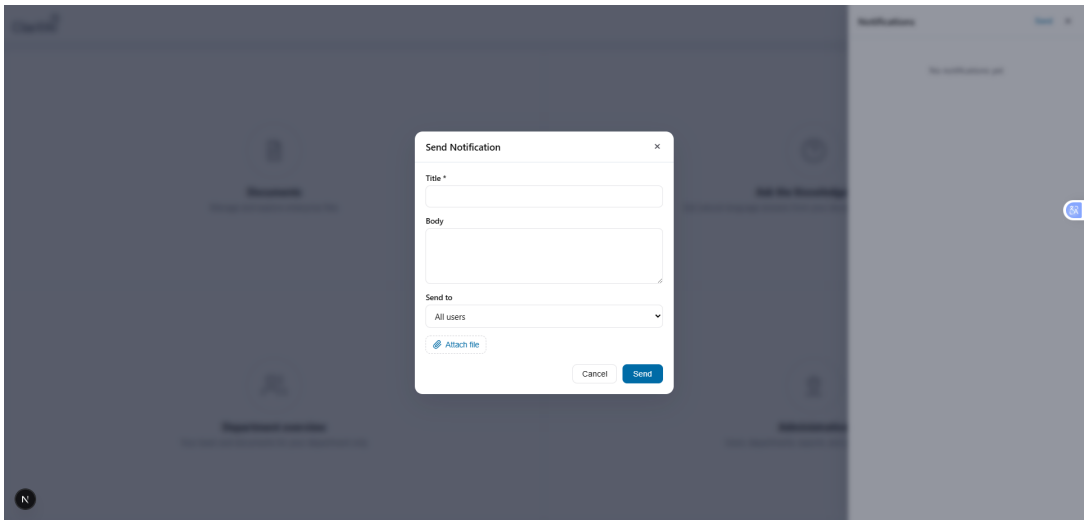

Figure 4.20 – Send notification modal with title, body, and recipient scope

Manager dashboard.

Figure 4.21 shows the department overview: team directory, department-scoped document list, and ingest status for manageable departments.

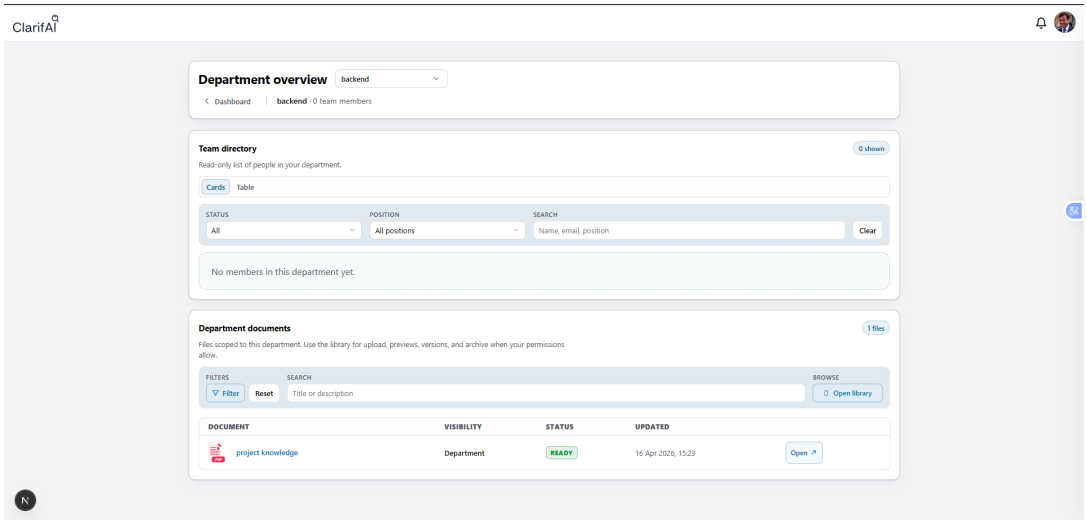

Figure 4.21 – Manager department overview with team directory and document list

Administration hub and directories.

The administration landing page (Figure 4.22) routes to six modules. Figure 4.23 shows the user directory with role, department, lock state, and bulk import. Figure 4.24 shows department cards with hierarchy and merge tool. Figure 4.25 shows the admin document library with processing filters and bulk delete.

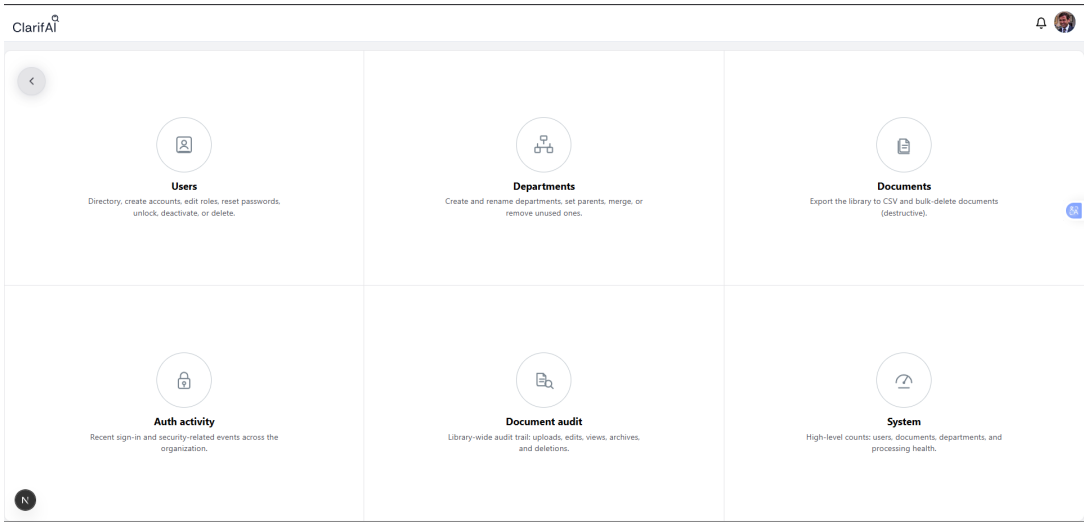

Figure 4.22 – Administration hub with Users, Departments, Documents, Auth activity, Document audit, and System modules

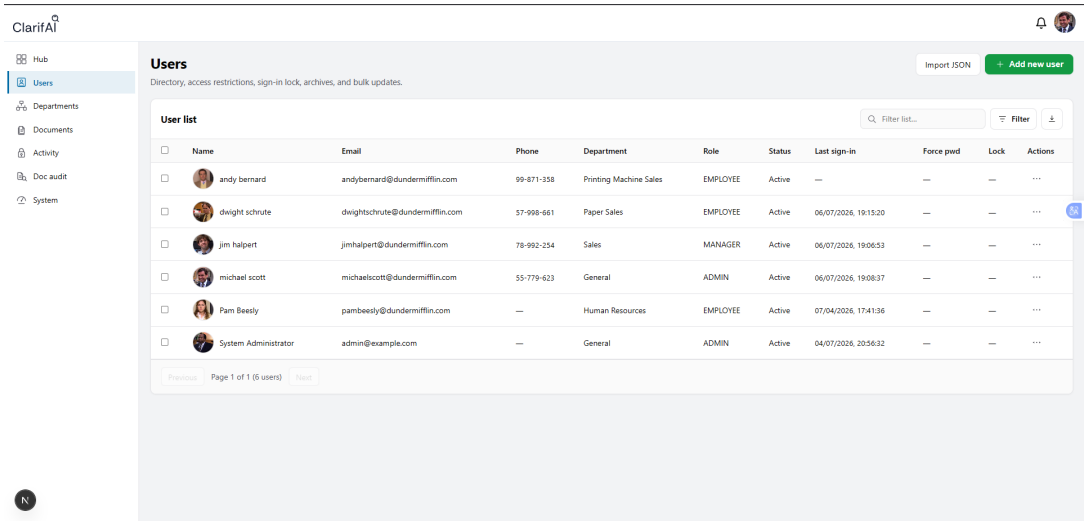

Figure 4.23 – Administrator user directory with roles, restrictions, and lock controls

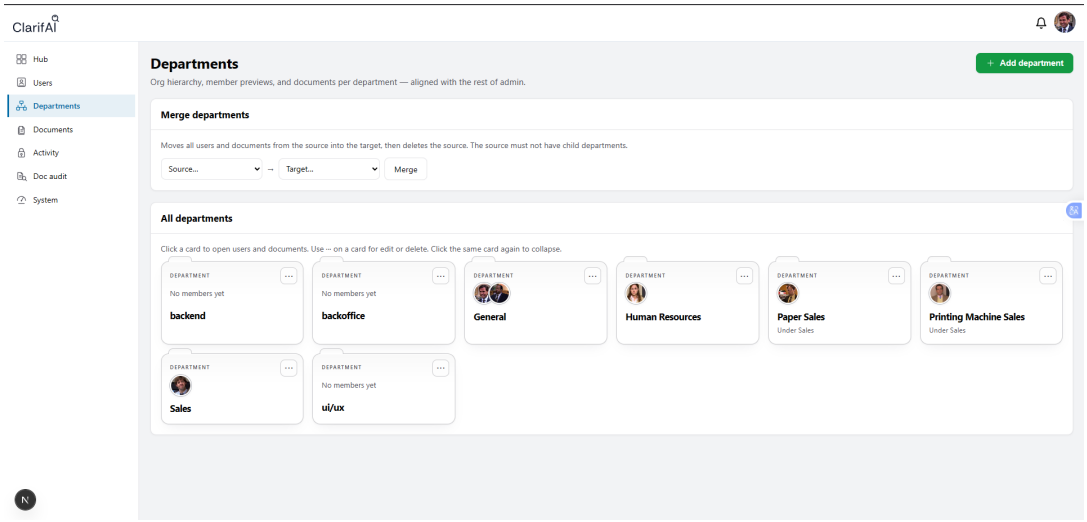

Figure 4.24 – Department management with hierarchy tree and merge tool

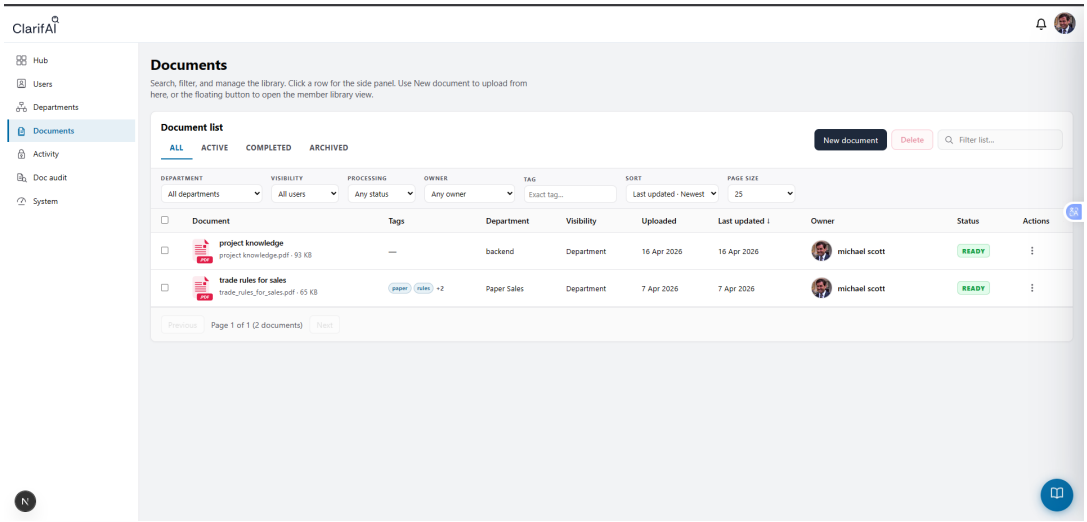

Figure 4.25 – Administrator document library with status filters and bulk operations

Audit trails and system KPIs.

Figure 4.26 shows the library-wide document audit log. Figure 4.27 shows authentication and security events with IP and client metadata. Figure 4.28 shows the system overview dashboard with period-over-period user, document, and department counts.

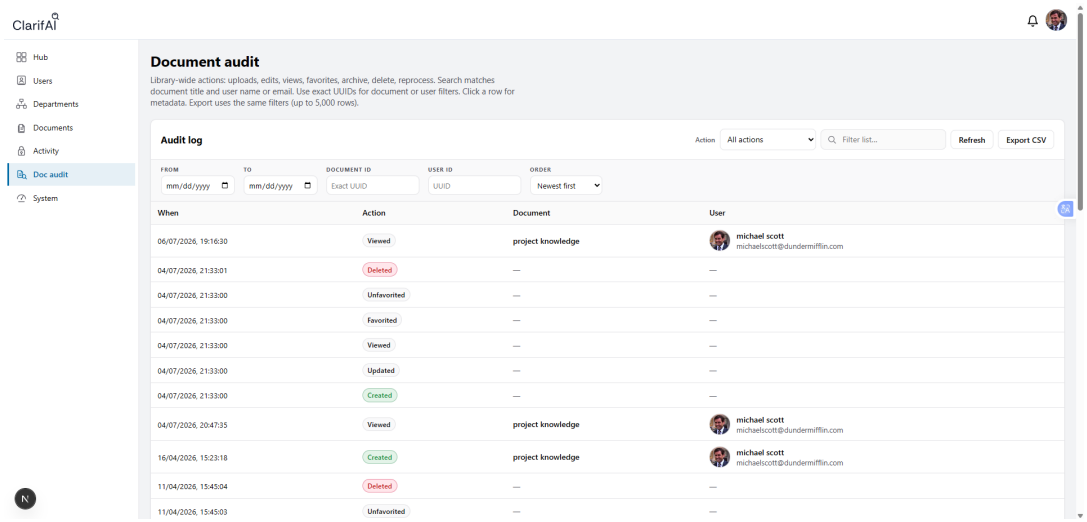

Figure 4.26 – Library-wide document audit log with CSV export action

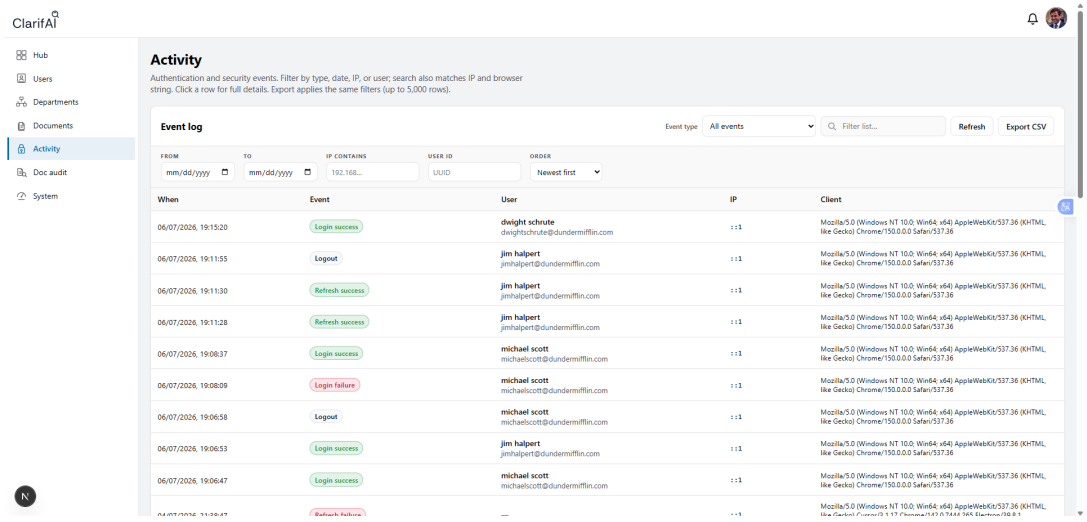

Figure 4.27 – Authentication and security event log with IP and client metadata

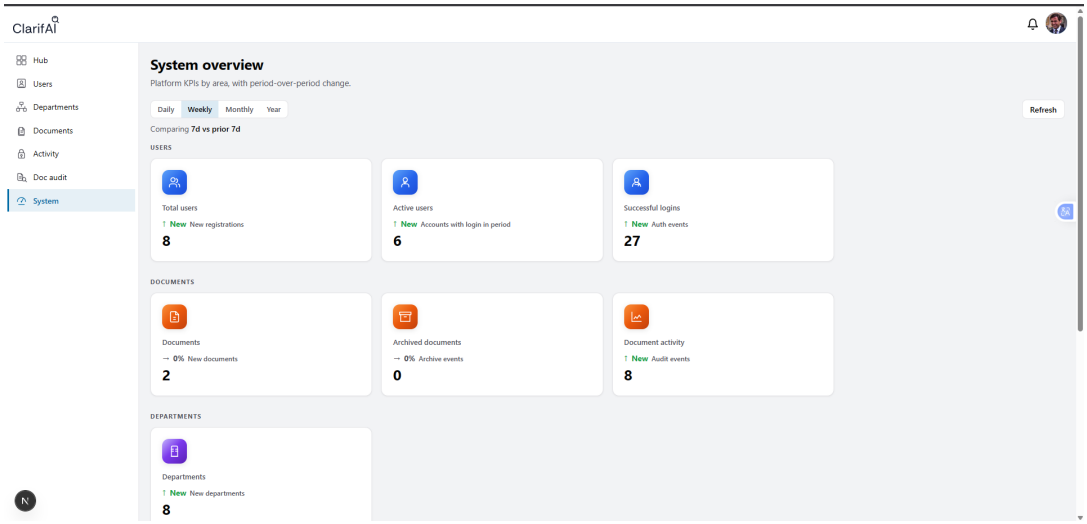

Figure 4.28 – System overview KPI dashboard with period-over-period comparison

4.2.5 Sprint 5: Operations, testing and deployment

Sprint 5 consolidated operational readiness. Health probes report PostgreSQL and Redis status; a 74-test integration suite exercises the API against real services; Docker Compose provides reproducible development and production stacks with automatic migrations; and a RAG evaluation CLI writes JSON reports for regression tracking. Operational readiness has no dedicated web UI; validation evidence is tabular in Chapter 5. Table 4.2 summarises all sprint sequence flows presented in Section 3.5.

Table 4.2 – Index of implementation sequence diagrams (Chapter 3, §3.5)

Sprint	Area	Fig.	Flow
S1	Auth	3.5–3.10	Login, logout, password reset/change, profile, restrictions
S2	Documents	3.13–3.16, 3.17–3.15	Upload, ingest worker, download, list, detail, favourites, metadata, versions, reprocess
S3	Search / RAG	3.24, 3.25	Semantic search, hybrid Ask SSE
S3	Conversations	3.26, 3.27, 3.28, 3.29	Threads, messages, feedback, CRUD supplement, admin stats
S4	Notifications	3.32–3.34	Automatic events, manual send, inbox
S4	Manager	3.35	Department dashboard
S4	Admin	3.36–3.38, 3.39–3.46	Users, departments, access, stats, audit, exports, bulk delete, lock
S5	Operations	3.49	Health check (tests, deploy, RAG eval: see §4.2.5)

4.3 Difficulties Encountered and Solutions

Table 4.3 – Implementation challenges and resolutions

Difficulty	**What went wrong**	**How it was resolved**
Avatar not displayed	Browsers blocked cross-origin profile images because the avatar route lacked a permissive CORP header.	The API now allows cross-origin embedding for public avatar responses.
Random logout on navigation	Parallel refresh requests rotated the refresh token twice, invalidating the session.	The web client serialises refresh into a single in-flight promise so only one rotation occurs.
Next.js dev webpack errors with avatars	The optimised image component correlated with RSC bundling issues in development.	Profile avatars use a standard HTML image element instead of the framework image wrapper.
Gemini rate limits	External API throttling caused intermittent failures under load.	Answers are cached with TTL; retries use exponential backoff.
Spreadsheet chunking	Tables split mid-row lost row semantics in retrieved chunks.	Chunking preserves table structure as markdown before embedding.

4.4 Produced Technical Deliverables

The project deliverables include:

— **Source code** — full-stack monorepo with API and web applications;

— **Database** — relational schema with migrations and seeded demonstration data;

— **Documentation** — README, architecture guide, and **54 PlantUML diagrams**

(sequence, use-case, and sprint class flows in Chapter 3, Section 3.5);

— **Tests** — ten integration test suites covering 74 scenarios;

— **Operations** — containerised deployment, RAG evaluation tooling, and maintenance scripts;

— **User interface** — dashboard, document library, Ask workspace, and administrative consoles (Figures 4.1–4.28, 4.17–4.18 in §4.2.1–4.2.4).

Representative screenshots of the running application are embedded per sprint above; authentication error and access-restriction validation evidence appears in Section 5.3.4.

4.5 Chapter Conclusion

This chapter described how the platform was built across five sprint increments, from authentication and document ingest through hybrid RAG, governance, and operational readiness. It recorded implementation constraints and the technical fixes applied, showing how architectural choices became operational code, testable workflows, and reproducible deliverables. Chapter 5 presents the validation results against the KPIs defined in Table 2.4.

CHAPTER 5

Validation

This chapter asks whether the implementation in Chapter 4 meets the functional, non-functional, and KPI targets set out in Chapter 2 (Table 2.4). The evidence comes from automated API tests, RAG evaluation harness runs, and manual browser scenarios. Section 5.4 discusses how much confidence each class of result actually warrants.

5.1 Validation Plan

Validation was organised at four levels. Unit tests cover pure helper functions. Integration tests exercise the API end-to-end against a live PostgreSQL and Redis stack. Browser scenarios on the Next.js client catch flows that are awkward to assert at the HTTP layer alone. A dedicated RAG evaluation harness then measures retrieval and answer quality with automated judge metrics when an API key is available.

Relative to the solutions surveyed in Chapter 1, the checks target capabilities that generic document stores or chatbots usually omit: **department-scoped hybrid retrieval**, **cited streaming answers**, **per-user feature restrictions**, and **administrative audit trails**. Unit coverage is kept deliberately narrow. The main automated quality gate is the **74-test integration suite**, with manual UX checks filling in security and restriction flows.

5.2 Protocols and Experimental Conditions

Table 5.1 summarises the experimental environment. Integration tests run against a seeded database with Docker-hosted PostgreSQL and Redis. RAG evaluation uses a quick mode (optimiser-only, 34 cases) and a full judge pipeline when a Gemini API key is configured. Manual scenarios were run in the browser on the local development server; they appear in Table 5.4, with screenshot evidence in Section 5.3.4.

Table 5.1 – Experimental setup for validation

Parameter	**Value**
Workstation	Windows 10/11; Node.js 20+
Test framework	Vitest 3.2 + Supertest (`npm run test:api`)
Database / queue	PostgreSQL 16 + pgvector; Redis 7 (Docker Compose)
Test corpus	Seeded users, departments, and documents (`npm run db:seed`)
RAG evaluation	`eval:rag:quick` (34 topic cases); full judge if API key configured
Automated run date	4 July 2026 (74/74 passed, duration ≈39 s)
Manual UX	Browser on `localhost:3000`; scenarios T1–T5 (Table 5.4)

5.3 Presentation of Results

5.3.1 KPI Validation Results

Table 5.2 maps each KPI from Chapter 2 to its measured outcome and validation conclusion.

Table 5.2 – KPI validation results

KPI	**Measurement method**	**Target**	**Obtained**	**Status**
Ingest reliability	Uploads reaching ready state without failure	$\geq 95\%$	100% (14 doc. tests)	Passed
Integration pass rate	Full API integration suite	74 / 74	**74 / 74**	Passed
RAG topic (no API key)	Quick eval optimiser pass rate	$\geq 15/34$	**15 / 34**	Passed
RAG judge (with API key)	Automated faithfulness / relevance judge	Documented	Pending full run	Partial
Security: refresh replay	Rotated token revokes family	Auth suite	Pass (11 tests)	Passed
Security: avatar CORP	Cross-origin avatar embedding permitted	Avatars test	Pass (1 test)	Passed

Analysis. Five of the six KPI rows passed fully. On the seeded corpus, ingest reliability clears the 95% bar. The integration suite gives solid evidence that API contracts match the design sequences in Chapter 3. RAG topic classification hits the degraded-mode threshold (15/34) without an external API key; full judge scores stay **partial** until a complete run with credentials is available.

5.3.2 Integration Test Results by Sprint Domain

On 4 July 2026 the automated API suite reported **74 tests passed** across 10 files with no failures. Table 5.3 groups the results by sprint domain, in line with Table 2.2.

Table 5.3 – Validation test summary by sprint domain

Spr.	**Objective**	**Expected**	**Obtained**	**Status**
S1	Secure login and token refresh	200 + new token	Pass (11 tests)	Passed
S2	Document access control	403/404 when restricted	Pass (14 tests)	Passed
S3	Search and Ask restrictions	AI flag enforced	Pass (7 tests)	Passed
S4	Admin RBAC	Non-admin rejected	Pass (18 tests)	Passed
S4	Notifications lifecycle	CRUD + unread count	Pass (8 tests)	Passed
S1/S5	Avatar CORP + health probe	Headers + DB/Redis ok	Pass (4 tests)	Passed

Analysis. Each sprint increment in Table 2.5 has at least one passing integration-test group, so the chain from requirements through design to executable verification stays intact. Administration (S4, 18 tests) and document access (S2, 14 tests) form the largest groups, which matches how heavy RBAC and visibility rules are in practice. Figure 5.1 shows the Vitest summary from the 7 July 2026 re-run (74/74, duration ≈41 s).

```
npm run test:api – Vitest integration suite

> test:api
> npm run test -w @knowledge-platform/api

> @knowledge-platform/api@0.0.1 test
> npx vitest run

 RUN  v3.2.4 C:/dev/finalproject/apps/api

 ✓ src/auth.integration.test.ts (11 tests) 7424ms
 ✓ src/admin.integration.test.ts (18 tests) 2449ms
 ✓ src/documents.integration.test.ts (14 tests) 1850ms
 ✓ src/search.integration.test.ts (7 tests) 1485ms
 ✓ src/manager.integration.test.ts (4 tests) 1453ms
 ✓ src/conversations.integration.test.ts (7 tests) 847ms
 ✓ src/notifications.integration.test.ts (8 tests) 904ms
 ✓ src/avatars.integration.test.ts (1 test) 433ms
 ✓ src/health.integration.test.ts (3 tests) 63ms
 ✓ src/lib/roleNameContract.test.ts (1 test) 2ms

 Test Files  10 passed (10)
      Tests  74 passed (74)
   Start at  16:04:15
   Duration  41.20s
```

Figure 5.1 – Vitest integration suite output showing 74 tests passed across 10 files

5.3.3 RAG Evaluation Results

Quick evaluation without a configured Gemini API key classified all optimiser topics as `general`, giving **15/34 PASS** on topic-label cases. That drop is expected when the LLM optimiser cannot run. With API credentials, the full harness writes JSON reports under `apps/api/eval-reports/` for faithfulness and citation metrics.

Interpretation. Retrieval and serving paths can be checked without the external API; semantic query optimisation and judged answer quality cannot. The 15/34 figure is best treated as a **minimum regression gate**, not as a score of production answer

quality. Figure 5.2 records the quick-mode harness output on 7 July 2026.

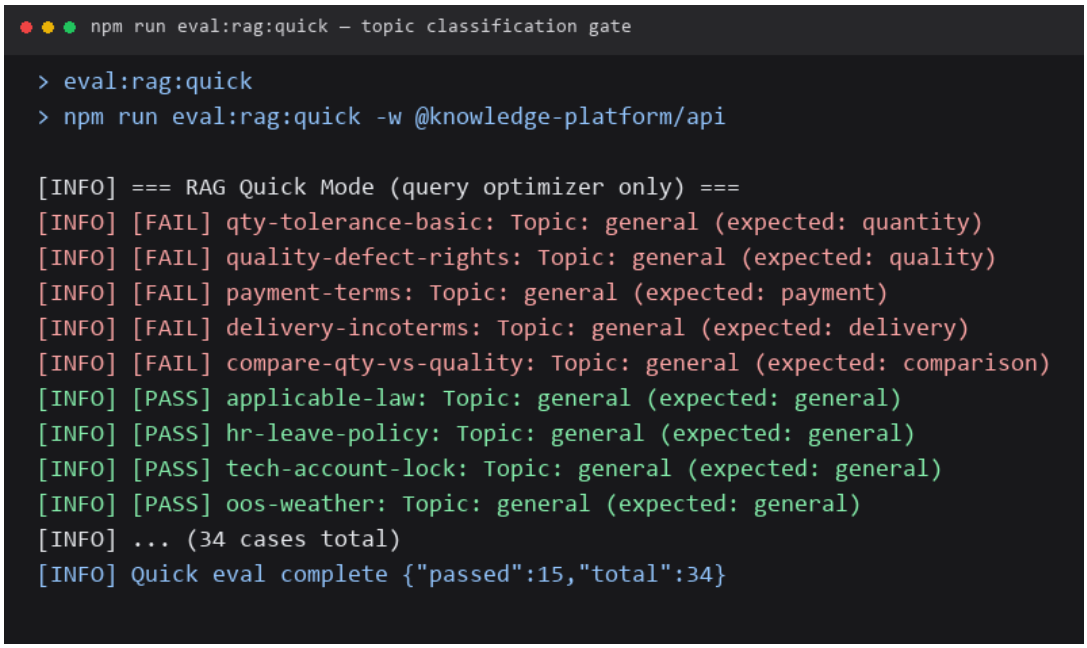

Figure 5.2 – RAG quick evaluation output showing 15 of 34 topic cases passed without Gemini API key

5.3.4 Manual UX Validation

Table 5.4 summarises the five manual browser scenarios. The figures below supply visual evidence; matching implementation notes sit in Chapter 4 with complementary captions.

Table 5.4 – Manual UX validation summary

Test	**Objective**	**Expected Outcome**	**Observed Outcome**	**Status**
T1	Reject invalid credentials without revealing account existence	Generic error message, no enumeration	Fig. 5.3	Passed
T2	Explain when document library access is disabled	Dedicated restricted page	Fig. 5.4	Passed
T3	Confirm ingest completes after upload	Processing status shown as complete on detail panel	Fig. 5.5	Passed
T4	Verify hybrid Ask returns cited answers	Answer with source citations	Fig. 5.6	Passed
T5	Show manager department overview	Dept-scoped team and documents	Fig. 5.7	Passed

Authentication failure (T1).

Invalid credentials produce a clear client-side error without revealing whether the email exists, consistent with the auth integration suite (Table 5.3).

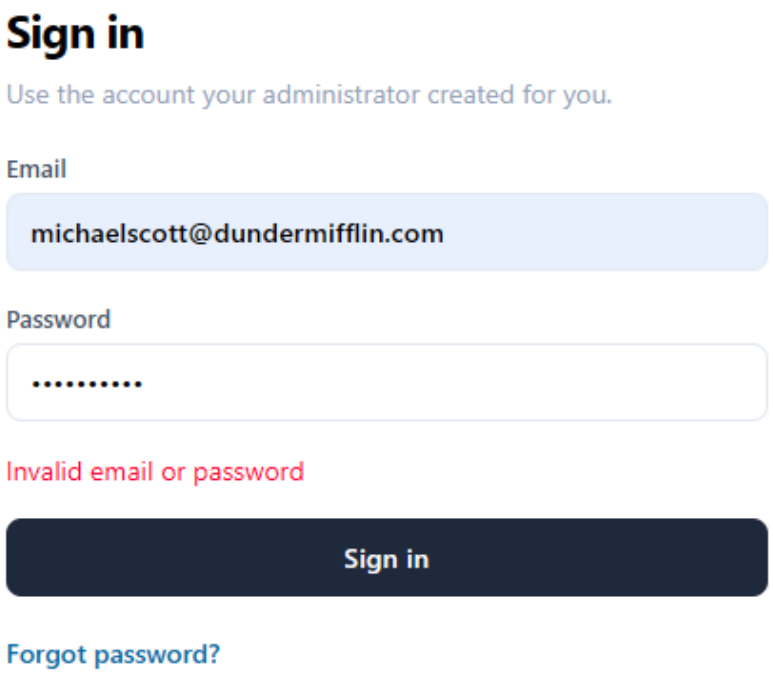

Figure 5.3 – Invalid email or password message after failed sign-in attempt

Feature restriction (T2).

When document library access is disabled for a user, the API returns a feature restriction response and the client redirects to a dedicated explanation page (Figure 3.10, Chapter 3).

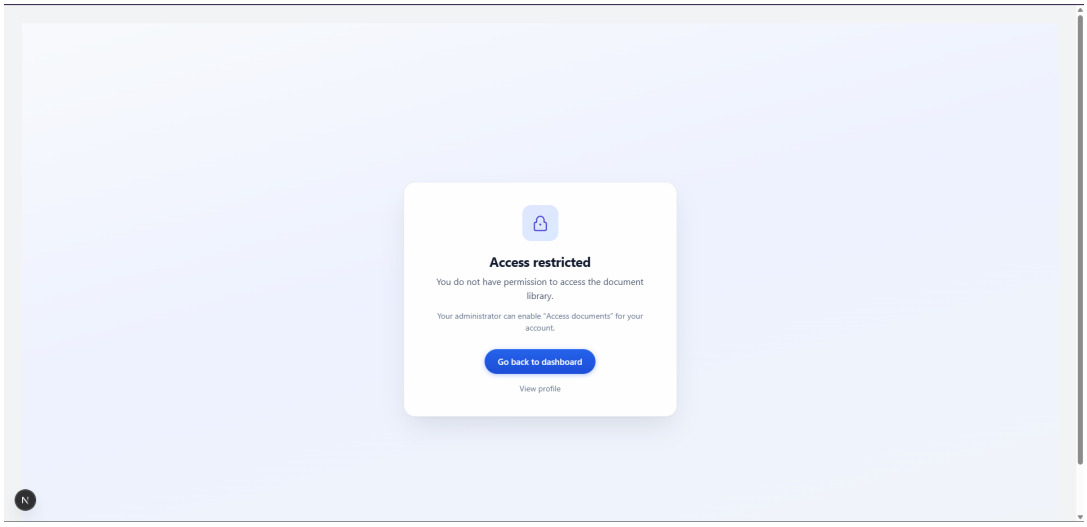

Figure 5.4 – Dedicated page explaining that document library access is disabled

Ingest completion (T3).

After upload and asynchronous processing, the file detail panel shows processing status **READY**, confirming the ingest worker completed embedding and chunk persistence.

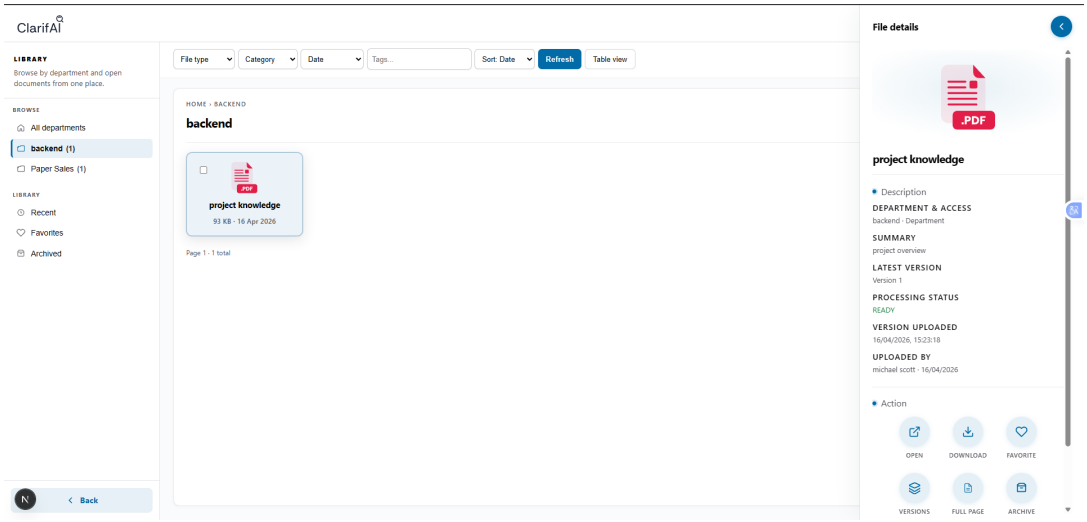

Figure 5.5 – Document detail panel showing READY ingest status

Hybrid Ask with citations (T4).

The Ask interface returns a structured answer with confidence indicator and clickable source citations grounded in retrieved chunks.

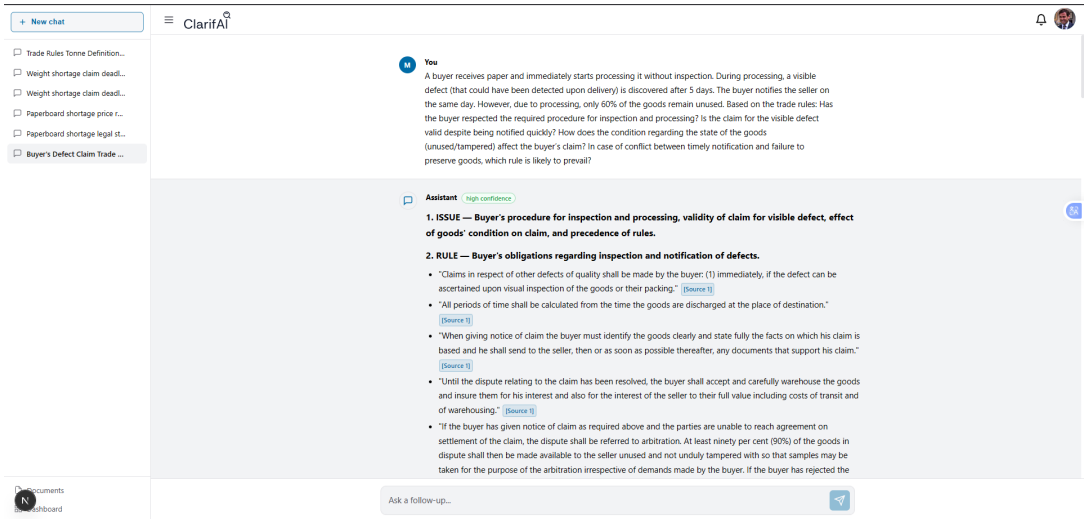

Figure 5.6 – Ask response with structured answer and source citations

Manager department overview (T5).

Managers view a read-only team directory and department-scoped document list for manageable departments.

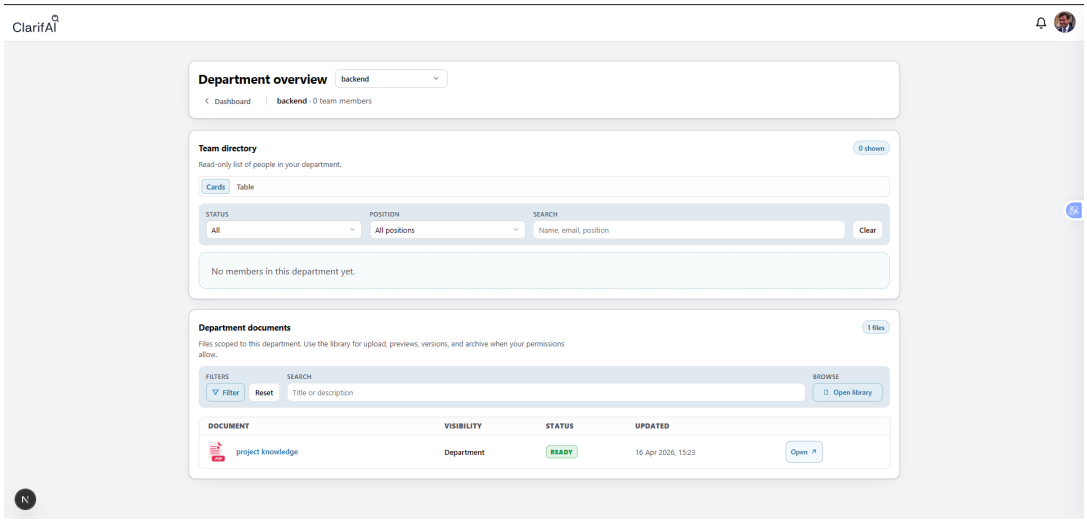

Figure 5.7 – Manager department overview with team directory and documents

5.4 Critical Discussion

5.4.1 Strengths

End-to-end coherence is the clearest strength of the platform. Requirements remain traceable, the architecture stays modular, the automated suite covers 74/74 cases, and the documented RAG pipeline goes well past a thin chat demo: hybrid retrieval, reranking, SSE, feedback, and an evaluation harness. Manual UX checks also show that security and restriction flows are intelligible to users, not merely correct at the API layer.

5.4.2 Deviations from Targets

A few targets were only partly met. The RAG judge KPI still lacks a full faithfulness and relevance run, so it remains **Partial** in Table 5.2. No formal load test was run for concurrent Ask streams, leaving throughput unvalidated. The topic evaluation at 15/34 clears the degraded threshold, yet it does not show production-grade query optimisation

when API credentials are absent.

5.4.3 Limitations

Several limits follow from the design choices. Latency, cost, and availability all hinge on Google Gemini as the external LLM. Default storage is local disk rather than cloud-native object storage. The automated judge is itself an LLM, so human evaluation still has a place. Answer quality also climbs or falls with how complete the ingest corpus is and how well chunking performs.

5.4.4 Threats to Validity and Confidence Level

Integration tests use a seeded database smaller than a production corpus. RAG eval cases are code-defined and may miss some host-company domains. Manual UX tests were run on a developer workstation.

Confidence assessment. Confidence is high for API contracts, RBAC, document access, and notification lifecycles, backed by the 74/74 integration suite. It is medium for RAG answer quality and citation faithfulness while the eval harness remains partial without a full judge run. Confidence stays low for concurrent load, production-scale corpus behaviour, and long-term operational monitoring.

5.4.5 Perspectives

Natural next steps include SSO or LDAP integration, S3-compatible storage, on-prem embedding models, and horizontal worker scaling. Multilingual UI, CI-integrated RAG regression once API keys are available, and field deployment with user acceptance testing on real organisational corpora would round out the present work.

5.5 Chapter Conclusion

This chapter checked the Knowledge Platform against the KPIs and sprint domains from Chapter 2. Automated integration tests passed without failure (74/74), ingest reliability cleared the 95% target on the seeded corpus, and five manual UX scenarios passed with

screenshot evidence. The RAG judge KPI is still partial until a full evaluation run with API credentials is completed, and scale testing was not performed. The critical discussion places high confidence in API-level correctness, medium confidence in RAG quality metrics, and notes limits tied to the external LLM and the size of the test corpus. The general conclusion weighs these outcomes against the objectives stated in the introduction.

General conclusion and perspectives

This final-year project tackled a recurring organisational difficulty: internal knowledge exists, but staff find it hard to locate, hard to govern, and hard to trust in daily work. Conducted at HyprCraft Solutions, the internship produced the Knowledge Platform, a system that brings document management, controlled access, semantic search, and cited AI-assisted answers into one coherent experience.

In practice, the platform offers a secure multi-role environment where documents can be uploaded, indexed, searched, and discussed in persistent conversations. Retrieval mixes semantic and keyword methods so that answers stay tied to organisational sources. Administration and management functions cover user provisioning, departments, notifications, and activity review. Automated tests confirmed core behaviour across authentication, documents, search, conversations, and governance, while manual scenarios exercised sign-in, document readiness, hybrid Ask, and profile management.

Against the objectives set out in the introduction, the main goals were met: a role-aware secure platform, better retrieval over heterogeneous documents, intelligent assistance with source transparency, structured validation, and a repeatable deployment model. Open limitations remain. The stack still depends on an external language-model provider, large-scale performance benchmarking was not the focus of this internship, and evaluation quality still depends on how rich and reliable the corpus is.

Beyond the technical deliverable, the work strengthened skills in requirements engineering, system design, information retrieval, AI integration, security awareness, and the critical evaluation of intelligent systems. Those competencies matter directly for enterprise software and data-oriented engineering.

Looking ahead, useful extensions include cloud object storage, enterprise single sign-on, on-premise model options, tighter continuous evaluation of answer quality, mobile access,

and a full deployment with real corpus onboarding and formal user acceptance testing at the host organisation.

Bibliography

[1] Y. Gao, Y. Xiong, X. Gao, K. Liu, J. Liu, H. Wang, and J. Zhang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2024, accessed: July 2026.

[2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781, 2020.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.

[5] O. Ram, Y. Levine, I. Dalmedigos, D. Jury, O. Kabeli, A. Shashua, K. Leyton-Brown, and Y. Shoham, "In-context retrieval-augmented language models," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023.

[6] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Nogueira, and J. Lin, "Pyserini: A python toolkit for reproducible information retrieval research with sparse and dense representations," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 2356–2362.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural*

Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[8] S. Es, J. James, F. Anton, L. Ott, M. Velizhev, and M. Buchwald, "RAGAS: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2024, accessed: July 2026.

[9] R. S. Sandhu, E. J. Coyne, H. J. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[10] D. F. Ferraiolo, D. R. Kuhn, and R. Chandramouli, "Role-based access control," *Artech House Computer Security Series*, 2003.

[11] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to attribute based access control (abac) definition and considerations," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-162, 2014.

[12] Microsoft Corporation, "Sharepoint documentation," 2025, [Online]. Accessed: July 2026. [Online]. Available: https://learn.microsoft.com/en-us/sharepoint/

[13] Pinecone Systems Inc., "Pinecone vector database documentation," 2025, [Online]. Accessed: July 2026. [Online]. Available: https://docs.pinecone.io/

[14] LangChain Inc., "Langchain documentation," 2025, [Online]. Accessed: July 2026. [Online]. Available: https://python.langchain.com/docs/

[15] LlamaIndex Inc., "Llamaindex documentation," 2025, [Online]. Accessed: July 2026. [Online]. Available: https://docs.llamaindex.ai/

[16] Microsoft Corporation, "Microsoft copilot for microsoft 365," 2025, [Online]. Accessed: July 2026. [Online]. Available: https://www.microsoft.com/en-us/microsoft-365/copilot

[17] Guru Technologies, Inc., "Guru: Enterprise ai search, intranet and wiki — product documentation," 2025, [Online]. Accessed: July 2026. [Online]. Available: https://help.getguru.com/

[18] Tettra, Inc., "Tettra: Ai-powered knowledge base and internal wiki — help center," 2025, [Online]. Accessed: July 2026. [Online]. Available: https://help.tettra.com/

[19] Glean Technologies, Inc., "Glean: Work ai platform for enterprise search and assistants," 2025, [Online]. Accessed: July 2026. [Online]. Available: https://www.glean.com/product/overview

[20] ——, “Glean developer and platform documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: https://developers.glean.com/

[21] Vercel Inc., “Next.js documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: https://nextjs.org/docs

[22] OpenJS Foundation, “Express.js web framework documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: https://expressjs.com/

[23] Prisma Data Inc., “Prisma orm documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: https://www.prisma.io/docs

[24] pgvector Contributors, “pgvector: Open-source vector similarity search for postgresql,” 2024, [Online]. Accessed: July 2026. [Online]. Available: https://github.com/pgvector/pgvector

[25] Google DeepMind, “Gemini api documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs

[26] Redis Ltd., “Redis documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: https://redis.io/docs/

[27] Taskforce.sh Inc., “Bullmq documentation,” 2025, [Online]. Accessed: July 2026. [Online]. Available: https://docs.bullmq.io/

APPENDIX A

Validation Summary

Table A.1 – Requirements Traceability Matrix

Req.	Design choice	Implemented element	Validation method	Status
FR-01	JWT authentication	Authentication service	Automated auth tests	Validated
FR-02	Session refresh and logout	Authentication service	Automated auth tests	Validated
FR-03	Profile field updates	Profile management service	Automated profile tests	Validated
FR-04	Avatar upload and CORP delivery	Public avatar endpoint	Automated profile tests	Validated
FR-05	Password change and reset	Authentication service	Automated auth tests	Validated
FR-06	Per-user feature restriction flags	Authorization middleware	Auth and search tests	Validated
FR-07	Multi-format upload and async ingest	Ingestion pipeline	Automated document tests	Validated

Req.	Design choice	Implemented element	Validation method	Status
FR-08	Department visibility rules	Access control layer	Automated document tests	Validated
FR-09	Tags, favourites, archive	Document library service	Automated document tests	Validated
FR-10	Document version history	Document library service	Automated document tests	Validated
FR-11	Reprocess existing versions	Ingestion pipeline	Automated document tests	Validated
FR-12	Detail view, preview, download	Document library service	Automated document tests	Validated
FR-13	Per-document activity audit	Document audit log	Automated document tests	Validated
FR-14	Metadata search and filtering	Document library service	Automated document tests	Validated
FR-15	Platform-wide document administration	Administration module	Automated admin tests	Validated
FR-16	Document catalogue export	Administration module	Automated admin tests	Validated
FR-17	Semantic vector search	Search service	Automated search tests	Validated
FR-18	Hybrid cited Ask answers	RAG completion service	Search and manual Ask tests	Validated
FR-19	Conversation thread CRUD	Conversation service	Automated conversation tests	Validated
FR-20	Thumbs up/down answer feedback	Conversation service	Automated conversation tests	Validated

Req.	Design choice	Implemented element	Validation method	Status
FR-21	Aggregated feedback analytics	Administration module	Automated admin tests	Validated
FR-22	Event-driven automatic notifications	Notification service	Automated notification tests	Validated
FR-23	Manual announcements	Notification service	Automated notification tests	Validated
FR-24	Notification file attachments	Notification service	Automated notification tests	Validated
FR-25	Personal notification inbox	Notification service	Automated notification tests	Validated
FR-26	Manager department overview	Manager module	Automated manager tests	Validated
FR-27	User account lifecycle	Administration module	Automated admin tests	Validated
FR-28	Department CRUD	Administration module	Automated admin tests	Validated
FR-29	Department merge	Administration module	Automated admin tests	Validated
FR-30	Department access grants	Administration module	Automated admin tests	Validated
FR-31	Platform KPI statistics	Administration module	Automated admin tests	Validated
FR-32	Auth and document audit exports	Administration module	Automated admin tests	Validated
FR-33	Service health monitoring	Health endpoint	Automated health tests	Validated

Req.	Design choice	Implemented element	Validation method	Status
FR-34	Bulk user operations	Administration module	Automated admin tests	Validated
FR-35	Bulk document operations	Administration module	Automated admin tests	Validated
FR-36	Administrative import/export	Administration module	Automated admin tests	Validated
NFR-01	Service health monitoring	Health endpoint	Automated health tests	Validated
NFR-02	HTTP security hardening	Application middleware	Integration tests and review	Validated
NFR-03	Safe session refresh	Client refresh strategy	Automated auth tests	Validated
NFR-04	Reproducible deployment	Container packaging	Manual deployment check	Validated
NFR-05	Automated regression suite	Integration test suite	Local and CI execution	Validated
NFR-06	RAG quality evaluation	Evaluation harness	Quick eval workflow	Partial

Table A.2 – Technology Architecture Comparison Matrix

Criterion	Mono-repo	Django	Micro-services	Rationale
Time to market	3	2	1	Shared TypeScript types; one repository
SSE streaming fit	3	2	2	Native Express streaming for RAG
Team skill alignment	3	2	1	Matches React + Node stack
Operational complexity	3	3	1	Single API process + worker

Table A.3 – Functional Validation Test Summary

T	Objective	Expected	Observed	Status
T1	Reject invalid credentials safely	Generic error, no enumeration	Clear sign-in error	Passed
T2	Explain disabled library access	Dedicated restricted page	Restricted page shown	Passed
T3	Confirm ingest completes	Status shown as complete	Ready on detail panel	Passed
T4	Verify cited Ask answers	Answer with citations	Answer with citations	Passed
T5	Show manager overview	Dept. team and documents	Team and files listed	Passed

APPENDIX B

Scientific Integrity and Reproducibility

B.1 Declaration of AI Tool Usage

AI writing and coding assistants were used sparingly during the project, mainly to speed up drafting, clarify wording, and offer general programming hints. Design choices, implementation, measured results, and conclusions were produced and checked by the author, who remains fully responsible for the submitted work.

B.2 Reproducibility Details

The following parameters define the minimum environment needed to reproduce the reported results. Commands may be run from the repository root unless noted.

Table B.1 – Reproducibility parameters

Parameter	**Value**
Repository	Monorepo with API and web applications (`apps/api`, `apps/web`)
Database	PostgreSQL 16 + pgvector; migrations via `npm run db:migrate`; seed via `npm run db:seed`
Services	`npm run docker:up` for PostgreSQL and Redis
Environment	API `.env` (JWT secret, Gemini API key, database URL); Web `.env.local` (public API URL)
Run	`npm run dev` from repository root (API port 3001, web port 3000)
Tests	`npm run test:api`; RAG eval: `npm run eval:rag:quick` or `npm run eval:rag`
Prompt versioning	`RAG_PROMPT_VERSION` environment variable invalidates answer cache

B.3 Integrity Checklist

— ✓ Front matter complete (cover, dedication, acknowledgements, abstract, table of contents, lists, abbreviations).

— ✓ Introduction announces problem statement, objectives, and report plan.

— ✓ Each chapter follows need → design → implementation → validation logic.

— ✓ Conclusion explicitly answers the stated objectives.

— ✓ Design choices justified with comparison tables (Chapters 1–3).

— ✓ Validation protocol documented with 74 integration tests (Chapter 5).

— ✓ Limitations and threats to validity discussed.

— ✓ Every borrowed figure, table, or idea is cited in the bibliography.

— ✓ Results are interpreted critically (limitations in Chapter 5).

— ✓ Figures and tables numbered and referenced in text.

— ✓ Generative AI usage declared in Section B.1.

— ● Similarity checking target: $< 15\%$ (institutional plagiarism tool).

— ● Run institutional similarity check before final submission.